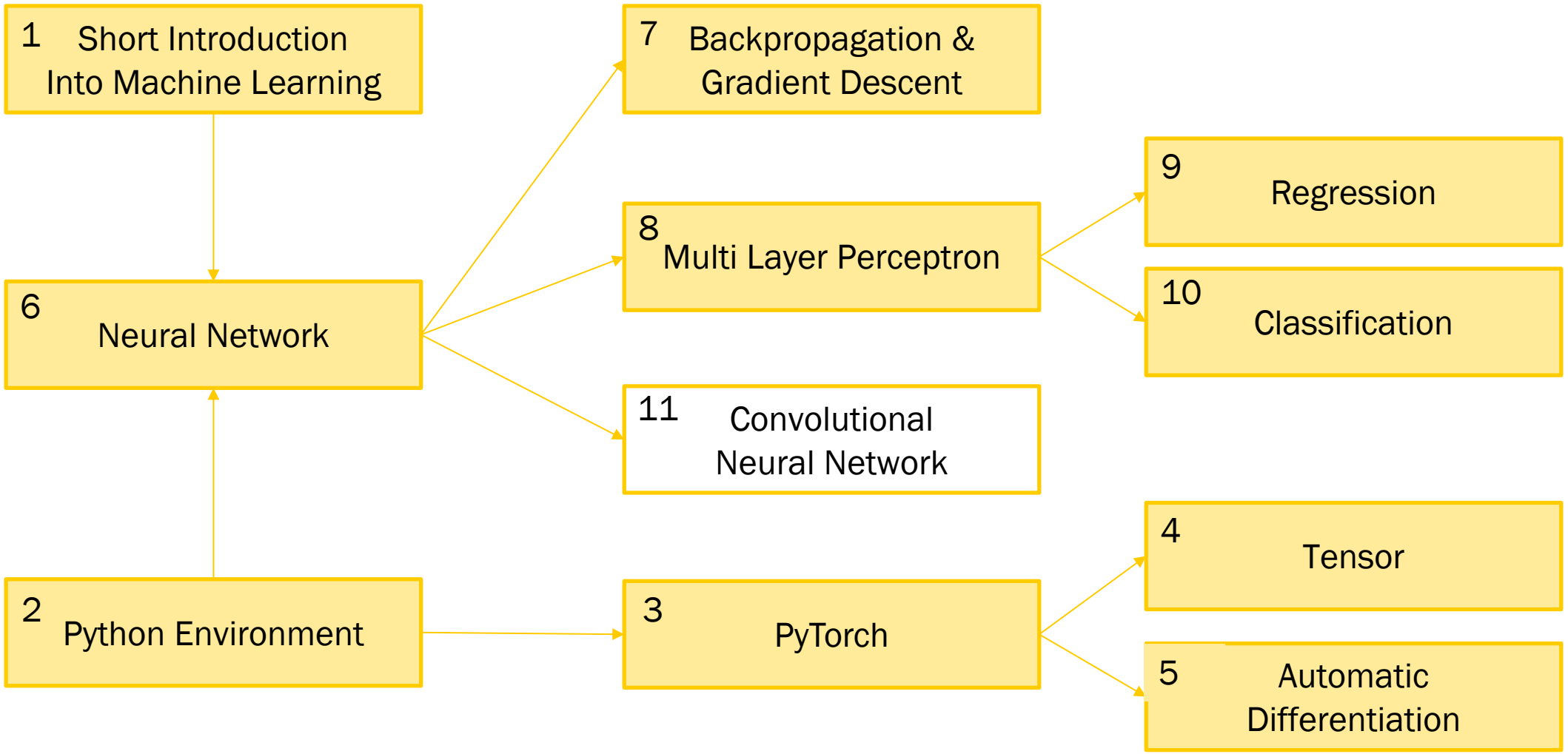
INF Informatik
Hochschule Reutlingen

Intelligente Informationssysteme
2 – Large Language Models

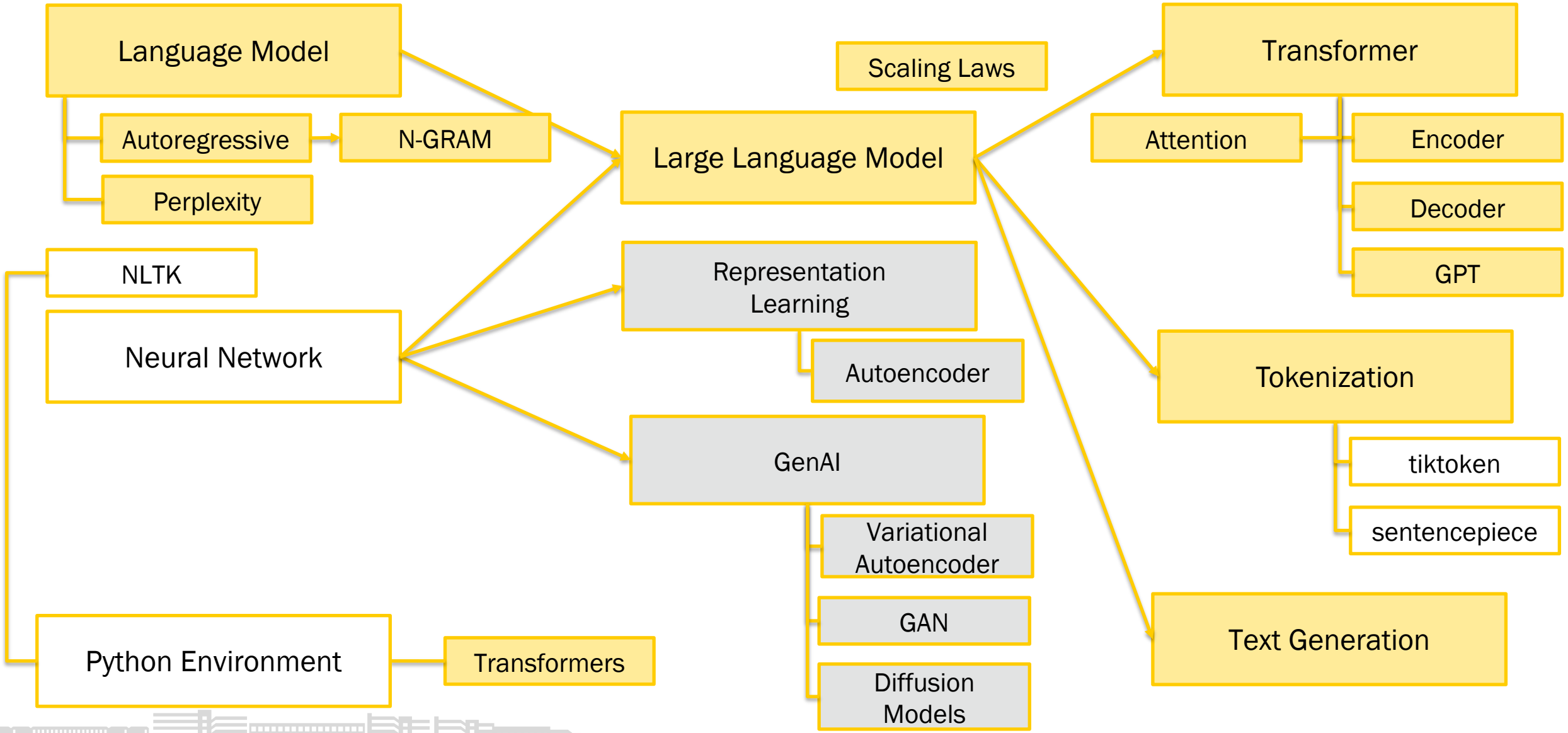
Dominik Neumann

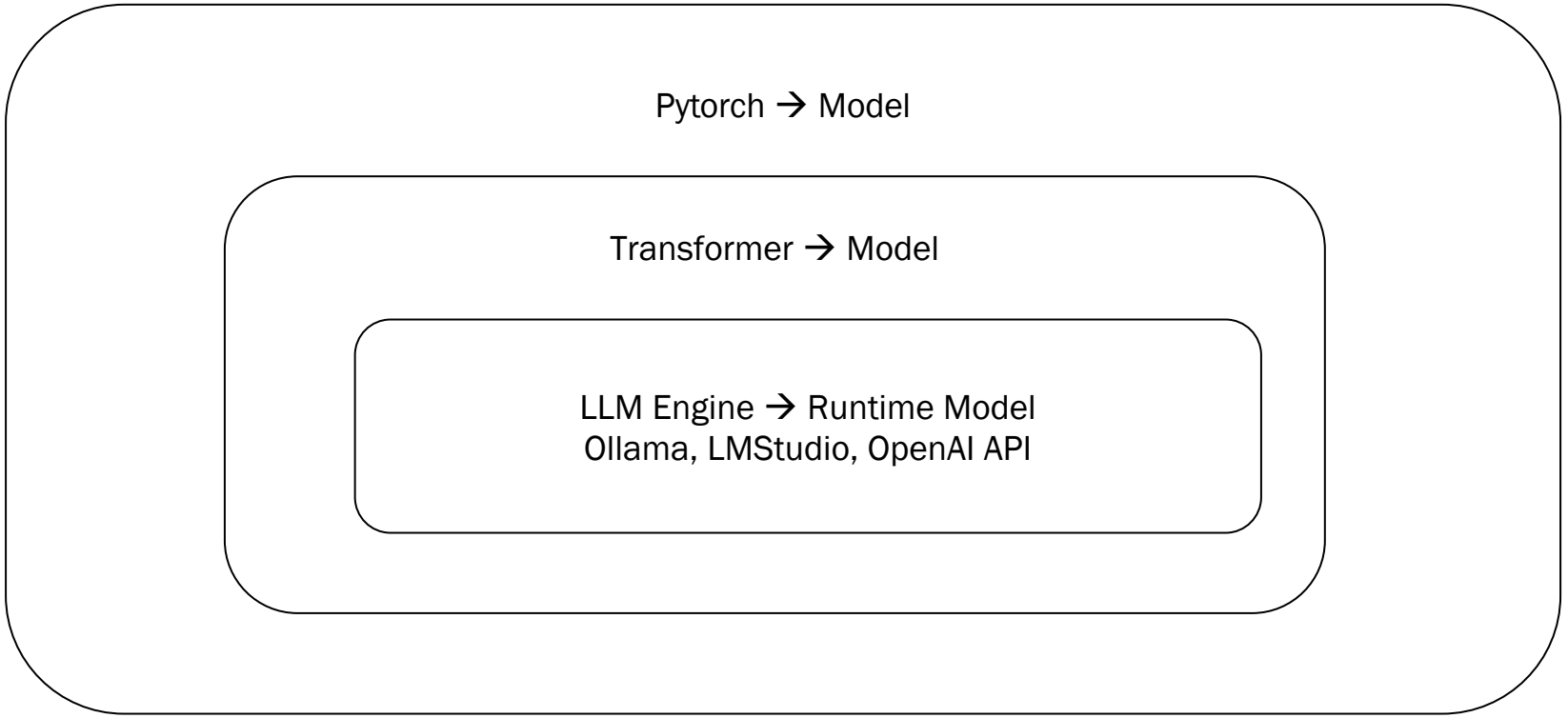
Bock 1 – Neural Network

RECAP



Bock 2 – Generative AI & Transformer




Pytorch → Model
Transformer → Model
LLM Engine → Runtime Model
Ollama, LMStudio, OpenAI API

02.01
Language Model

INF
Informatik
Hochschule Reutlingen

Language Models - Agenda

Hochschule Reutlingen

Language Model

2. What is a **model**?
1. Why is **language** so important in Artificial Intelligence?

3. What is a Language Model?
4. How to build a Language Model?

Language
Why is **language** so important in Human Cognition?

Informatik
Hochschule Reutlingen

1. Language is a system of communication that uses symbols in a regular way to create meaning.

2. Language gives us the ability to communicate to others by talking, reading, and writing. It gives groups of people a way to network human brains together.

3. Language is the main vehicle of culture. It allows people to transmit ideas precisely, preserve history, coordinate action, and teach abstract concepts. It enables the development of innovations across generations.

4. Language is the jewel in the crown of cognition. – Steven Pinker

Language
Why is **language** so important in Artificial Intelligence?

Artificial Intelligence needs Natural Language Processing and Understanding to simulate human intelligence.

Without language, AI would lack

- means for nuanced interaction (conversational Interaction),
- knowledge representation, and
- adaptive learning.

This makes language critical for AI's advancement and integration into society.

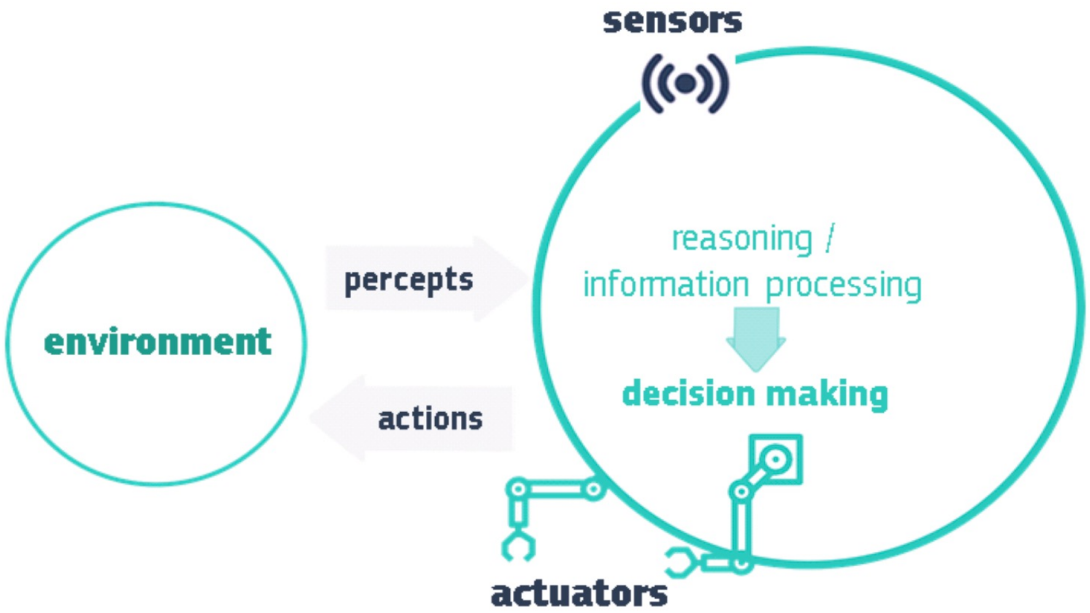


Figure 1: A schematic depiction of an AI system.

RECAP

Hochschule Reutlin

What does it mean to “model something”?

Imagine that we have a model of a physical world.

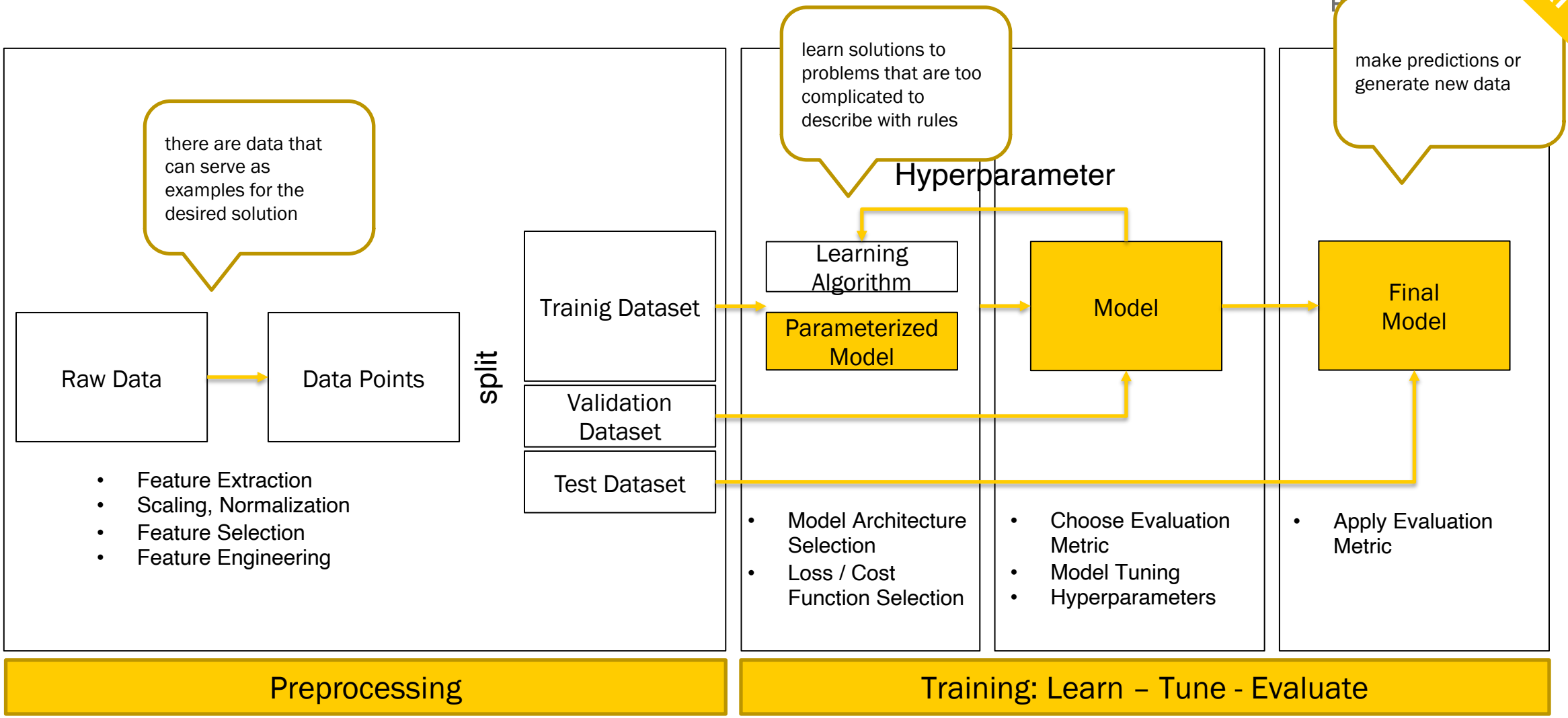
What do you expect it to be able to do?

- A good model would predict what happens next, given some description of “context”.
- A good model would simulate the behavior of the real word.
- It would “understand” which events are in better agreement with the world. Which of them are more likely.

Concept of a Model in Machine Learning

- learned in some machine learning process

RECAP



Concept of a Model in Machine Learning

- as a „black box“ function with input and output performing some task

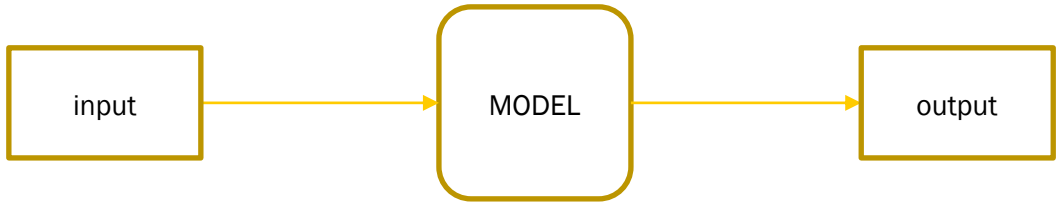
RECAP

The universal Function Approximation Theorem tells us, that every function can be approximated by a deep neural network:

A ML task is a type of prediction or inference being made by the model. It is based on the problem or question that is being asked, and the available data.

For example, the classification task assigns data to categories, and the clustering task groups data according to similarity.

A Model is like a function with input and output performing some task.



Concept of a Model in Machine Learning

Models and their Deep Learning Network Architectures

RECAP

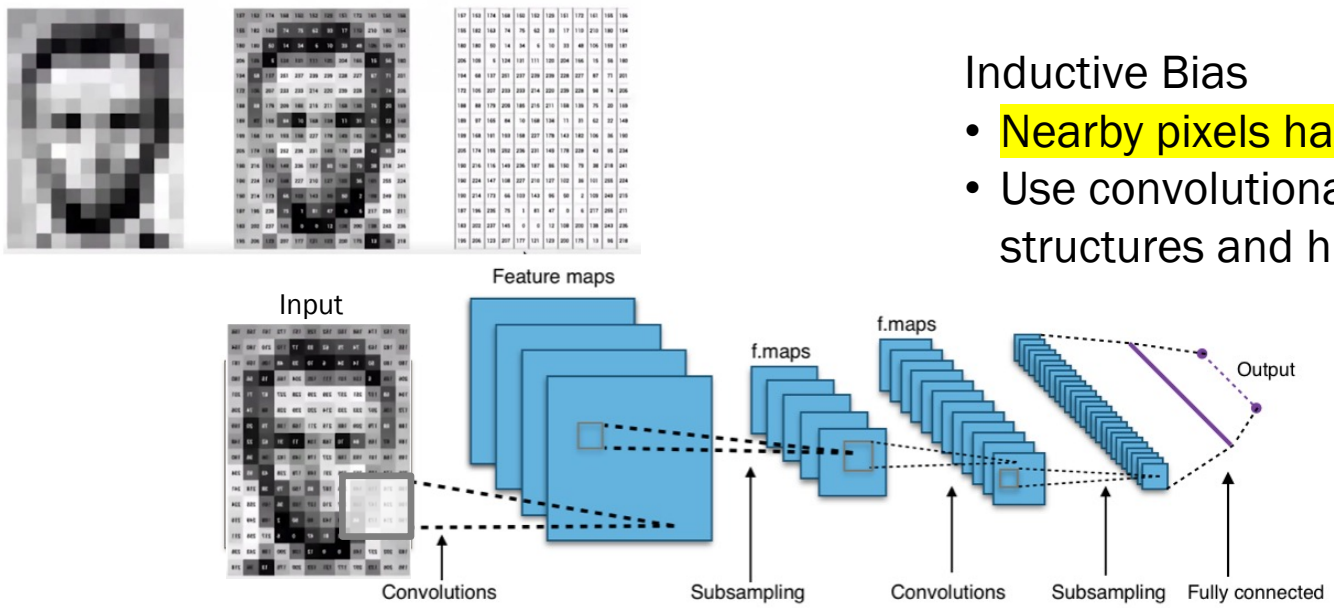
While designing a model, we make assumptions about our data depending on the geometry of the data: called Inductive Bias

Concept of a Model in Machine Learning

Models and their Deep Learning Network Architectures

RECAP

1. Images are represented as Matrix or Tensor



Inductive Bias

- Nearby pixels have similar colors
- Use convolutional filters to extract structures and hidden features

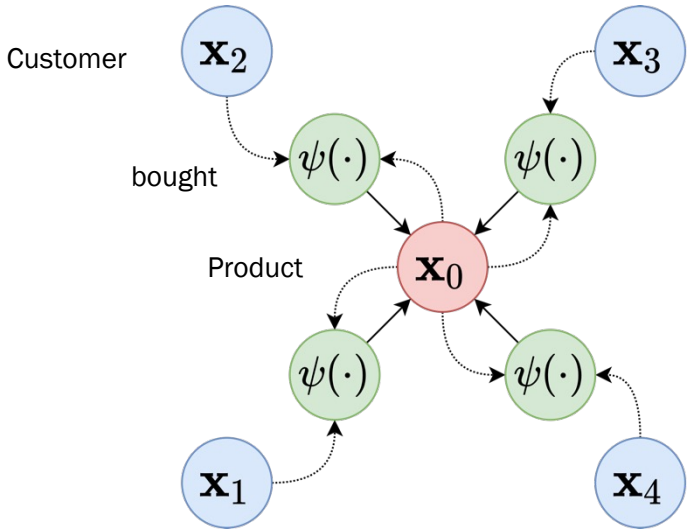
Source: Wikipedia - https://commons.wikimedia.org/w/index.php?curid=45679374

Concept of a Model in Machine Learning

Models and their Deep Learning Network Architectures

RECAP

2. Data like objects with relationships to each other can be represented as a graph



Inductive Bias

- Use the graph structure of the data to build a graph neural network.
- In a Message Passing GNN the network nodes update their representations by aggregating the messages received from their neighbors.
- Nearby Nodes have similar attributes and representations.
- Node representation could be used to downstream task, like classification, clustering or link prediction.

Source: Wikipedia - https://commons.wikimedia.org/w/index.php?curid=119852416

Concept of a Model in Machine Learning

Models and their Deep Learning Network Architectures

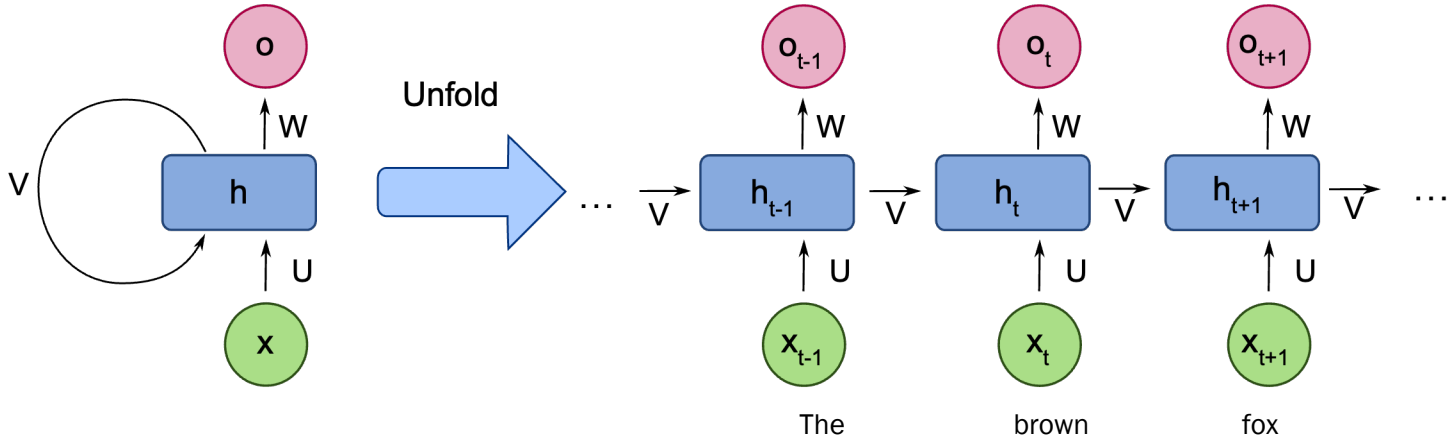
RECAP

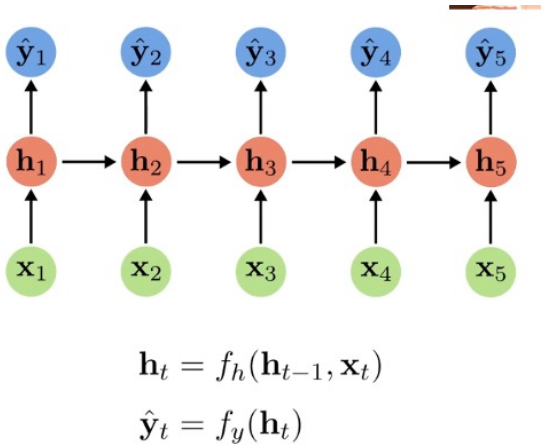
3. Language is represented as a Sequence:

["The", "brown", "fox", ...]

Inductive Bias

- Nearby words belong to the same semantic meaning
- sequence-2-sequence models like recurrent neural networks could be used to model context with hidden states





$$\mathbf{h}_t = f_h(\mathbf{h}_{t-1}, \mathbf{x}_t)$$
$$\hat{\mathbf{y}}_t = f_y(\mathbf{h}_t)$$

Source: Wikipedia - https://commons.wikimedia.org/w/index.php?curid=60109157

Language Models
What is a Language Model?

Hochschule Reutlingen

In language, an event is a linguistic unit: text, sentences, word, token, symbol

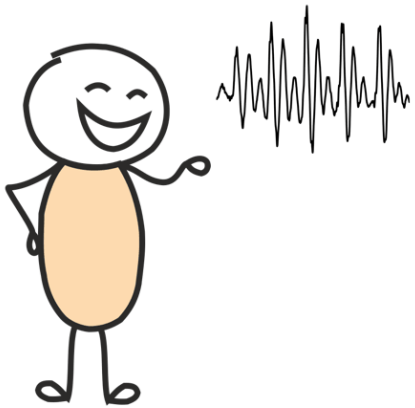
The goal of a language model is to estimate the probabilities of these events.

Definition:

A Language model is a probability distribution over linguistic units. It estimates the probability of different linguistic units: symbols, tokens, token sequences, … .

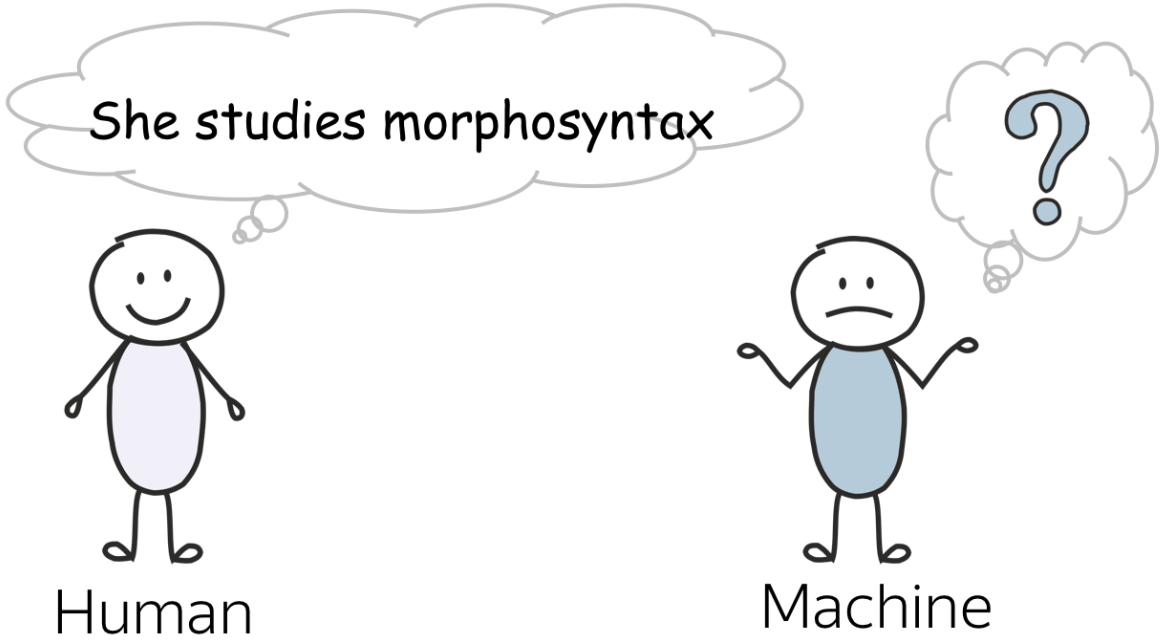
Hochschule Reutlingen

What is easy for humans, can be very hard for machines



Similarly sounding options

She studies morphosyntax
She studies more faux syntax
She studies morph or syntax
....



Source: the morphosyntax example is from the slides by Alex Lascarides and Sharon Goldwater, Foundations of Natural Language Processing course at the University of Edinburgh.

Language Models

Hochschule Reutlingen

But how a machine is supposed to understand?

- A machine needs a language model, which estimates the probabilities of sentences.

- If a language model “is good”, it will assign a larger probability to a correct sentence.

  P(„this is a correct sentence.“) > P(„htis is a coreckt sentence“.)

02.02 Autoregressive Language Model

Autoregressive Language Models

Hochschule Reutlingen

A language model is a probability distribution over sequences of a vocabulary.

General Framework:

- Our goal is to estimate probabilities of text fragments.
- Without loss of generality (w.l.o.g.) we deal with sentences.
- We want these probabilities to reflect knowledge of a language.
- We want sentences that are “more likely” to appear in a language to have larger probability according to our language model.

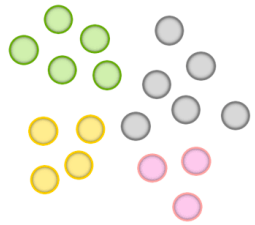
Source: https://lena-voita.github.io/nlp_course/language_modeling.html

Autoregressive Language Models
How to build a Language Model - General Framework

Hochschule Reutlingen

Could simple probability theory help?

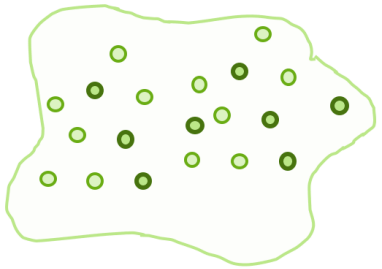
What is the probability to pick a green ball?



$$\frac{5}{5+6+4+3} = \frac{5}{18}$$

The probability to pick a ball of a certain color (let's say green) from this basket is the frequency with which green balls occur in the basket.

Can we do the same for sentences?



Text corpus

$$P(\text{the mut is tinming the tebn}) = \frac{0}{|\text{corpus}|} = 0$$

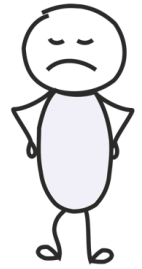
$$P(\text{mut the tinming tebn is the}) = \frac{0}{|\text{corpus}|} = 0$$

With this approach, sentences that never occurred in the corpus will receive zero probability

Since we can not possibly have a text corpus that contains all sentences in a natural language, a lot of sentences will not occur in the corpus.

While among these sentences some are clearly more likely than the others, all of them will receive zero probability.

But the first sentence is “more likely” than the second!
This method is not good!



Source: https://lena-voita.github.io/nlp_course/language_modeling.html

Autoregressive Language Models
How to build a Language Model - General Framework

Hochschule Reutlingen

Solution: Decompose a sentence into smaller parts!

We can not reliably estimate sentence probabilities if we treat them as atomic units. Instead, let's decompose the probability of a sentence into probabilities of smaller parts.

For example, let's take the sentence **I saw a cat on a mat** and imagine that we read it word by word. At each step, we estimate the probability of all seen so far tokens.

$$P(\text{I saw a cat on } \ldots) =$$

$$\underbrace{P(\text{I}) \cdot P(\text{saw}|\text{I}) \cdot P(\text{a}|\text{I saw}) \cdot P(\text{cat}|\text{I saw a}) \cdot P(\text{on}|\text{I saw a cat})}_{\text{Probability of I saw a cat on}} \cdot \underbrace{\ldots}_{\ldots}$$

Autoregressive Language Models
How to build a Language Model - General Framework

Hochschule Reutlingen

Formally, let be *(x_1, ..., x_n)* tokens in a sentence, and *P (x_1, ..., x_n)* the probability to see all these tokens (in this order).

Using the product rule of probability (chain rule), we get:

$$P(x_1, x_2, \dots x_n) = P(x_1) * P(x_2|x_1) * P(x_3|\, x_1, x_2) * \cdots * P(x_n|x_1, x_2, \dots, x_{n-1})$$
$$= \prod_{t=1}^{n} P(x_t|x_{<t}).$$

Source: https://lena-voita.github.io/nlp_course/language_modeling.html

Autoregressive Language Models
How to build a Language Model - General Framework

Hochschule Reutlingen

- We decomposed the probability of a text into conditional probabilities of each token given the previous context.
- Defines a joint distribution over an infinite sample space in terms of conditional distributions, each over a finite sample space (but with potentially infinite history!)
- What we got is the autoregressive left-to-right language modeling framework.

Source: https://lena-voita.github.io/nlp_course/language_modeling.html

Autoregressive Language Models
How to build a Language Model - General Framework

Hochschule Reutlingen

Need to define, how to compute $P(x_n|x_1, x_2, \dots, x_{n-1})$.

Aus:

$$P(x_1, x_2, \dots x_n) = P(x_1) * P(x_2|x_1) * P(x_3|\, x_1, x_2) * \cdots * P(x_n|x_1, x_2, \dots, x_{n-1})$$

folgt:

$$P(x_n|x_1, x_2, \dots, x_{n-1}) = \frac{P(x_1, x_2, \dots x_n)}{P(x_1) * P(x_2|x_1) * \cdots * P(x_{n-1}|x_1, x_2, \dots, x_{n-2})}$$

Source: https://lena-voita.github.io/nlp_course/language_modeling.html

Autoregressive Language Models
How to build a Language Model - General Framework

Hochschule Reutlingen

Generate text using a Language Model:

If we have a sequence of tokens $x_1 \ldots x_k$, then we can use the language model with vocabulary V to predict the next token x_{k+1}:

I ____

$$x_{k+1} = argmax_{x \in V} P(x|x_1 \ldots x_k)$$

We do it one token at a time: predict the probability distribution of the next token given previous context, and sample from this distribution.

Source: https://lena-voita.github.io/nlp_course/language_modeling.html

1. N-gram Language Models
2. Neural Net Language Models (neural networks)

30

02.03 Build a N-Gram Language Model with Python

INF Informatik
Hochschule Reutlingen

N-GRAM

Hochschule Reutlingen

N-gram models make all terms finite with the Markov assumption:

„only the last seen N tokens are relevant for the context to predict the next token i“

$$P(x_i|x_1, x_2, \dots, x_{i-1}) \approx P(x_i|x_{i-N}, x_{i-N+1}, \dots, x_{i-1})$$

Example: (3-gram)

- we have seen 9 tokens and want to predict the 10th token
- Use a 3-gram LM then N = 3. (We expect that the next token only depends on the 3 previous tokens.): $P(x_{10}|x_1, \dots., x_9) \approx P(x_{10}|x_7, x_8, x_9)$

Hochschule Reutlingen

How to compute $P(x_4|x_1, x_2, x_3)$?

→ Estimate conditional probabilities from 3-gram counts in the training data D.

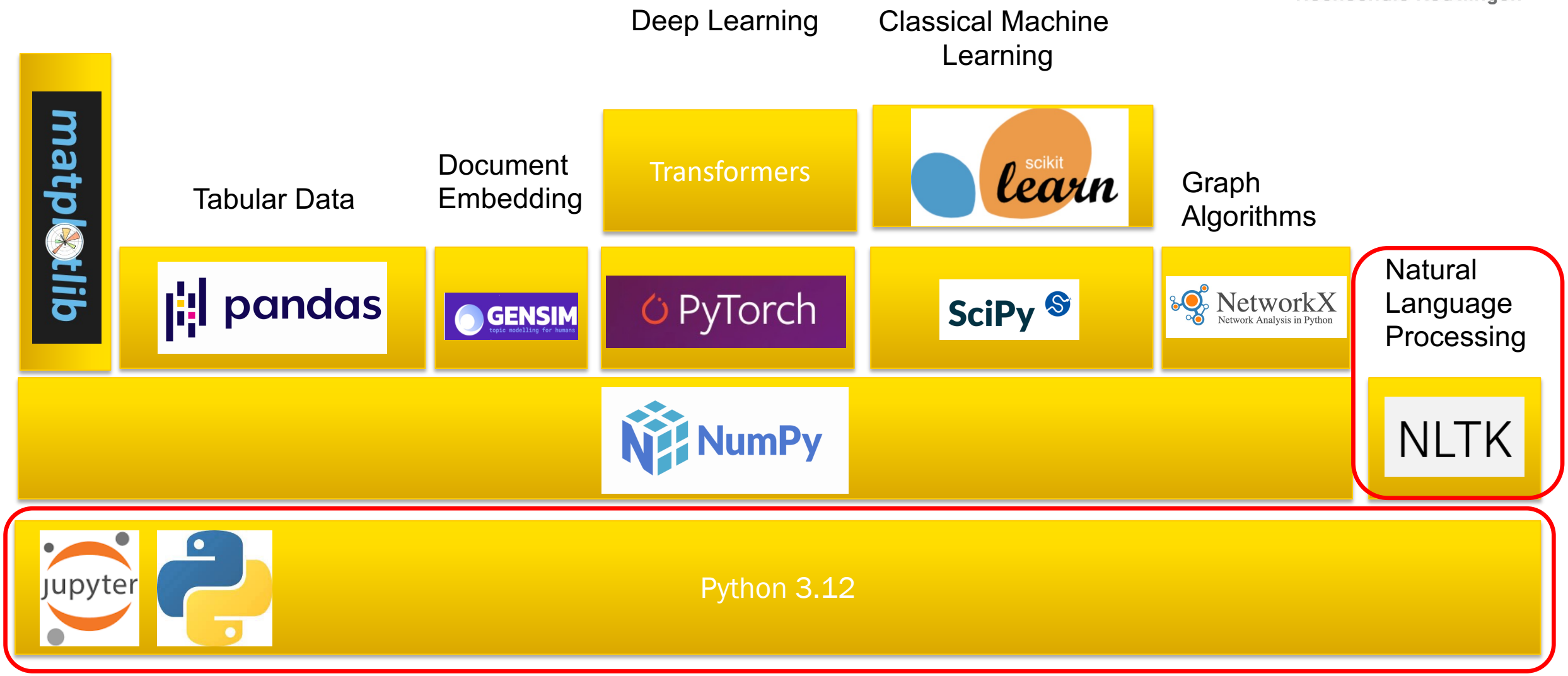
$$P(x\_2 \mid x\_1) = \frac{\text{Count}(x\_1, x\_2)}{\text{Count}(x1)}, \quad P(x\_1, x\_2) = \frac{\text{Count}(x\_1, x\_2)}{\text{Count(all tokens)}}$$

$$P(x\_3 \mid x\_1, x\_2) = \frac{\text{Count}(x\_1, x\_2, x\_3)}{\text{Count}(x\_1, x\_2)}, \ldots$$

https://opencourse.inf.ed.ac.uk/sites/default/files/https/opencourse.inf.ed.ac.uk/nlu-11/2024/l03-conditional-lm-handout.pdf

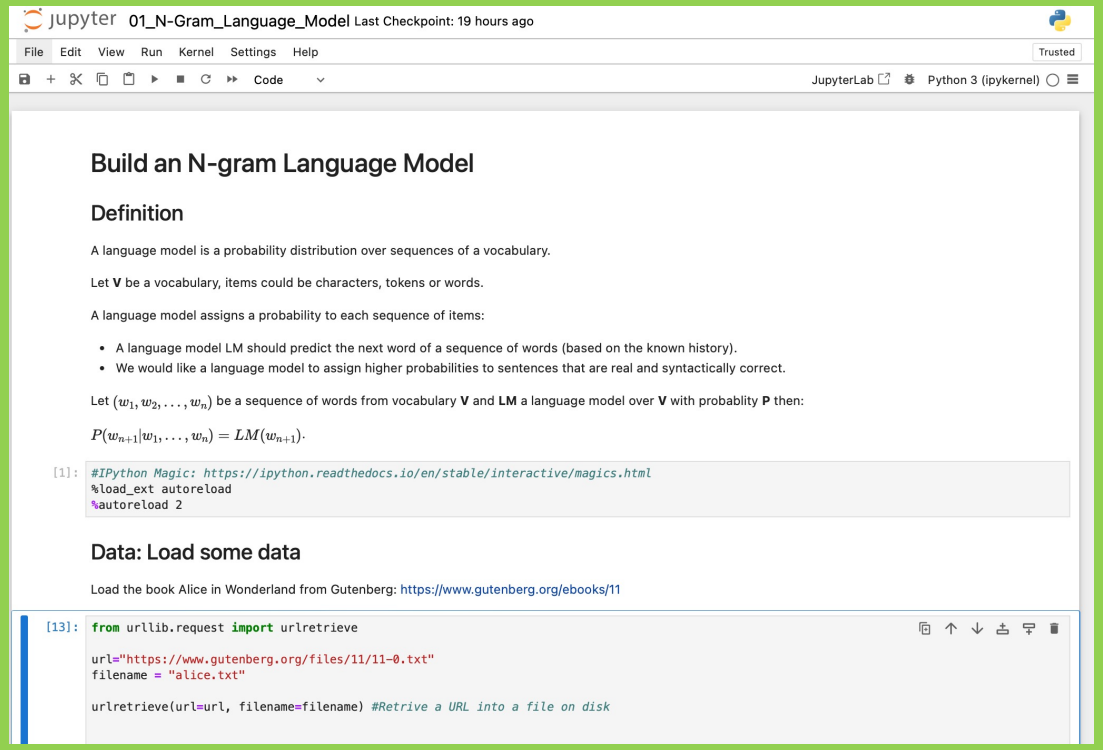
MACHINE LEARNING PYTHON STACK

Knowledge Foundation @ Reutlingen University
INF Informatik
Hochschule Reutlingen



N-Gram Notebook

01_N-Gram_Language_Model.ipynb




```
conda create --name ws25_2 python=3.12
conda activate ws25_2
pip install jupyter
pip install nltk
```

02.04
Neural Net Language Models

INF Informatik
Hochschule Reutlingen

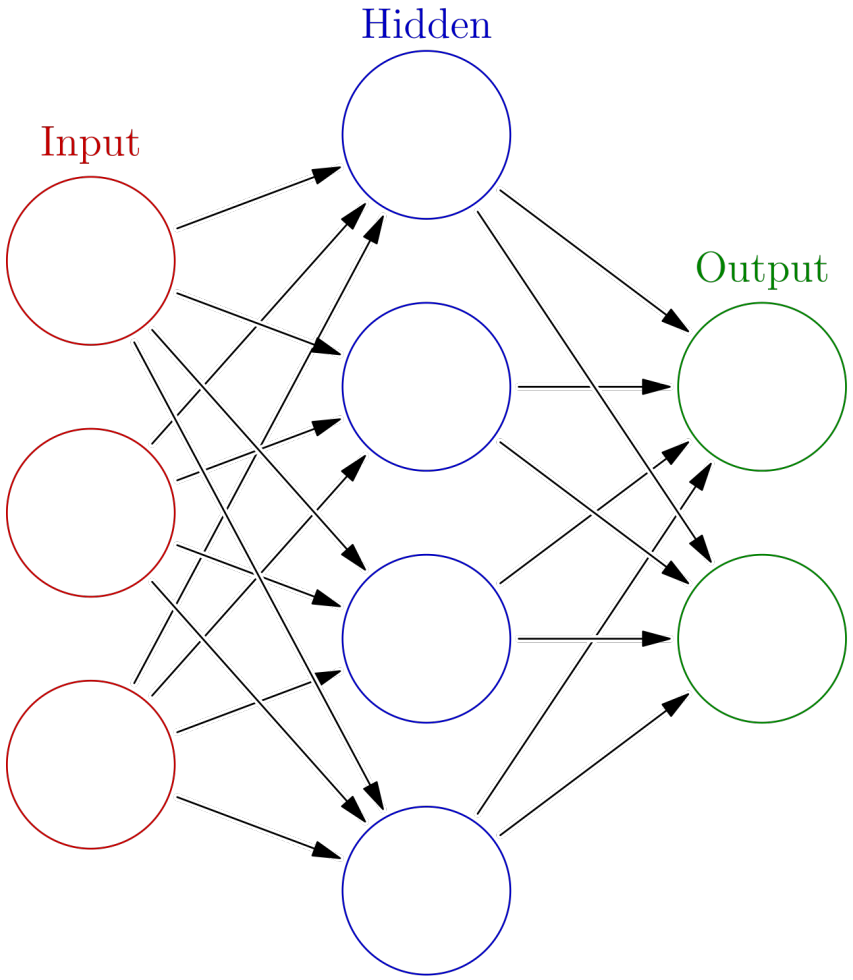
Neutral Net Language Models

Hochschule Reutlingen

Recap: Need to define, how to compute $P(x_n | \underbrace{x_1, x_2, \ldots, x_{n-1}}_{\text{history or context}})$.

- N-Gram
- Neural Models (neural networks)

A very short Introduction into Neural Networks

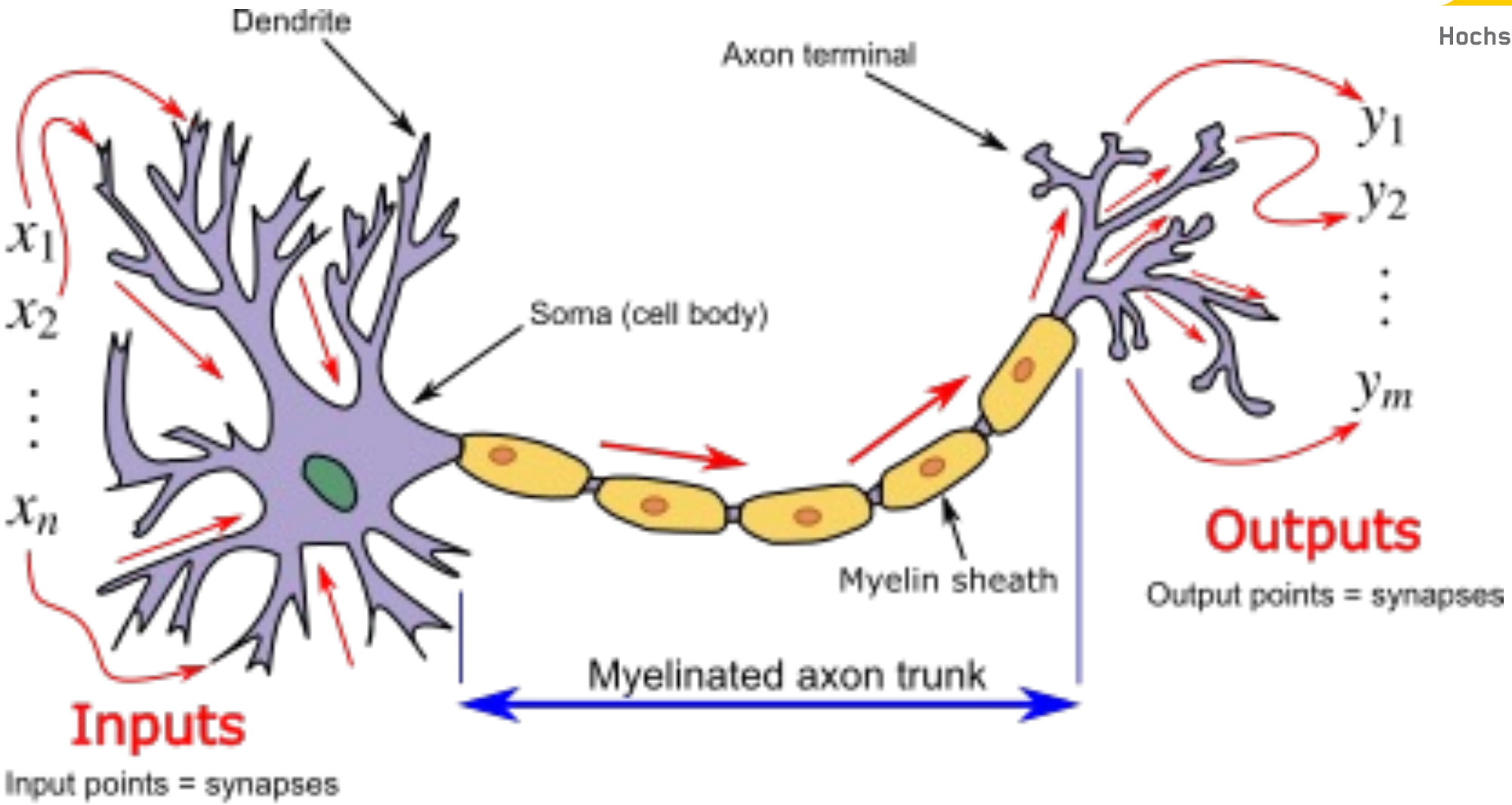


- In machine learning, a neural network (NN) is a model inspired by the structure and function of biological neural networks in animal brains.
- An NN consists of connected units or nodes called artificial neurons, which loosely model the neurons in the brain.
- These are connected by edges, which model the synapses in the brain.
- Each artificial neuron receives signals from connected neurons, then processes them and sends a signal to other connected neurons. The "signal" is a real number, and the output of each neuron is computed by some non-linear function of the sum of its inputs, called the activation function.
- The strength of the signal at each connection is determined by a weight, which adjusts during the learning process.

Source: https://en.wikipedia.org/wiki/Neural_network_(machine_learning)

A very short Introduction into Neural Networks

Hochschule Reutlin



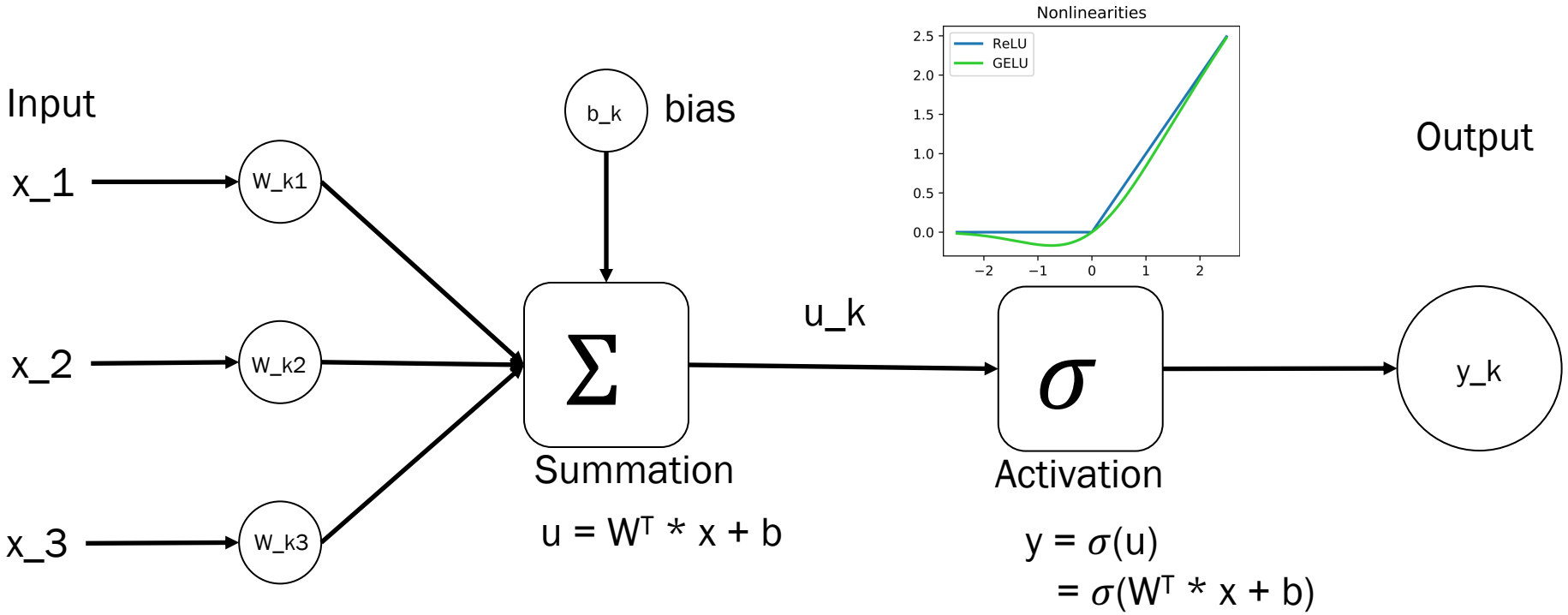
Source: https://en.wikipedia.org/wiki/Neural_network_(machine_learning)

A very short Introduction into Neural Networks

RECAP

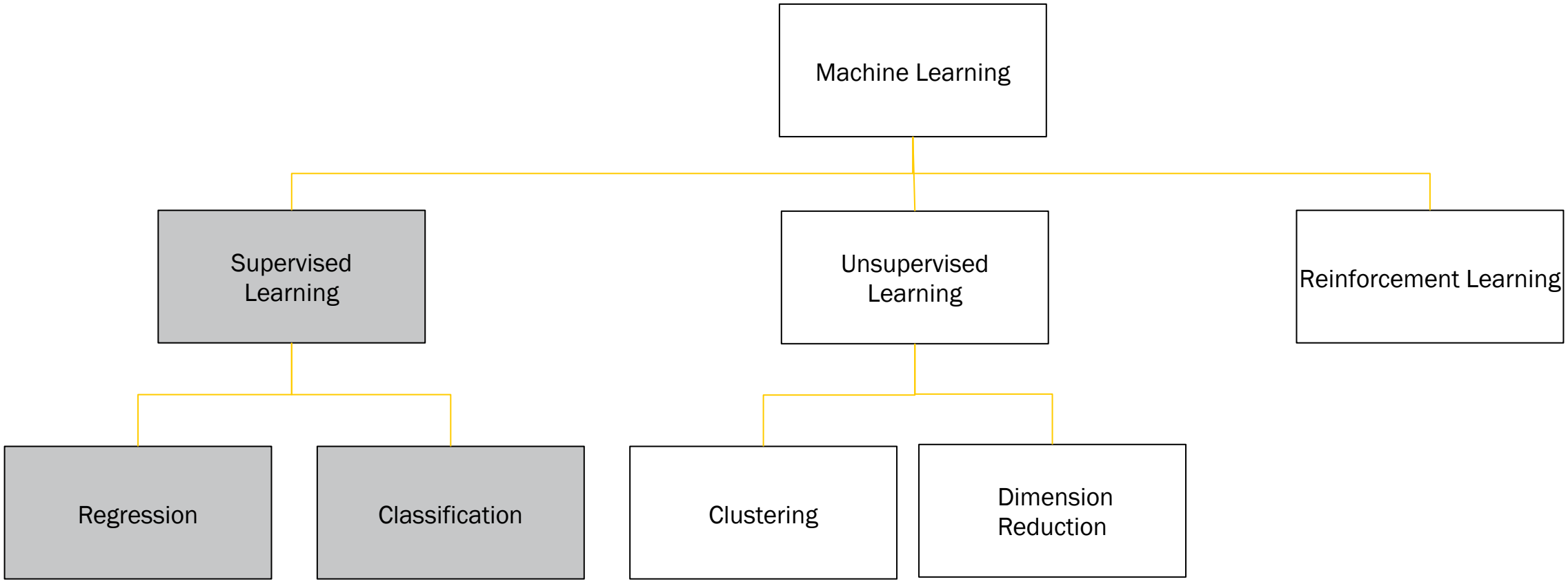
Hochschule Reutlin

Each neuron of the hidden layer does the following computation



A very short Introduction into Neural Networks

RECAP



RECAP

Hochschule Reutlin

Supervised Learning

(2) Collect (labeled) Data

Training Data

Data Points

FEATURES

LABELS

(1) Define the problem to be solved

Unknown Pattern

$$f: X \rightarrow Y: f(x) = y$$

(3) Chose Algorithm and Optimization Metric

ML Algorithm
Model.fit(X,y)

Hyperparameter:
- Learning Rate
- epochs
…

Model
(Hypothesis)

$$h: X \rightarrow Y: h(x;\ \theta) = \hat{y}$$

Cost / Loss
-function

$$J(\theta) = \sum L(y, h(x;\ \theta))$$

(4) Chose a Metric to evaluate the final model

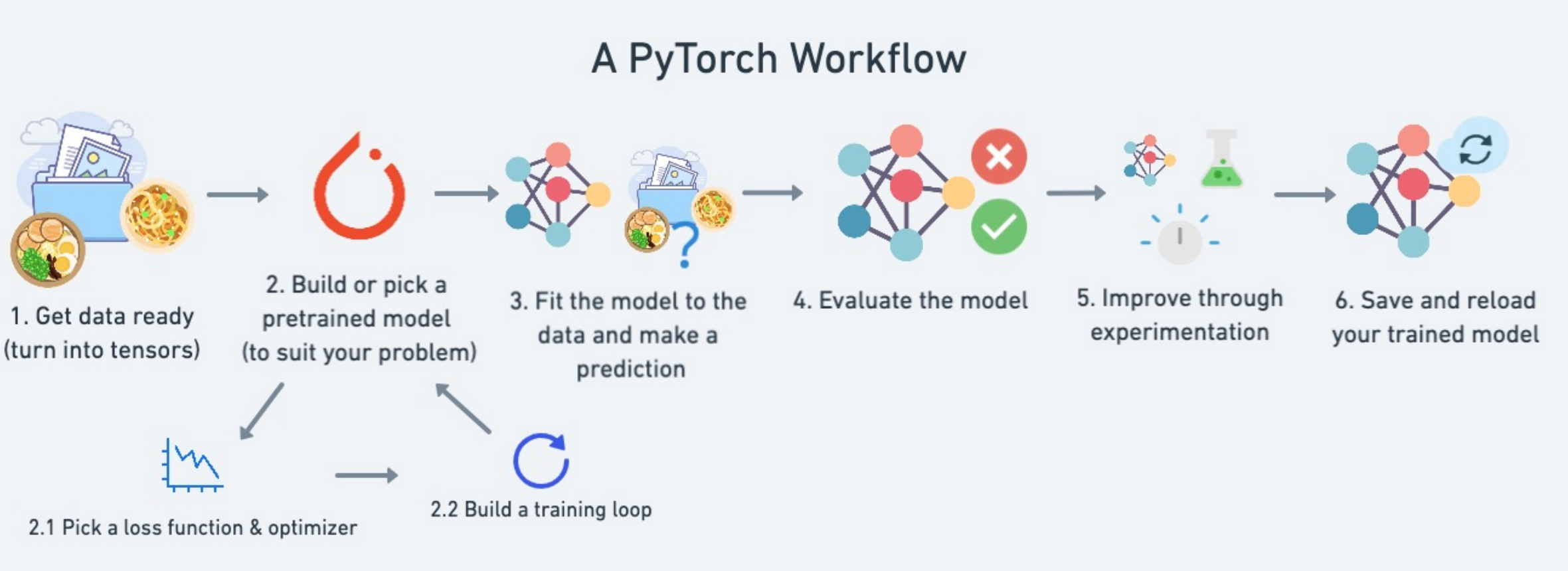
New Data → Predictive Model **Model.predict(X)** → **Prediction**

A very short Introduction into Neural Networks

RECAP

Hochschule Reutlin

Neural Networks with PyTorch



Source: https://www.learnpytorch.io/01_pytorch_workflow/

Neutral Net Language Models

Hochschule Reutlingen

Intuitively, Neural Net Language Models do two things:

1. **process context** $(x_1, x_2, \ldots, x_{n-1})$ → model-specific

   The main idea here is to get a vector representation for the previous context. Using this representation, a model predicts a probability distribution for the next token. This part could be different depending on model architecture (e.g., RNN, CNN, whatever you want), but the main point is the same - to encode context.

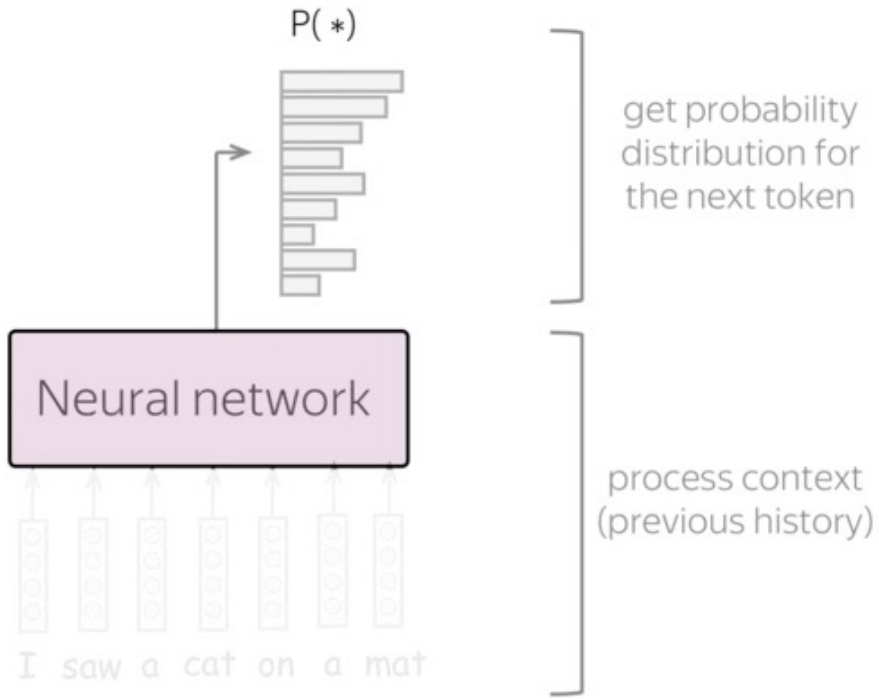
Source: https://lena-voita.github.io/nlp_course/language_modeling.html

Neutral Net Language Models

Hochschule Reutlingen

2. **generate a probability distribution for the next token (x_n)** → model-agnostic

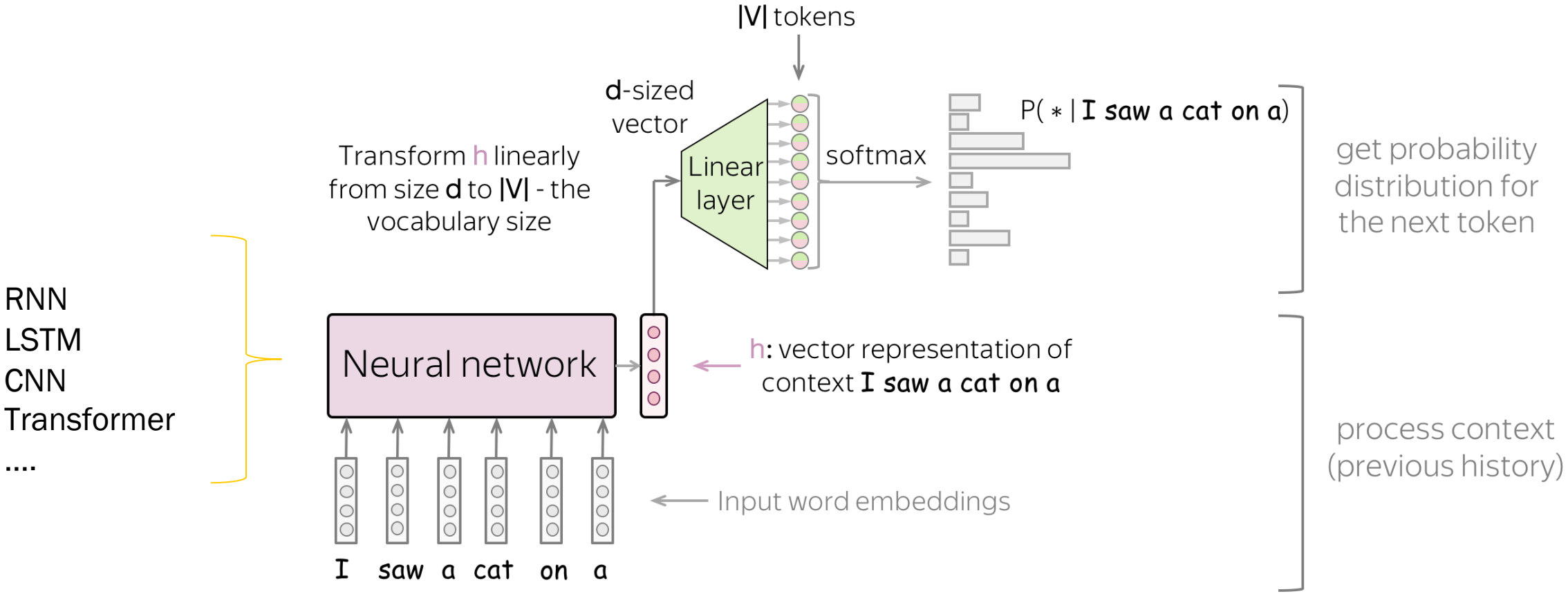
Once a context has been encoded, usually the probability distribution is generated:



Source: https://lena-voita.github.io/nlp_course/language_modeling.html

Neutral Net Language Models

We can think of neural net language models as neural classifiers, where the classes are |V| vocabulary tokens.



Source: https://lena-voita.github.io/nlp_course/language_modeling.html

Neutral Net Language Models

Hochschule Reutlingen

Pipeline:

1. feed word embedding for previous (context) words into the network
2. get vector representation of context back from the network
3. Transform vector representation into |V|-dim vector (logits)
4. from this vector representation, predict a probability distribution for the next token

(1) & (2)

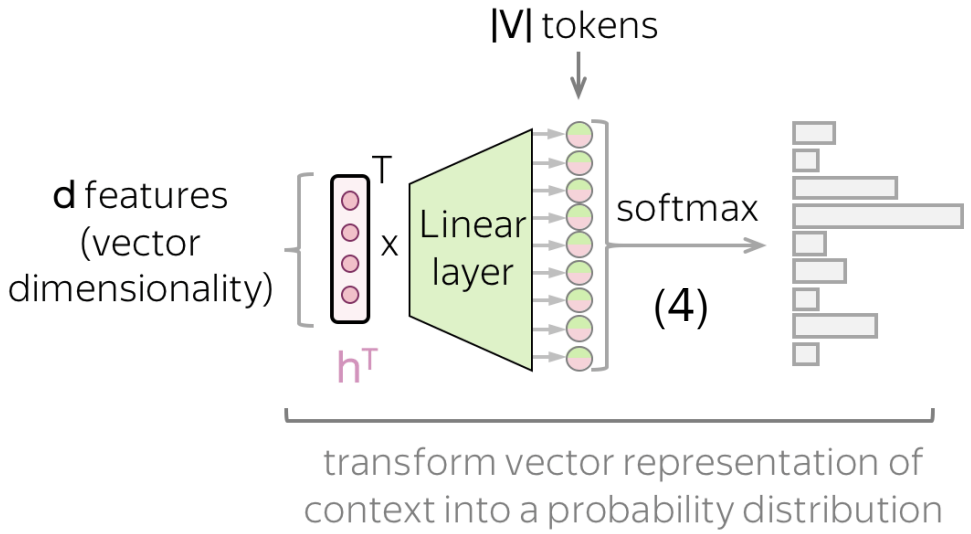
We have:

- h - vector of size **d**

We need:

- vector of size |V| – probabilities for all tokens in the vocabulary

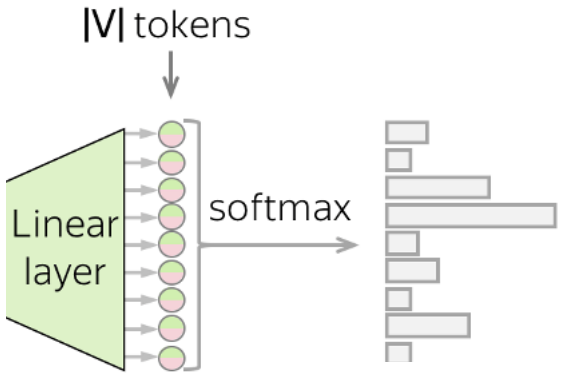
(3)
Transform linearly from size **d** to size |V|



Source: https://lena-voita.github.io/nlp_course/language_modeling.html

Hochschule Reutlingen

Using Softmax to convert logits into probabilities:



$$\mathbf{z} \longmapsto \sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{|V|} e^{z_j}} .$$

```
import numpy as np

def softmax(z, beta=1.0):
    return np.exp(beta * z)/ np.sum(np.exp(beta * z))

z = np.array([1.0, 2.0, 3.0, 4.0, 1.0, 2.0, 3.0])
p = softmax(z)
p_max = np.argmax(p)
```

the softmax applies the standard exponential function to each element z_i of the input vector z (consisting of |V| real numbers), and normalizes these values by dividing by the sum of all these exponentials.

https://pytorch.org/docs/stable/generated/torch.nn.Softmax.html

Neutral Language Models

RECAP

We know Cross-Entropy Loss as standard loss function for neural net classifiers:

$$L(q,p) = < -q, log(p) >= -\sum_{\mathrm{i}}^{|V|} \mathrm{q}_i * log(p_i)$$

With:

- Target probability $q = (0, \ldots 0,1,0, \ldots . . 0)$, 1 for target label, 0 for the rest.
- Predicted probability $p = (p_1, \ldots . p_{|V|})$, $p_i = p(i|x)$

02.05
Perplexity

INF
Informatik
Hochschule Reutlingen

How to measure „Goodness“ of a Language Model?

1. Predicting with Language Models is a classification problem with |V| classes (V vocabulary of LM)
2. Accuracy is usually used as a metric for classification problems:
   - There is a single ground-truth label.
   - The model outputs one predicted label.
   - Accuracy = fraction of examples where prediction == ground truth.

How to measure „Goodness“ of a Language Model?

3. But a LM does not make one „single correct prediction“.

I always order pizza with cheese and _____

- salami 0.2
- pepperoni 0.1
- anchovoes 0.01
- ....
- bananas 0.00001
- ....
- nails 1e-100

Language is not binary. There are many „good“ sentences.

How to measure „Goodness“ of a Language Model?

4. Instead it assigns a probability distribution over the vocabulary:

$$P(\,nexttoken \mid context\,) \in [0,1], \sum_{v \in V} P(v|context) = 1$$

A „good“ language model should assign a higer probability to real and frequently observed sentences than ungrammatical or impossible or rarely observed sentences.

Perplexity captures what accuracy misses:

A language model defines a probability distribution over sequences and perplexity always measures how that model performs on some data.

For a sequence w = $(x_1, x_2, \ldots x_N)$ the perplexity is defined as

$$Perplexity(w) := P(x_1, x_2, \ldots x_N)^{-1/N} = \sqrt[N]{\frac{1}{P(x_1, x_2, \ldots x_N)}}$$

Perplexity

Hochschule Reutlingen

Interpretation:

Perplexity is meant to measure how surprised a model is, on average, by seeing the true sequence:

Lower perplexity ⟶ model assigns higher probability to real data ⟶ less “surprise.”

We want a positive that:

- increase when the model’s predictions are poor (low probabilities),
- decrease when predictions are good (high probabilities),
- be normalized by sequence length so it’s comparable across test sets of different sizes.

Perplexity

Hochschule Reutlingen

Perplexity is highly related to Cross-Entropy:

For a sequence w = $(x_1, x_2, \dots x_N)$

- PP(w)
- = exp (ln (PP(w))
- = exp (ln ($P(x_1, x_2, \dots x_N)^{-1/N}$)) | *use chain rule*
- = exp (ln (($\prod_{t=1}^{N} P(x_t|x_{<t})$)$^{-1/N}$)) | ln($a^{b)}$) = b ln(a)
- = exp (-1/N * ln ($\prod_{t=1}^{N} P(x_t|x_{<t})$)) | ln(a*b) = ln(a) + ln(b)
- = exp (-1/N * $\sum_{t=1}^{N} \ln P(x_t|x_{<t})$) | H(w) = -1/N * $\sum_{t=1}^{N} \ln P(x_t|x_{<t})$
- = exp(H(w))

Perplexity

Hochschule Reutlingen

Perplexity grows exponentially with cross-entropy:

PP(w) = exp(H(w))

- Cross-entropy H(w) = -1/N * $\sum_{t=1}^{N} \ln P(x_t|x_{<t})$ measures the average surprise per token (in nats).
- Perplexity then converts that additive measure into a multiplicative one — it tells you the effective branching factor of the model:
  - If the cross-entropy is H, the model behaves as if it has exp(H) equally likely options for each token.

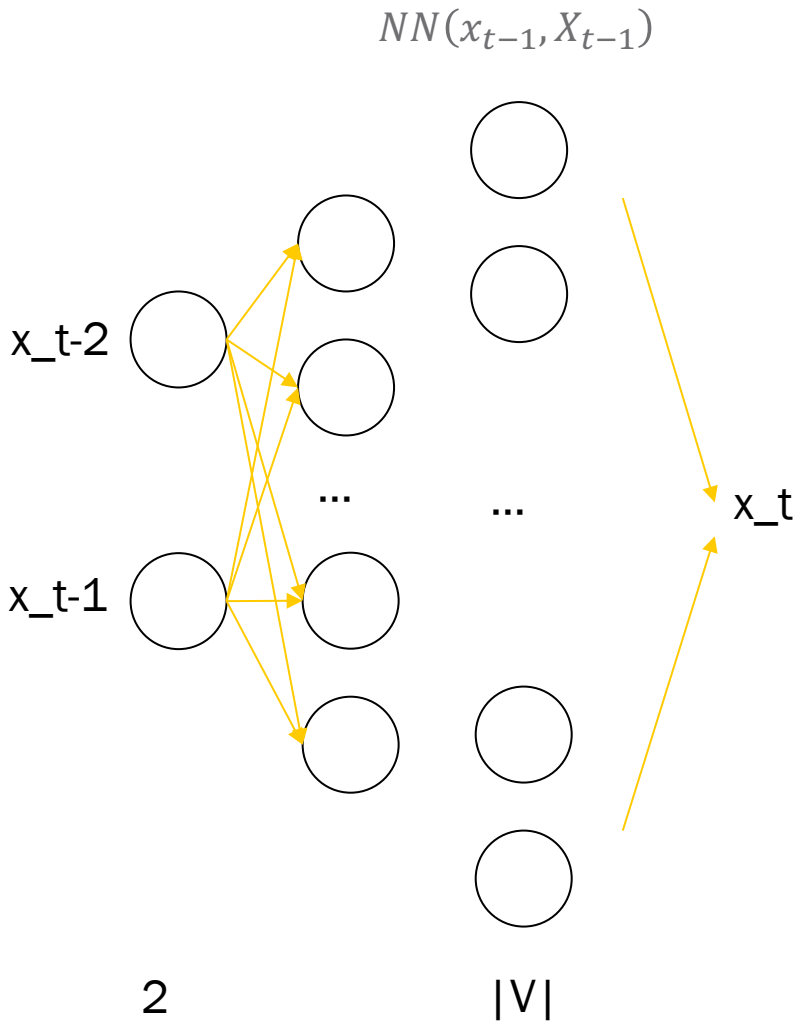
02.06 Attention (Memory)

Attention

Lets build a NN Language Model with vocabulary V following a Markov assumption with a history of 2 tokens

$$softmax\big(NN(x_{t-1}, x_{t-1})\big) = p(x_t|x_{t-2}, x_{t-1};\ \theta)$$

What is the problem ?



Attention

Example:

The dog walked to the park.

[redacted] dog walked __ Pretty easy

[redacted] walked to __ A location but where ?

[redacted] to the __ A location but who ?

We need much more history!

Attention

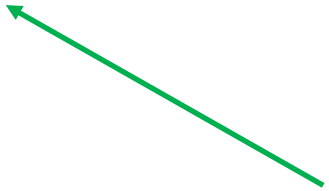
Example: (newspaper article)

We spoke with David Parkes in C5

…

…

We looked into that“, said Professor _____



We need memory!

~~$P(x_t|x_1, x_2, \ldots, x_{t-1}) \approx P(x_t|x_{t-2}, x_{t-1})$~~

→ We need a fully autoregressive model that use all the previous tokens.

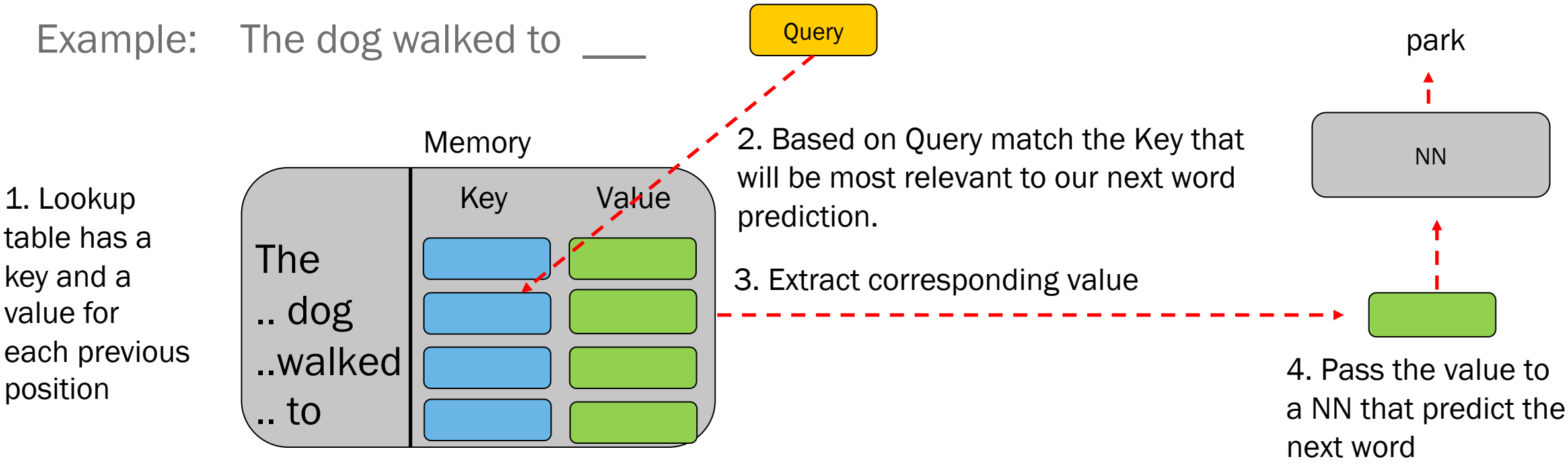
→ $p(x_t|x_1, x_2, \ldots, x_{t-1};\ \theta)$

Attention

Solution:

Build a NN version of something like a „random access memory“ or „lookup table“ to save previous information in case you need it later.

Example: The dog walked to ___



Attention

Process:

Step 1: Query matches Key

Step 2: Argmax match is selected

Step 3: Return Value of that Key

Issue: NN can not learn argmax

Solution: replace argmax with softmax

Attention

New process:

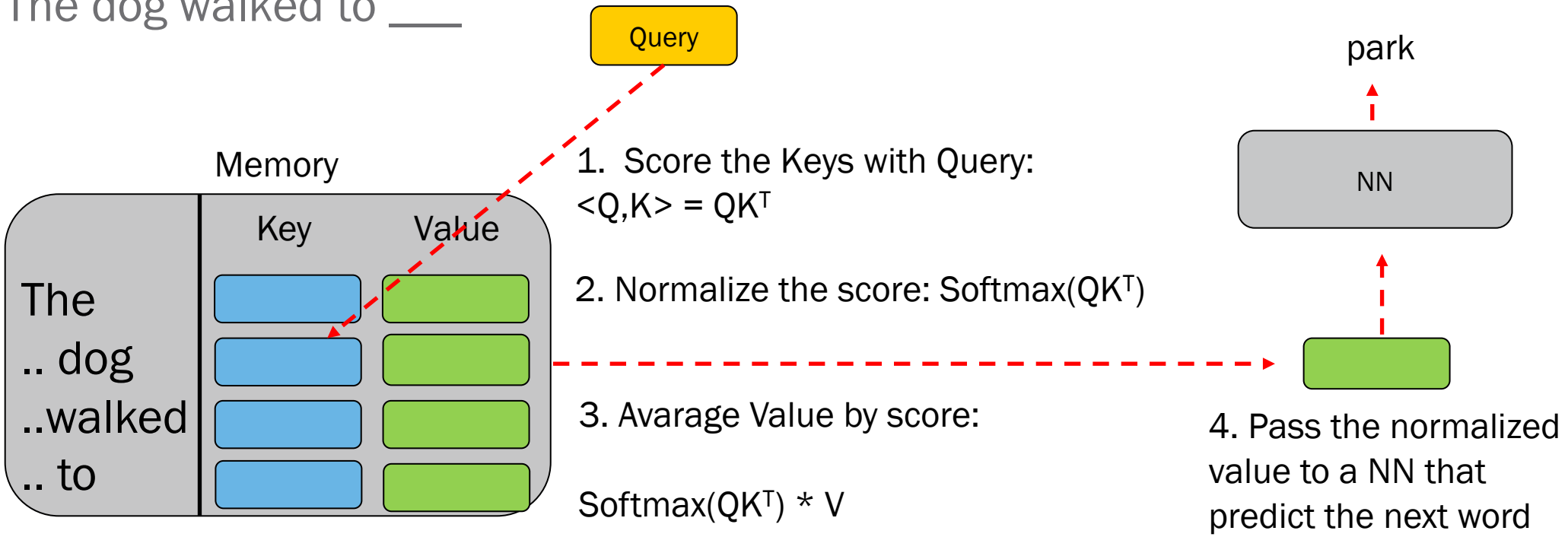
Step 1: Query scores Key (instead of picking the highest Key)

Step 2: Softmax normalizes score

Step 3: Average Value by score (instead of picking one singular Value)

Attention

Example: The dog walked to ___



Lookup table has a key and a value for each previous position

1. Score the Keys with Query: $<Q,K> = QK^T$

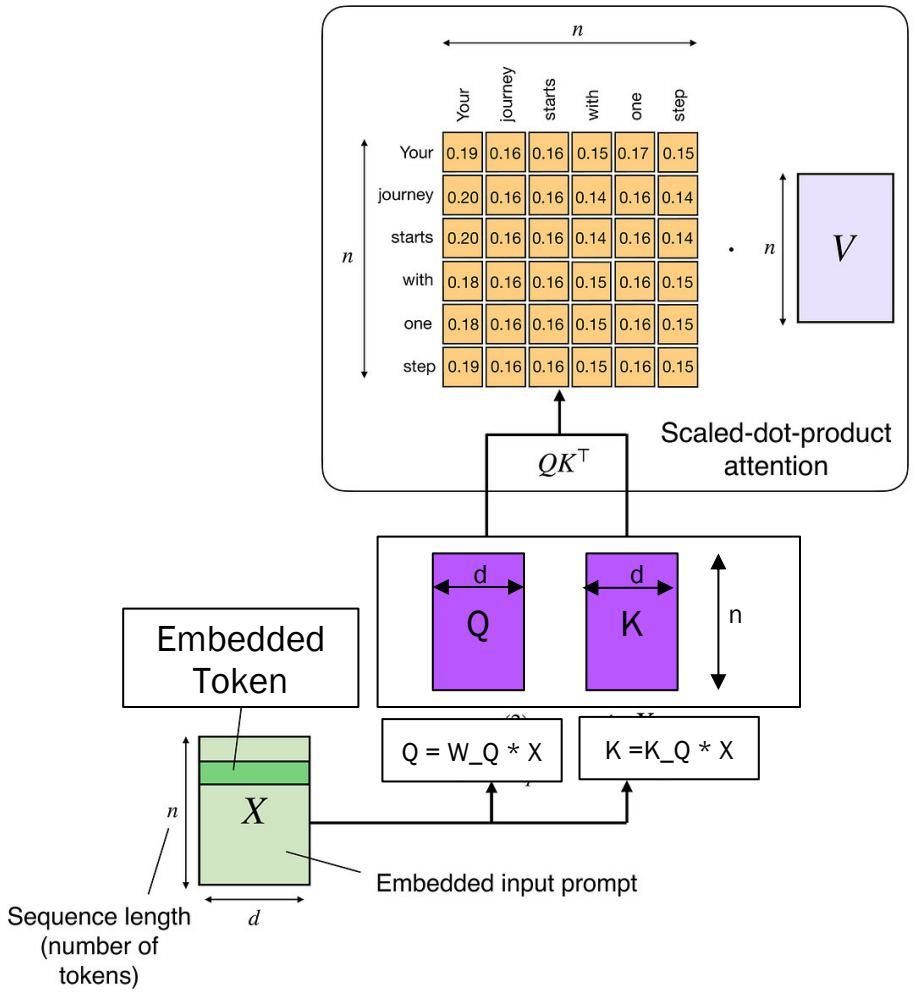
2. Normalize the score: Softmax(QK^T)

3. Avarage Value by score:

Softmax(QK^T) * V

4. Pass the normalized value to a NN that predict the next word

Attention



Details: For each given token in the sentence, the model should extract relevant information from all other tokens – weighted according to their significance for the current position.

This is done using three matrices:

Query Q = X*W_Q – represents the current position

Key K = X*W_K – represents every possible context token

Value V = X*W_V – contains the actual information that is conveyed.

X is an n x d matrix and contains an embedding vector of dimension d for each of the n tokens, W_* are the learned parameter matrices

Source: Sebastian Raschka

Attention

Hochschule Reutlingen

From „Attention is all you need“, 2017

$$\text{Softmax}(Z)_{ij} = \frac{e^{Z_{ij}}}{\sum_{k=1}^{n} e^{Z_{ik}}}$$

$$\text{Attention}(Q, K, V) = \underbrace{\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)}_{\text{Attention Matrix A}} V$$

Attention calculates:

- Similarity ($Q * K^T$)
  → How strongly does the context token belong to the current focus?
- Softmax
  → normalisation to weights.
- Weighted mean of V
  → combination of information.

Attention Matrix A has shape (SeqLen × SeqLen):

- Each row → focus of one token on all other tokens.
- Each Entry A_i_j indicates how strongly token i pays attention to token j.
- This matrix can be represented as a heat map (tokens on the X and Y axes).

02.07 Transformer Architectures

RECAP

Hochschule Reutlin

Recap: Language Models are statistical Models

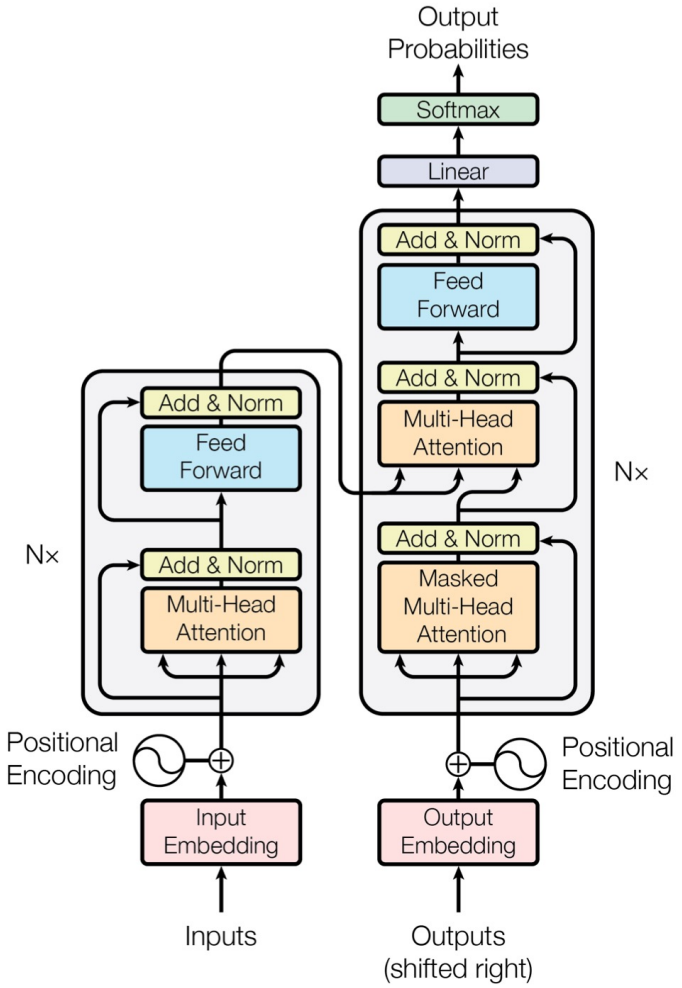
$$p(\mathbf{x}) = p(x_1, \ldots, x_T) = \prod_{t=1}^{T} p(x_t | x_1, \ldots, x_{t-1})$$
$$= p(x_1)\, p(x_2|x_1)\, p(x_3|x_1, x_2) \ldots$$

Language Model:

- Probability distribution over a sequence of **discrete tokens** $\mathbf{x} = (x_1, \ldots, x_T)$ where each token can take a value from a **vocabulary** $\mathcal{V}$ $(x_t \in \mathcal{V})$
- Decomposes into a sequence of conditional distributions
- The history (conditioning variables/tokens) is called **context** in NLP

From: Deep Learning (Prof. Andreas Geiger, University of Tübingen) - Lecture 9.2 (Natural Language Processing: Traditional Language Models)

Transformer Architectures



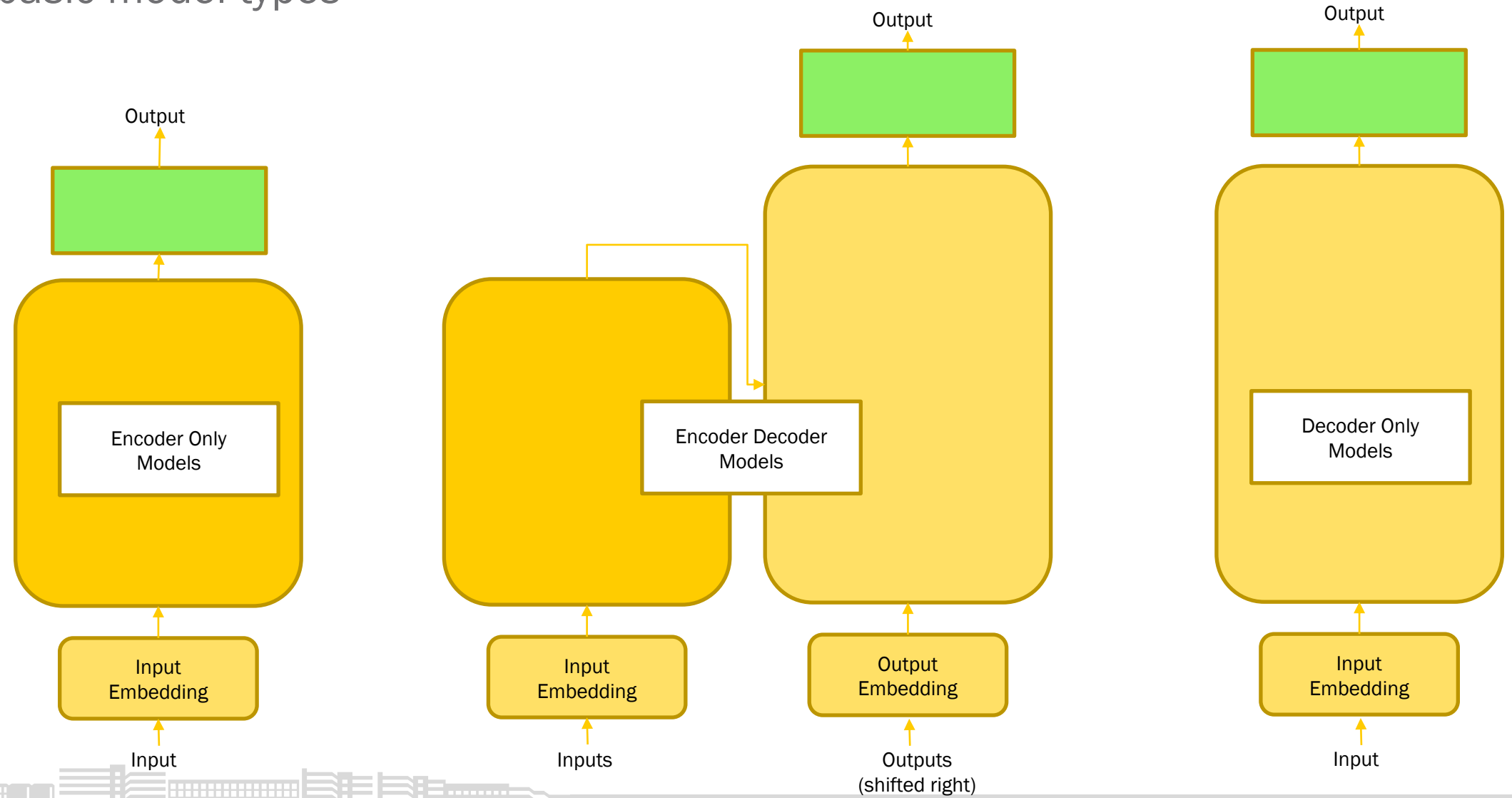
From: Attention Is All You Need - https://arxiv.org/abs/1706.03762

Transformer Architecture enables 3 types of models:

1. Encoder-Only: Natural Language Understanding
2. Encoder-Decoder: Seq2Seq Models for e.g. Machine Translation
3. Decoder-Only: Text Generation

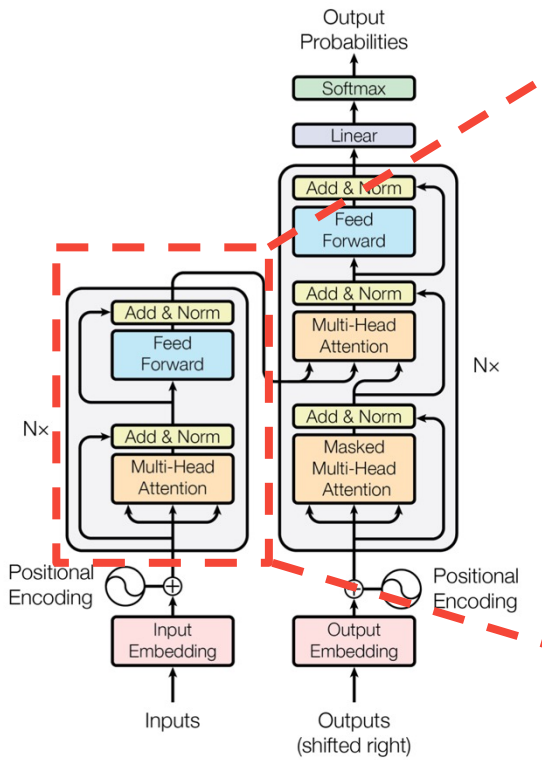
Transformer Architectures

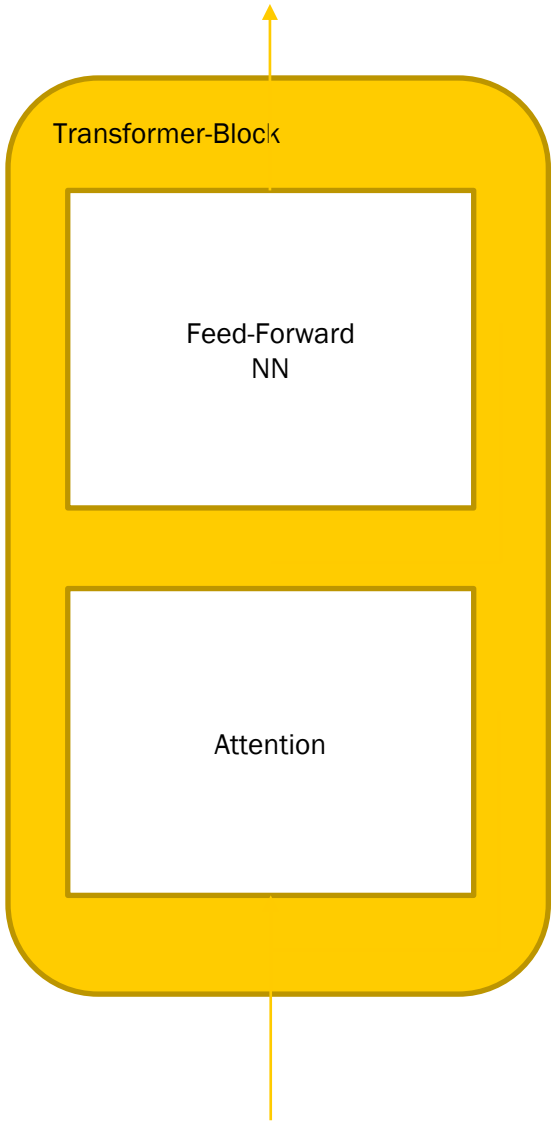
3 basic model types



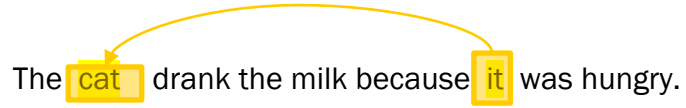
Transformer Architectures

Encoder-Block & Self Attention





While processing a word, Attention enables the model to focus on other words in the input that are closely related to that word.



The Transformer-Block is a reusable module that is the defining component of all Transformer architectures

Transformer Architectures

Hochschule Reutlingen

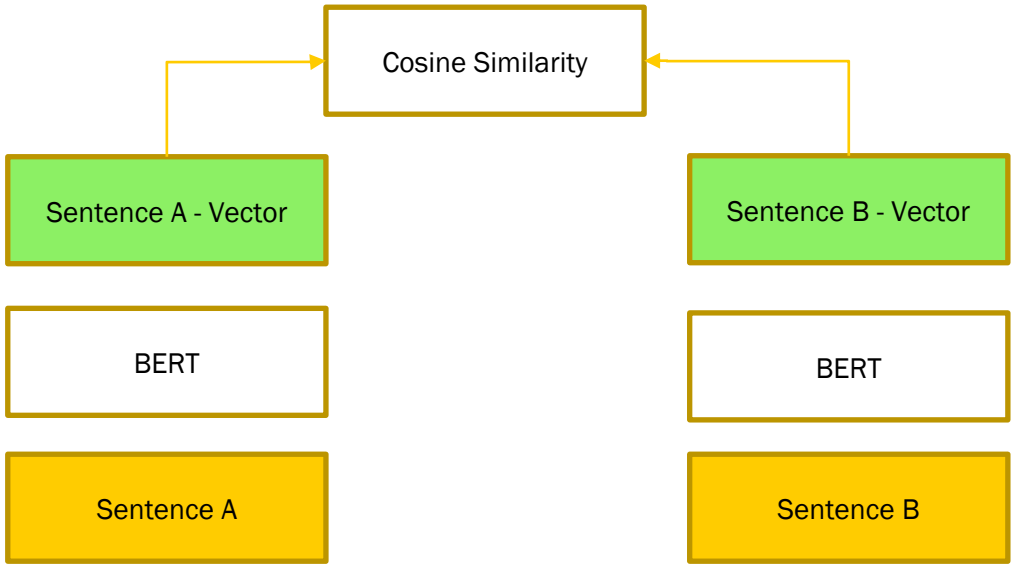
Transformer Architecture enables 3 types of models:

1. Encoder-Only: Natural Language Understanding
2. Encoder-Decoder: Seq2Seq Models Machine Translation
3. Decoder-Only: Text Generation

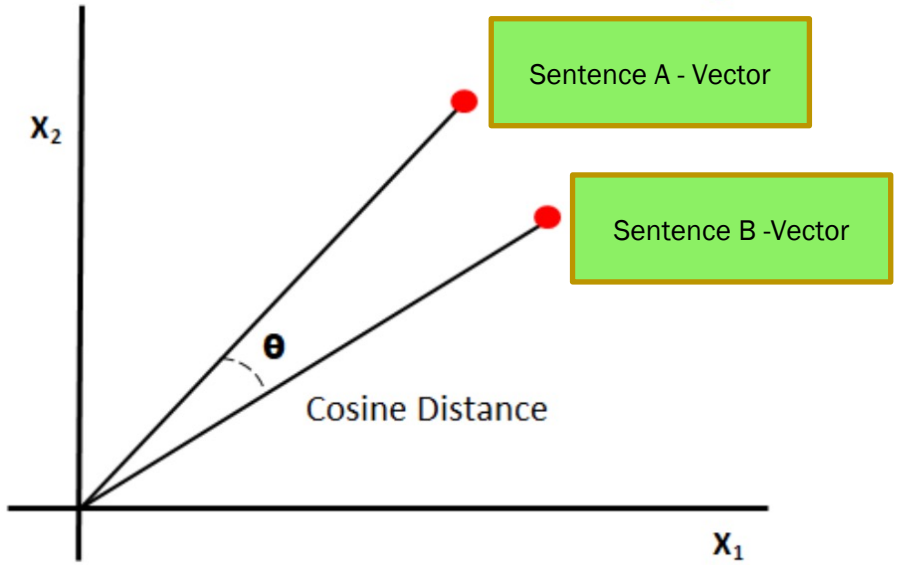
Hochschule Reutlingen

Encoder-Only Models for Vector Embeddings

- produced vector embeddings can be used for storing data in vector databases
- and computing similarity between data using cosine similarity:

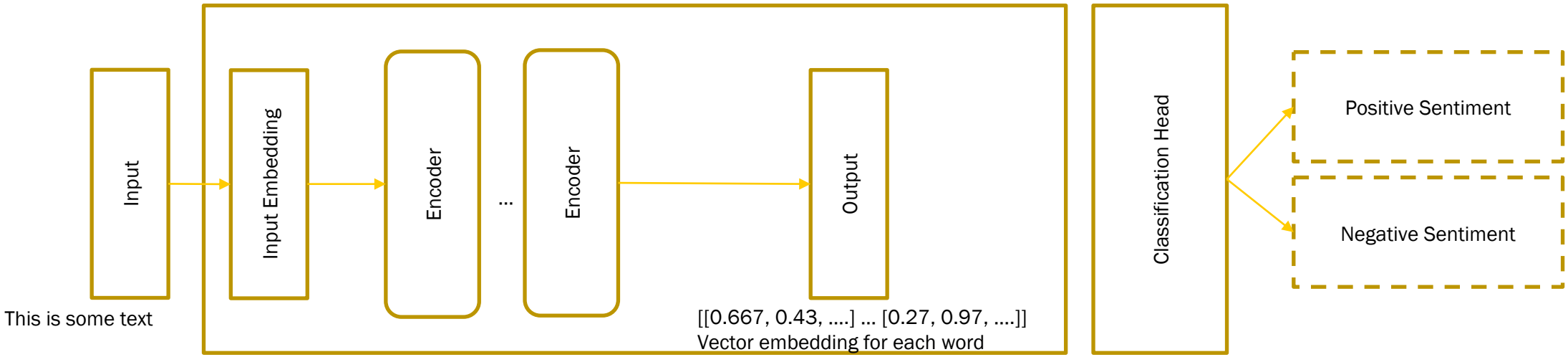


$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \cdot \sqrt{\sum_{i=1}^{n} B_i^2}}$$



Encoder-Only Models for Natural Language Understanding

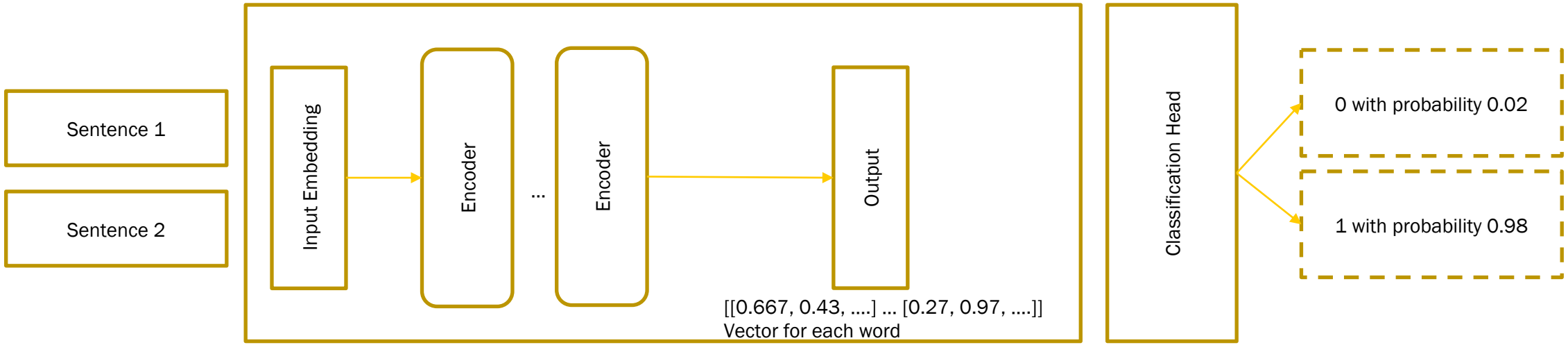
- BERT (Birectional Encoder Representations from Transformers) can be fine-tuned by adding a classifier layer for many language understanding tasks, such as text classification, sentiment analysis, named entity recognition



Hochschule Reutlingen

Encoder-Only Models for Semantic Similarity Computation

- CrossEncoder is used to measure similarity between pairs of sentences
- The input of the model always consists of a data pair, for example two sentences, and outputs a value between 0 and 1 indicating the similarity between these two sentences



Decoder-Only Models as Next Word Predictors

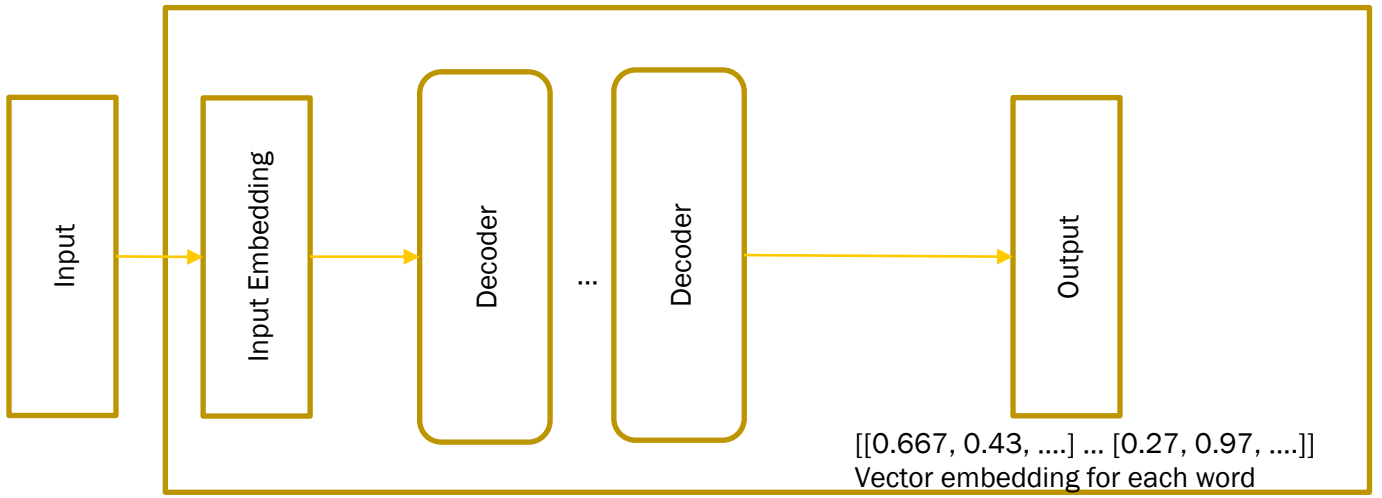
- Autoregressive Models, pre-trained using causal language modeling
- Objective is to predict the next word (token)
- They build a statistical representation of language
- Used for text generation
- Evolve emergent behavior depending on model size

Hochschule Reutlingen

Decoder-Only Models as Next Word Predictors

- GPT - Generative Pre-trained Transformer

„The theory of relativity usually encompasses two interrelated theories by Albert“



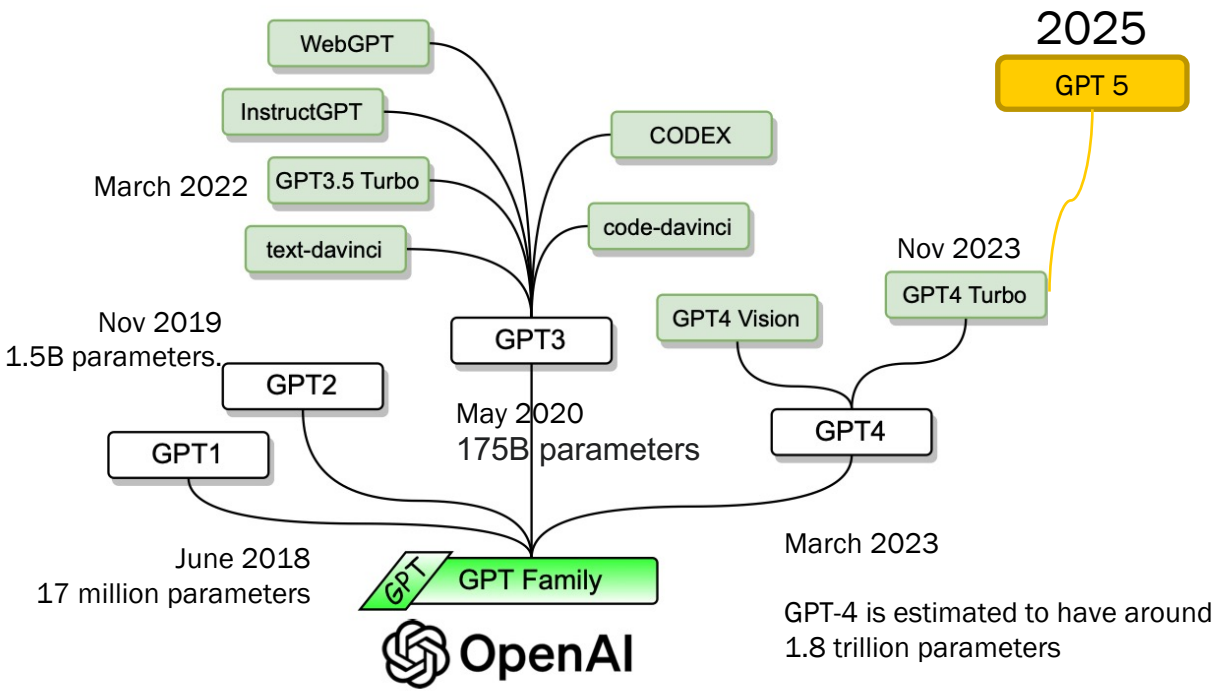
Produces vocabulary probabilities

0.957	Einstein
0.003	and
0.003	H
0.002	Spe
0.001	Hof
0.001	M
0.001	A
0.001	Jay
0.001	
0.001	de

The output layer typically consists of a linear transformation followed by a softmax function, which produces a probability distribution over the possible output words.

Transformer Architectures

Example: GPT Familiy



From: https://arxiv.org/pdf/2402.06196 Fig 8

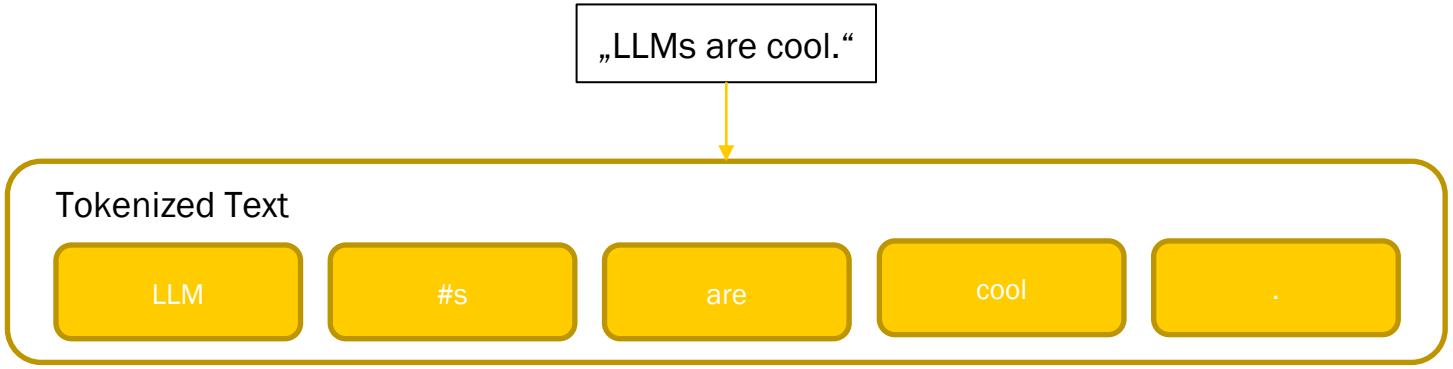
Model	Phase	Example
GPT 1- 3	**Predict** Continue a text string	"Write the next word in a story."
GPT 4	**Reason** Understand and infer	"Explain why this code is wrong."
GPT 4 -5	**Plan** Devise multi-step actions	"Outline steps to fix the bug."
GPT 5	Act Use tools / APIs / environment	"Run the code to verify the fix."

02.08
Tokenization & Generation

INF Informatik
Hochschule Reutlingen

Tokenization & Generation

- A token is just a word or sub-word in a sequence of text.
- Given a sequence of raw text as input. First step is allways to tokenize the raw text.
- We break it into a sequence of discrete tokens.

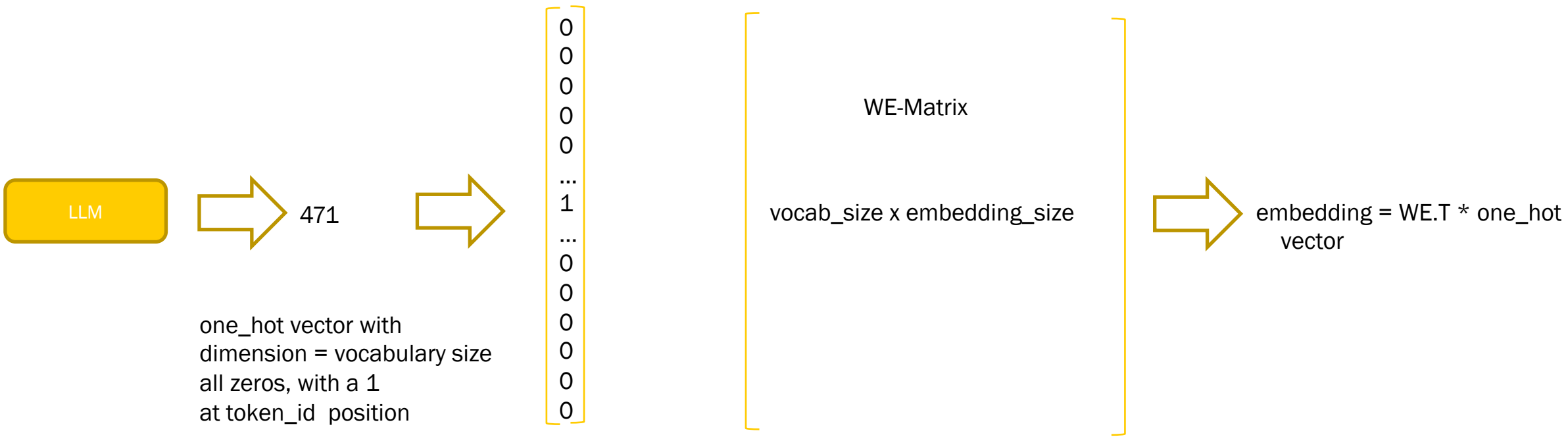


- This process is supported by a tokenizer. (For each LLM there is a special tokenizer).
- Tokenizer is trained over an unlabeled textual corpus to learn a fixed-size, unique set of tokens that exist: the vocabulary.
- The vocabulary contains all tokens that are known by the language model.

- Each token has an id in the vocabulary. LLM Has e.g. id 471

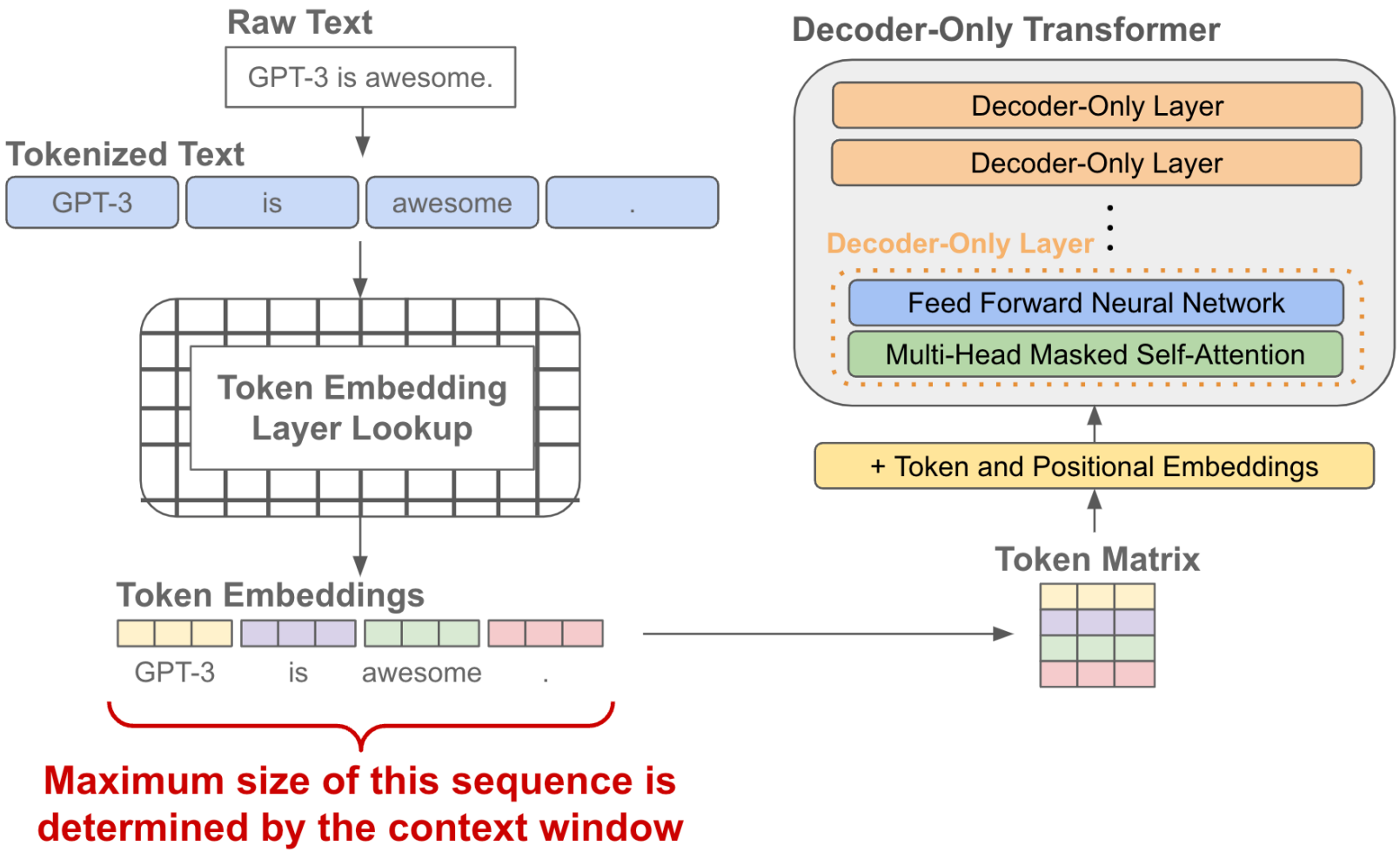
Tokenization & Generation

Once we have tokenized our raw text, we look up the embedding for each token within the embedding layer of the language model. The embedding layer is stored as part of the language model`s parameters. The result is a sequence of embedding vectors.



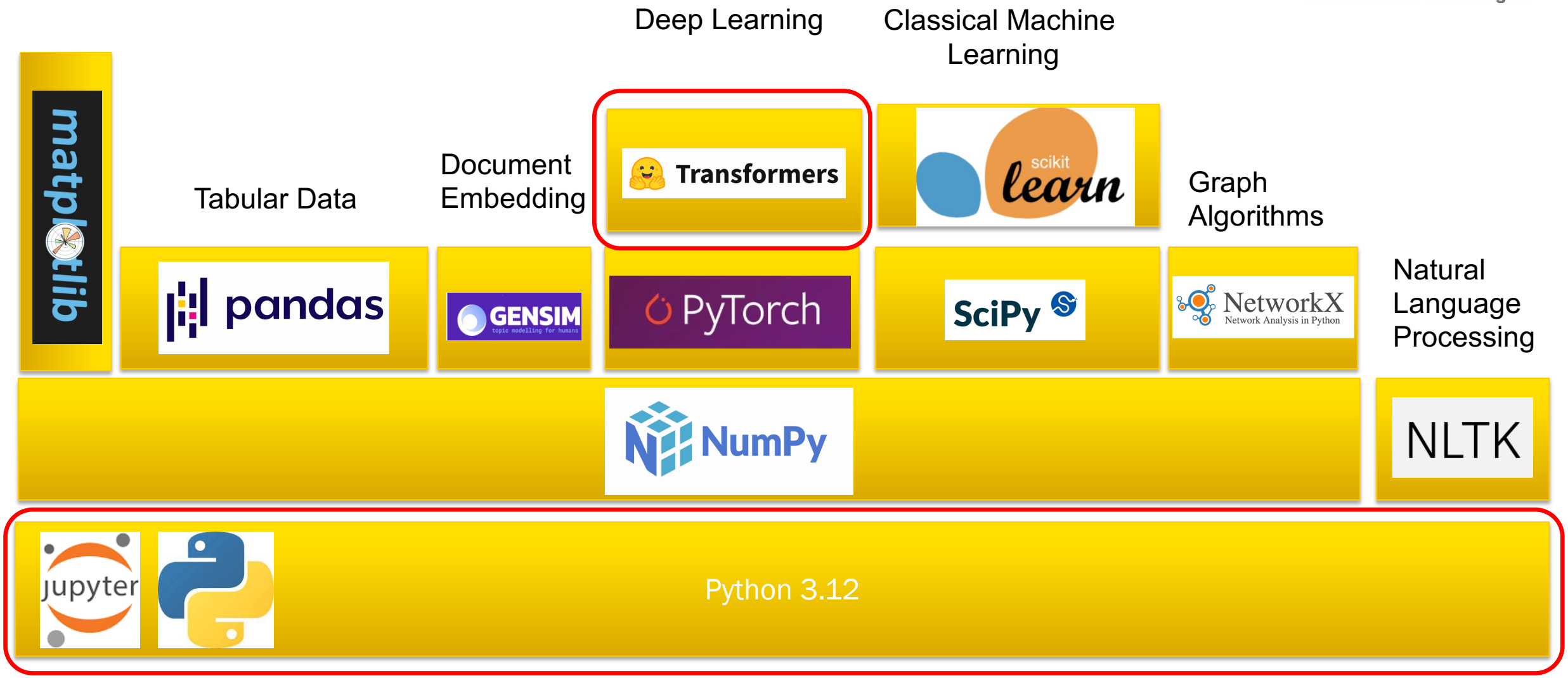
In addition a positional embedding is used to capture te position of the token.

Tokenization & Generation



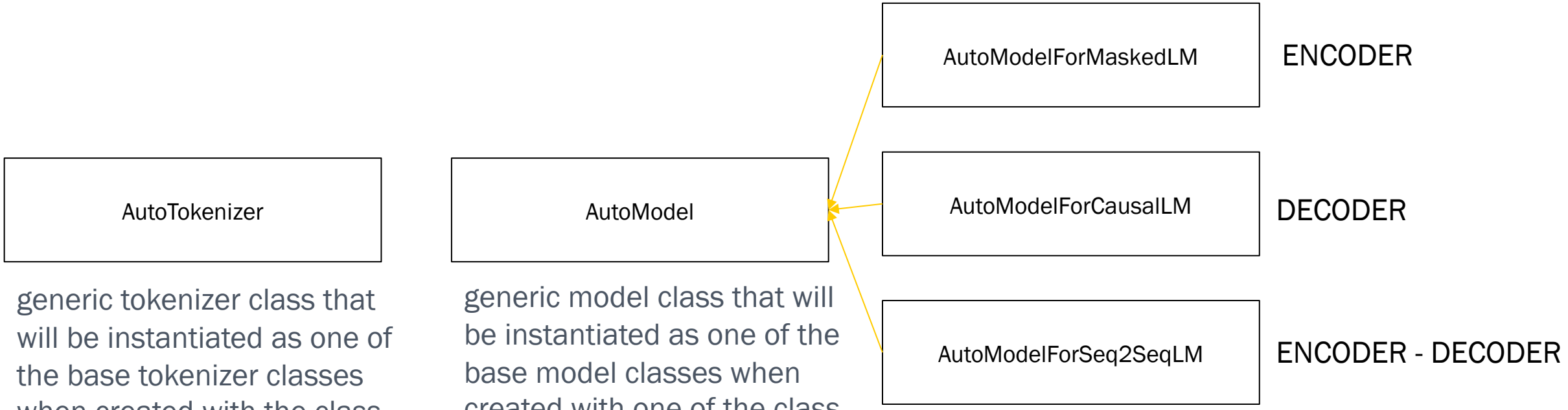
MACHINE LEARNING PYTHON STACK

Knowledge Foundation @ Reutlingen University
INF Informatik
Hochschule Reutlingen



Transformers

Transformers provides APIs and tools to easily download and train state-of-the-art large language models.



generic tokenizer class that will be instantiated as one of the base tokenizer classes when created with the class method:

- from_pretrained(model_id)

https://huggingface.co/docs/transformers/fast_tokenizers

generic model class that will be instantiated as one of the base model classes when created with one of the class methods:

- from_pretrained(model_id)
- from_config(config)

Source: https://pypi.org/project/transformers/

INF Informatik
Hochschule Reutlingen

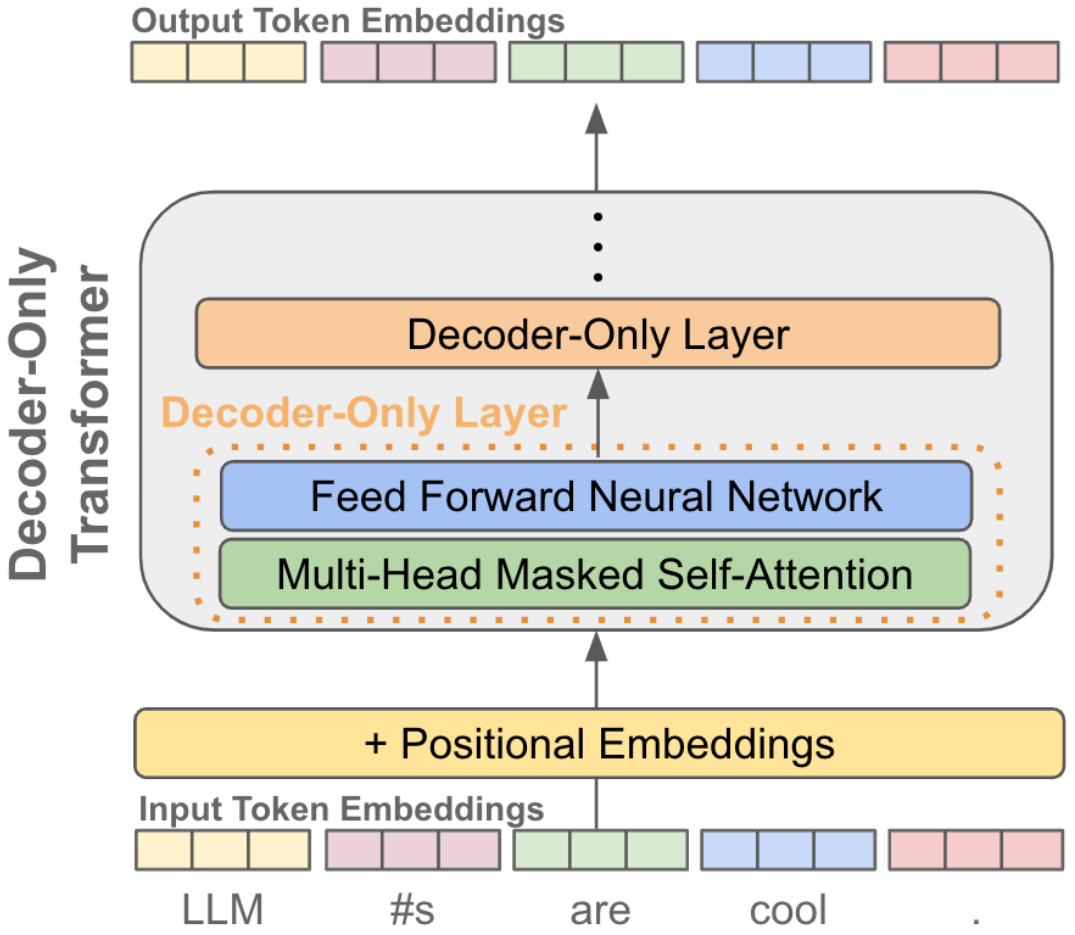
Tokenizer on Notebook

- Sentence Tokenization
- Token Embedding

```
pip install torch
pip install transformers
pip install tiktoken
```

Tokenization & Generation

Each Token Sequence is embedded and transformed into an output token embeddings.



Tokenization & Generation

Given an output vector for each token , we can perform next token prediction by

i. Taken the output vector for a token

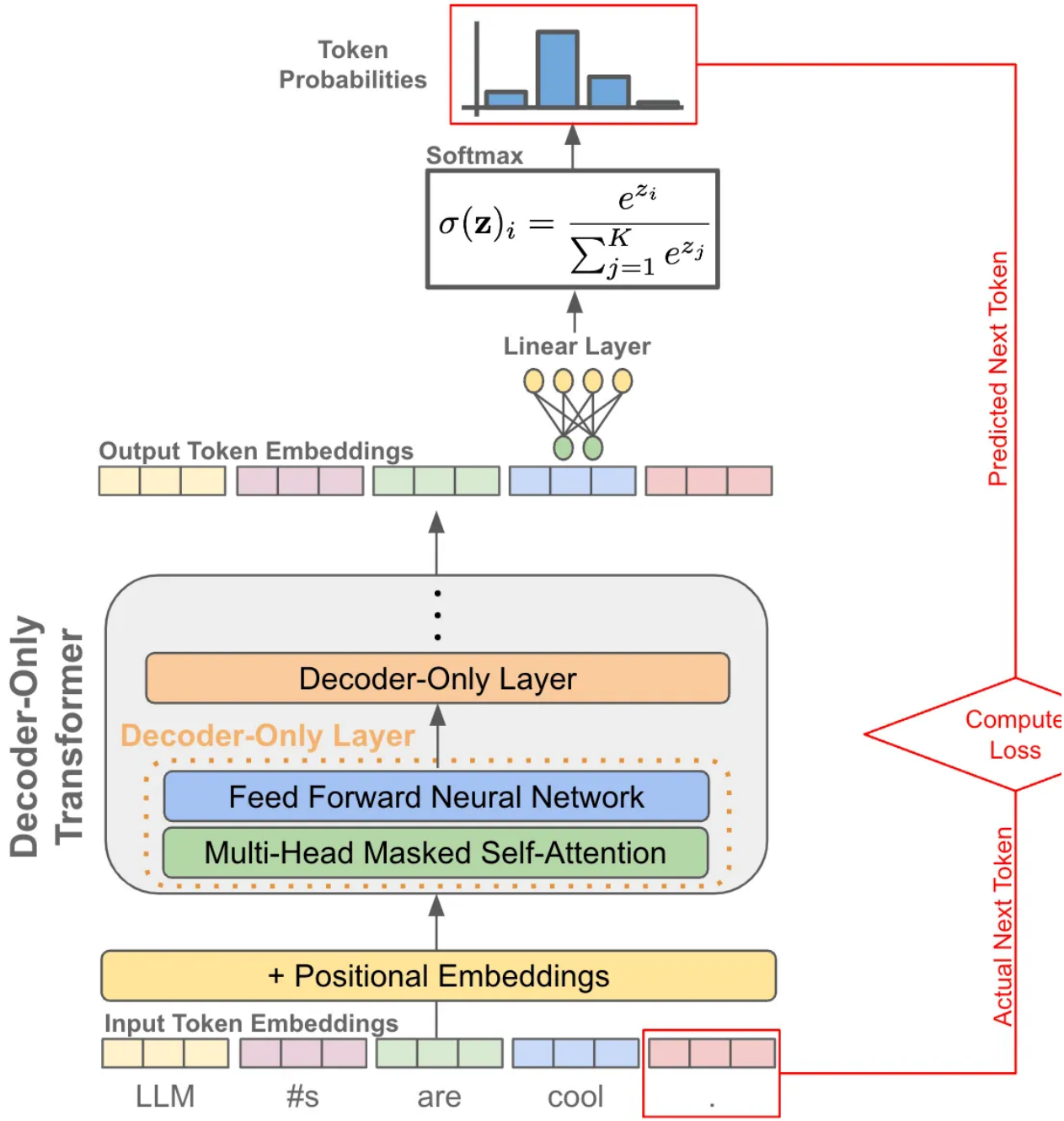
ii. Passing it to a linear layer. Which outputs a vector with same saize as our vocabulary.

iii. Applying a Softmax function to transform the vector items into probability distribution over vocabulary

Then we can either

i. Sample the next token out of the probability distribution.

ii. Or train the model to maximize the probability of the correct next token during pretraining



Tokenization & Generation

Around the model is a generate function, that controls the generation loop:

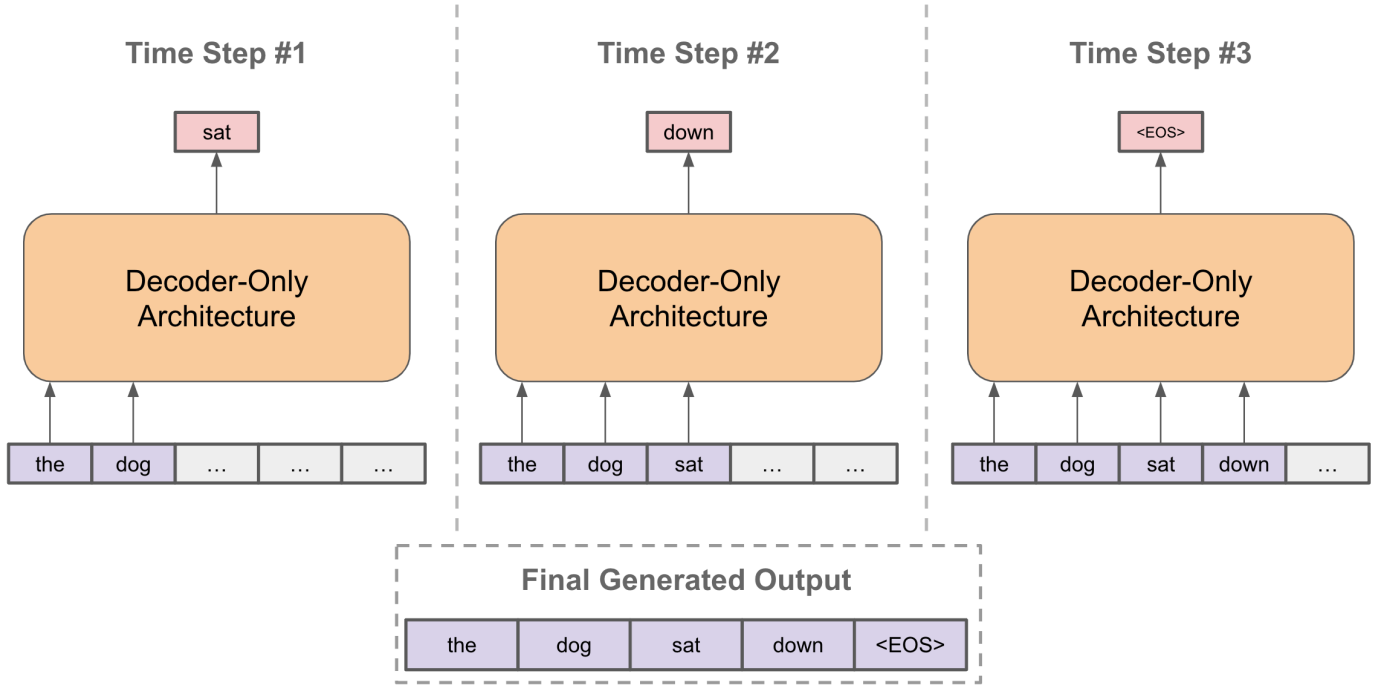
In this loop there are many ways to sample the next token from the given probability distribution.

Search strategies:

- Greedy search
- Beam search

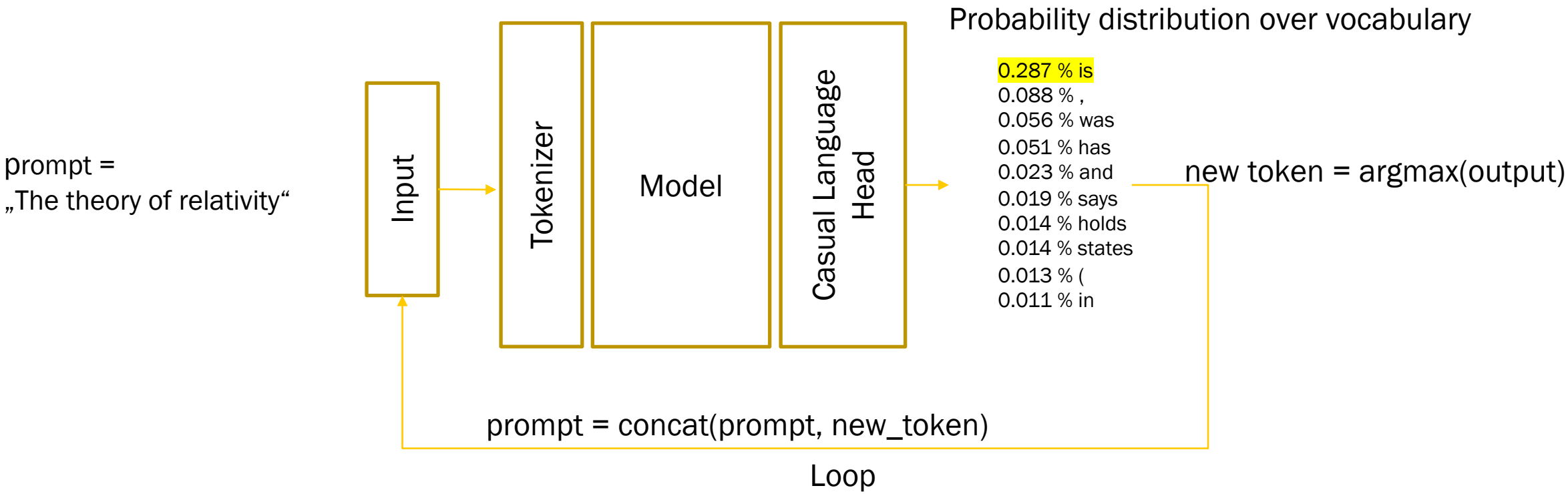
Sampling strategies:

- Nucleus sampling
- Top-k sampling
- Temperature



Search Strategies

Start with Greedy Search:

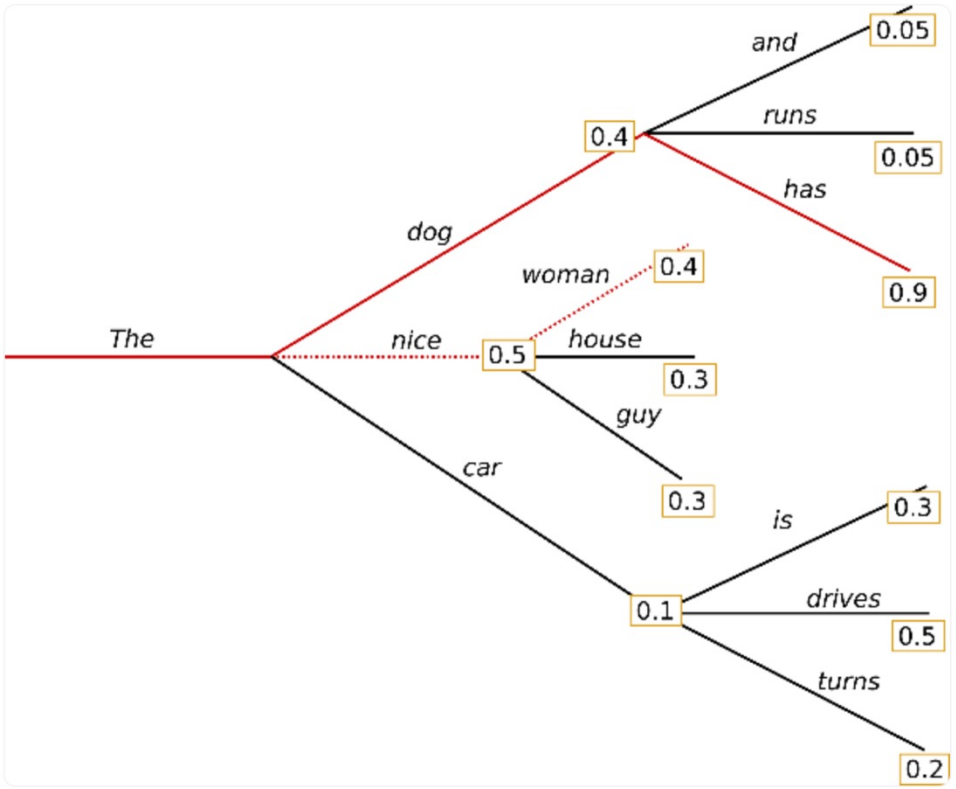


Search Strategies

Beam Search: reduces the risk of missing hidden high probability word sequences by keeping the most likely *num_beams* of hypotheses at each time step and eventually choosing the hypothesis that has the overall highest probability.

Let num_beams = 2:

Max computation time: vocab_size * vocab_size !



See: https://huggingface.co/blog/how-to-generate

Sampling Strategies

The drawbacks of and greedy and beam search approaches are, that we're picking the most probable choice. Instead of picking the most probable word from P(w | context), we could sample a word with P(w | context).

Now its time to add some randomness!

Even if P("eating" | "I am") > P("drinking" | "I am") we could still sample the word "drinking". Using sampling, we'll have a very high chance of getting a new sentence in each generation.

Sampling Strategies

There are 3 ways of sampling:

- Top-k Sampling:
  Instead of sampling from full P(w | context) (there are as many choices as words in the vocabulary), we only sample from top K words according to P(w | context).

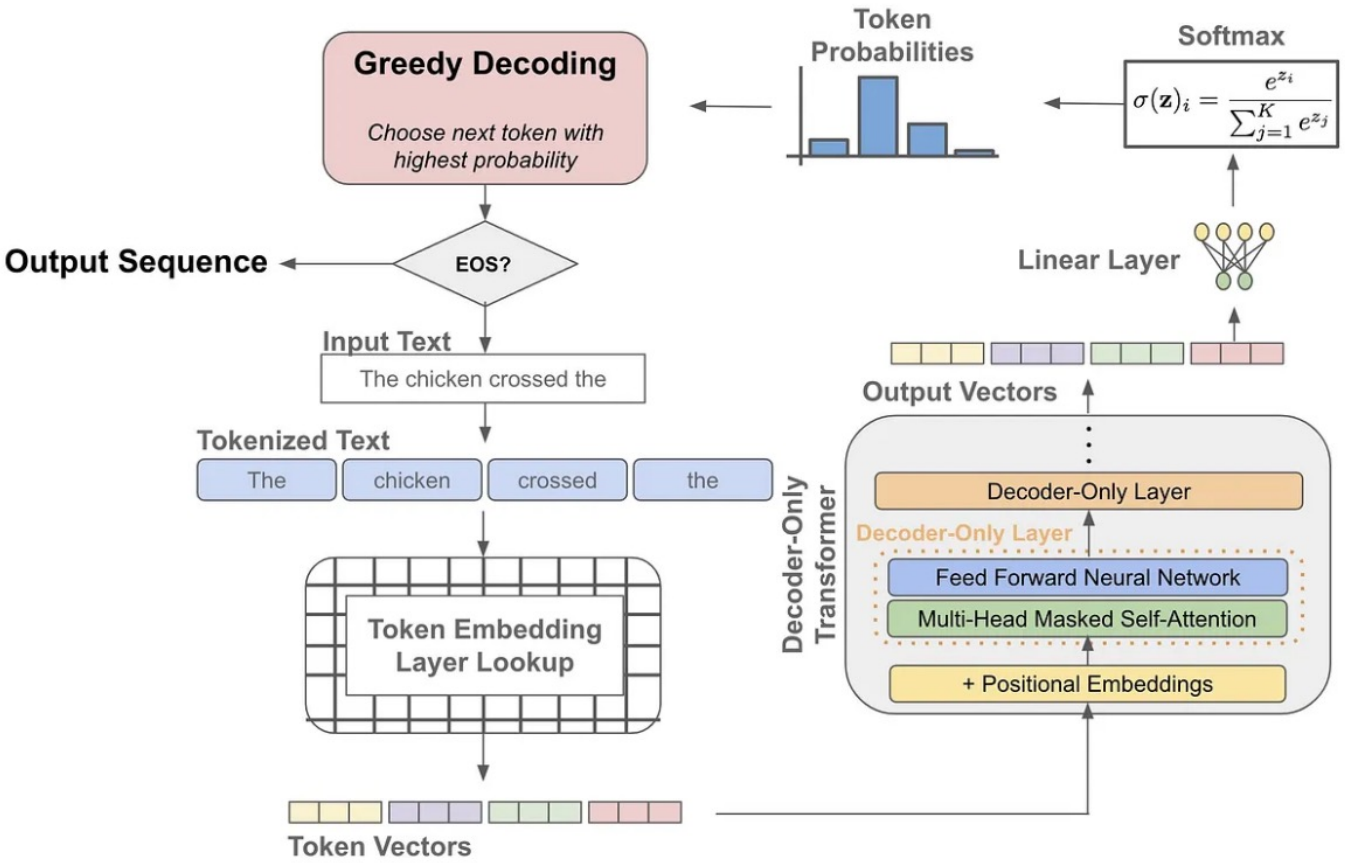
- Sampling with Temperature:
  It means we reshape the P(w | context) with a temperature factor t, where t is > 0. With temperature T we reshape the probability P(w | context) by dividing each logit value (output of the causal language head) by T before applying the softmax function. As T > 0, dividing it will amplify the logit value. Smaller values become more likely.

Sampling Strategies

- Top-p (nucleus) Sampling:

  Instead of sampling only from the most likely K words, in Top-p sampling chooses from the smallest possible set of words whose cumulative probability exceeds the probability p. The probability mass is then redistributed among this set of words.

Tokenization & Generation



Most LLMs: Decoder-only architectures

See: https://medium.com/@amansinghalml_33304/temperature-llms-b41d75870510

05_GPT2.ipynb

Understanding GPT2

see: https://huggingface.co/openai-community/gpt2

[1]:
```
import torch
```

[2]:
```
model_id = "gpt2"
```

Tokenization

[3]:
```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained(model_id, clean_up_tokenization_spaces=True)

########## Let have a look at the vocabulary of GPT-2 ##########
vocab = tokenizer.get_vocab() #key=subword, value=token_id
print(type(vocab))
print(len(vocab))
vocab_as_list = [subword for subword, _ in vocab.items()]
tokens_as_list = [token_id for _, token_id in vocab.items()]
token_dict = {token_id:subword for subword, token_id in vocab.items()}
```

```
<class 'dict'>
50257
```

[11]:
```
# vocab
```

[5]:
```
prompt = "The theory of relativity is"
```

[6]:
```
input_ids = tokenizer.encode(prompt)
input_ids
```

[6]:
```
[464, 4583, 286, 44449, 318]
```

INF Informatik
Hochschule Reutlingen

Show GPT2 on Notebook

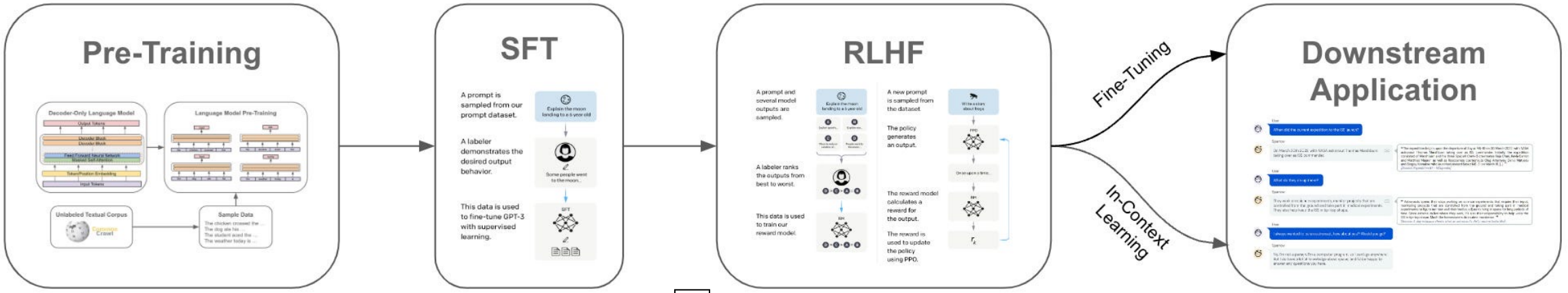
- Next Word Prediction
- Probabilities, Perplexity

02.09
Training

LLM-Training

Hochschule Reutlingen

Conventional LLMs typically undergo a 3-step training procedure:



Supervised Fine-Tuning (SFT)

Reinforcement Learning from Human Feedback

Foundation Models (Base Models)

Large, general-purpose models trained on broad, diverse data (internet text, books, code, etc.). They learn language and world knowledge.

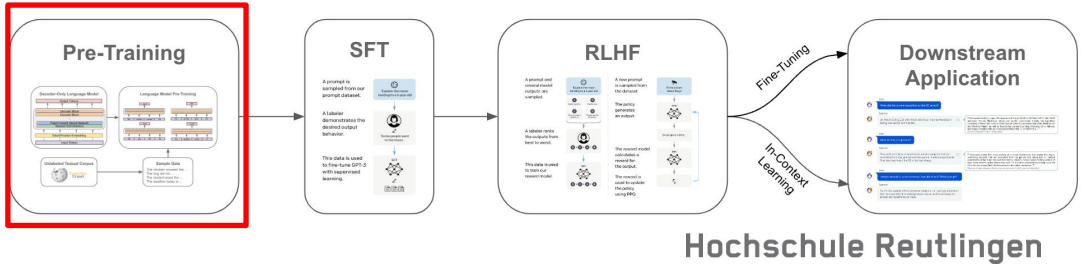
Goal: Be a platform that other specialized models can be built on.

Instruction-Tuned Models (Instruct / Chat Models)

Foundation models fine-tuned on datasets where humans give instructions and the model gives good answers.

Goal: Make the model follow instructions reliably, concisely, safely.
- Supervised Fine-Tuning (SFT): Human-generated instruction → ideal answer pairs
- Preference tuning (RLHF, DPO, ...) Models learn “which answer humans prefer.”

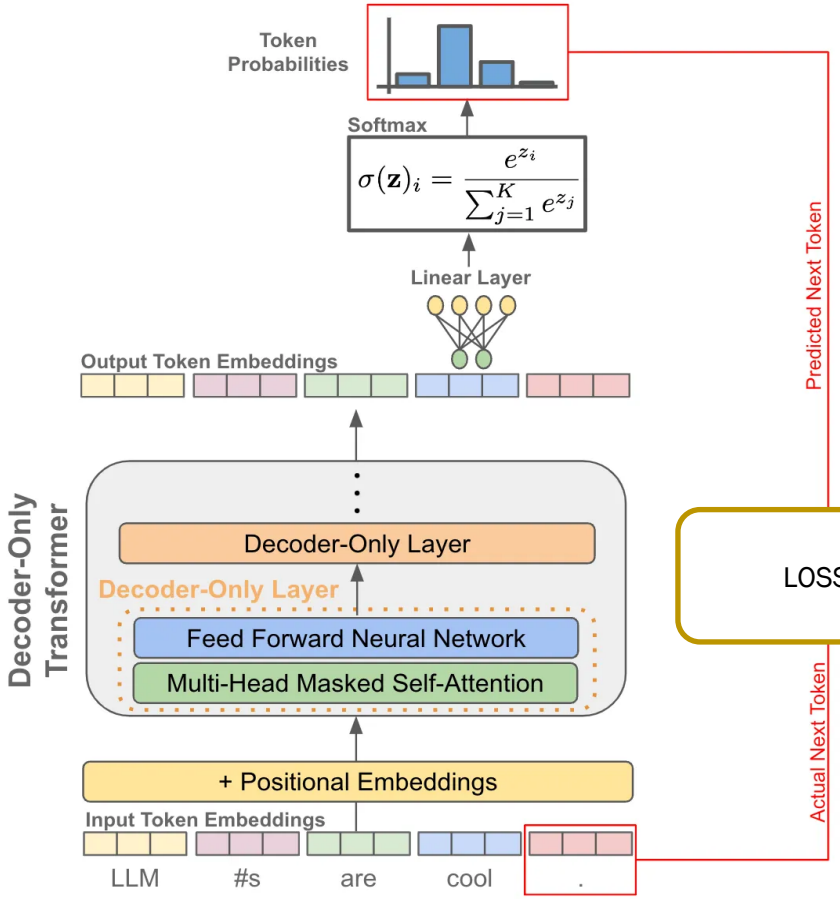
LLM-Training



Pre-Training

Training Loop

1. Take a text sequence (e.g., “LLMs are cool.”)
2. Mask future tokens so the model only sees the left context.
3. Predict the next token at every position.
4. Compute cross-entropy loss at each step.
5. Backpropagate and update parameters.



- Probability for next token is $p_\theta(x_t|x_{<t})$
- The goal in pretraining is to make the mode assign high probability to the training text
- We maximize the Likelihood

$$L(\theta) = \sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t})$$

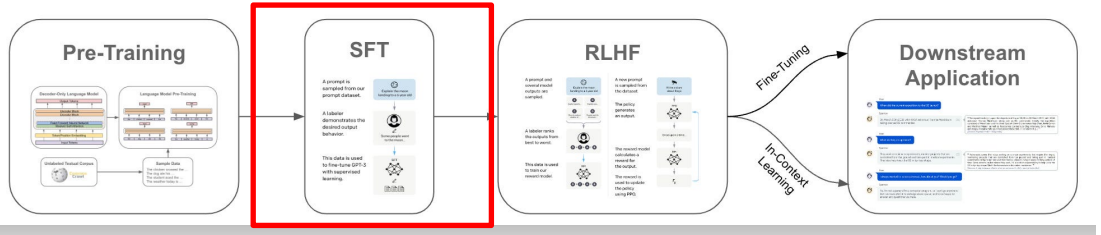
- This leads to minimize the cross entropy los

$$\mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t})$$

Intuition

- At each position t, the model outputs a distribution over the vocabulary.
- The true token is known → we penalize the model if it assigns low probability to it.

LLM-Training



Supervised Finetuning:

1. Start with Instruction Dataset

```
{
    "instruction": "Rewrite the following sentence using passive voice.",
    "input": "The team achieved great results.",
    "output": "Great results were achieved by the team."
},
```

Apply prompt style template (for example, Alpaca-style)

2. Apply prompt style template

Goes into the LLM

```
Below is an instruction that describes a task. Write a response
that appropriately completes the request.

### Instruction:
Rewrite the following sentence using passive voice.

### Input:
The team achieved great results.

### Response:
```

Expected output from LLM

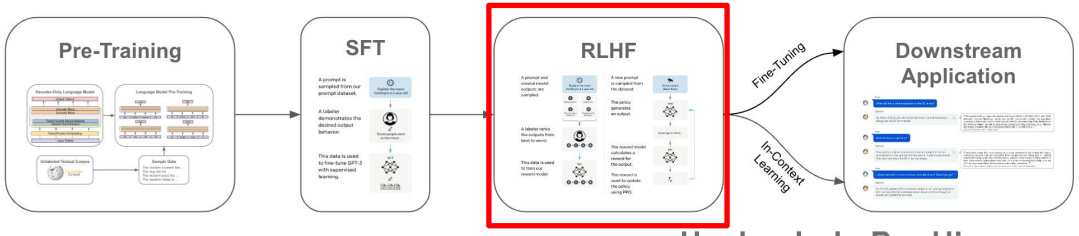
```
Great results were achieved by the team.
```

Pass to LLM for supervised instruction finetuning

LLM

Source: https://magazine.sebastianraschka.com/p/finetuning-large-language-models

LLM-Training



Reinforcement Learning from Human Feedback:

RLHF is usually done in three stages (RLHF-Pipeline):

LLM as policy

Base LLM → 1 Supervised fine-tuning (SFT) → Instruction tuned LLM → 2 Reward model training → Reward Model → 3 Reinforcement learning finetuning → Preference tuned LLM

- Knowledge
- Language Understanding

- Train a base LLM on high-quality, human-written instructions.

- Instruction Understanding

- Sample from LLM
- Collect pairs of model outputs, labeled by humans ("response A is better than response B").
- Train a reward model $R_\phi(x, y)$ to predict human preference

- Optimize the policy (the LLM) with reinforcement learning, using the reward model as the reward signal.
- Using Proximal Policy Optimization (PPO) in combination with Generalized Advantage Estimation

- Dialog like a human

Reinforcement Learning from Human Feedback

Stage 2 - Reward Model Training

A reward model assigns a scalar score $R_\phi(x, y)$ to a given prompt x and candidate response (or completion) y.

Unlike an LLM (which outputs probabilities over tokens), the reward model outputs a single number indicating how "good" the response is, according to human preference.

We turn the finetuned LLM into a reward model:

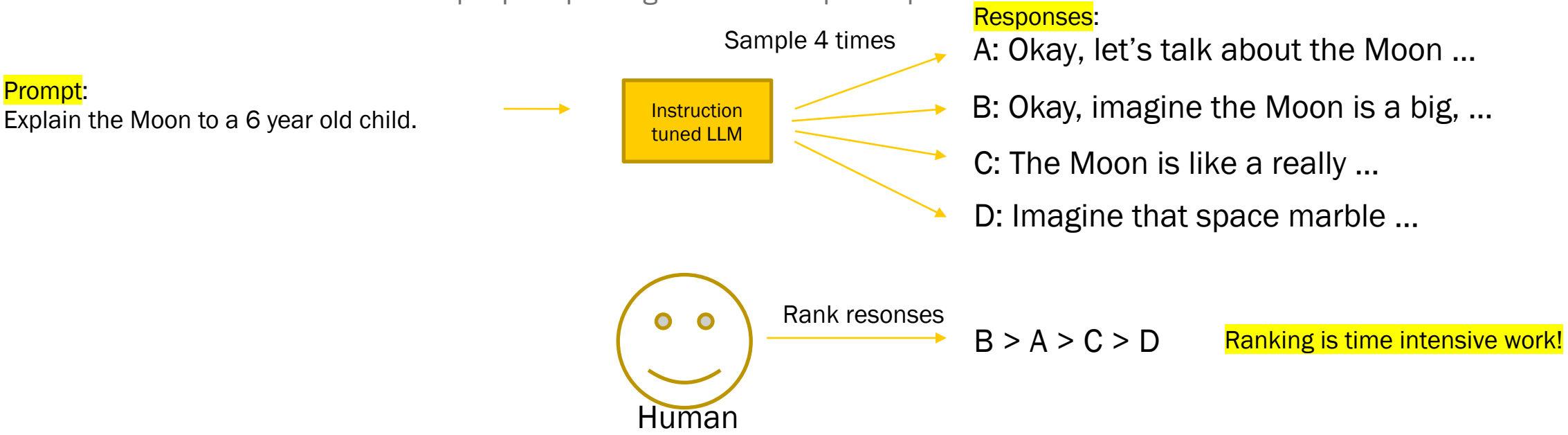
Reinforcement Learning from Human Feedback

Stage 2 - Reward Model Training

Generate a reward model (RM or preference model) calibrated with human preferences:

a reward model is a model that takes (prompt, response)-pair as input and outputs a reward/score.

1. Use the finetuned LLM and sample prompts to generate multiple responses



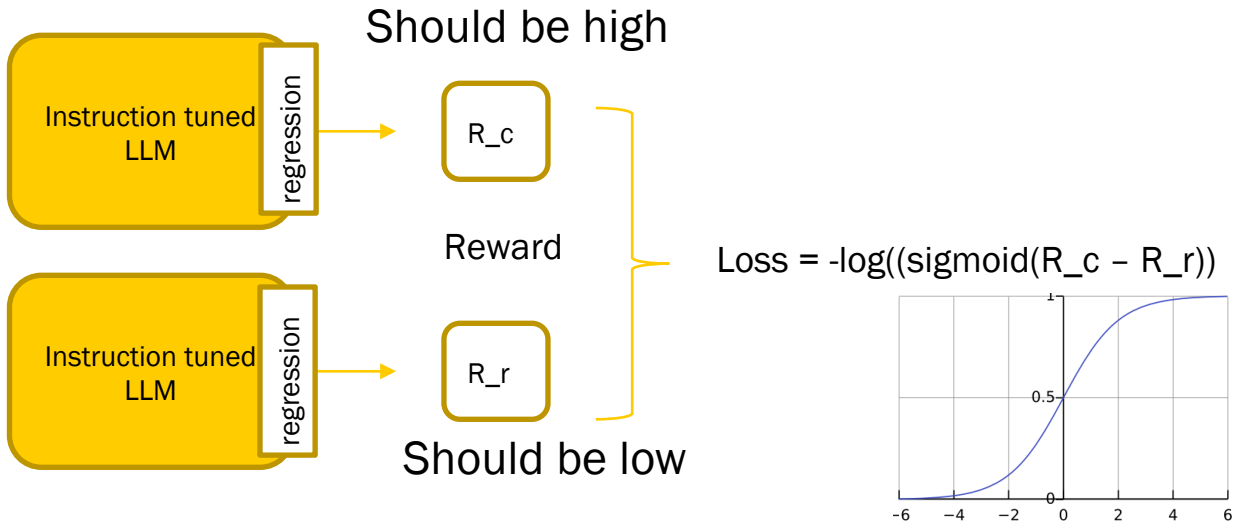
Reinforcement Learning from Human Feedback

Stage 2 - Reward Model Training

2. To train a reward model the comparison dataset should be in the format of (prompt, chosen response, rejected response)
   The reward model (RM) generally originates from the finetuned LLM created in the prior supervised fine-tuning (SFT) step.
   To turn the model into a reward model, its output layer (the next-token classification layer) is substituted with a regression layer, which features a single output node.

Triple:
- Prompt „Explain the Moon to a 6 year old child.“
- Reponse „Okay, imagine the Moon is a big, … “
- Label: Chosen

Triple:
- Prompt „Explain the Moon to a 6 year old child.“
- Reponse „ Imagine that space marble …“
- Label: Rejected



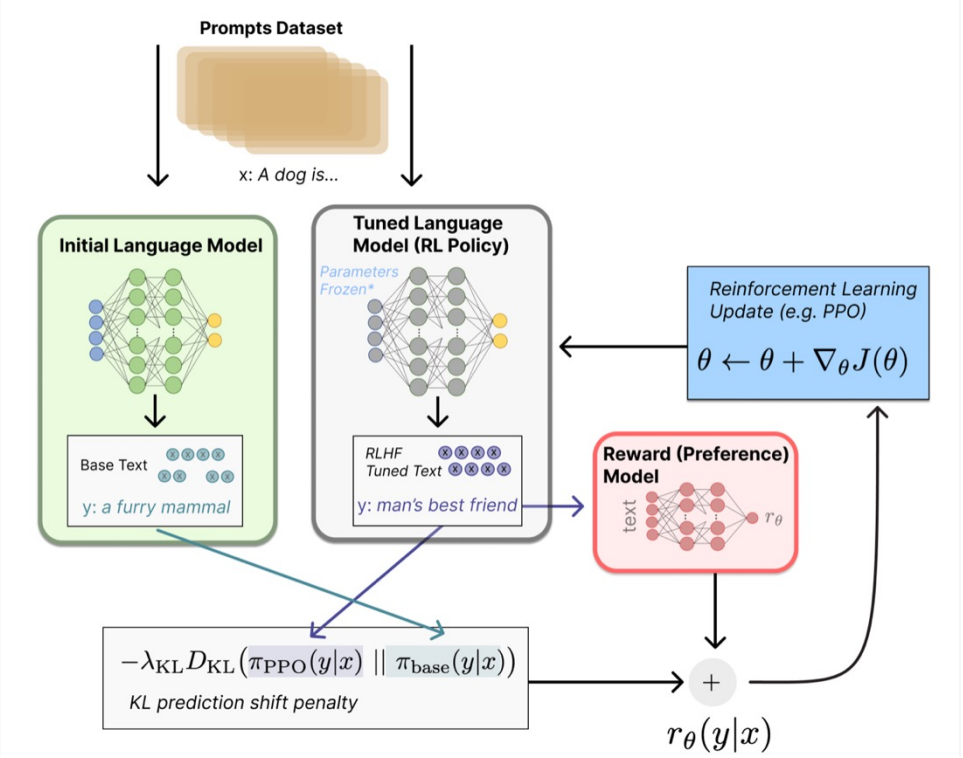
The intuition behind the loss function is to maximize the gap between chosen response score and rejected response score.
For a high reward score for chosen response R_c and a low reward score for rejected response R_r, the loss should be near zero: log(1) = 0.

Reinforcement Learning from Human Feedback

Stage 3 - Reinforcement Learning Finetuning

Use the reward model for finetuning with Reinforcement Learning
Assumptions:

- Probability Distribution of initial LLM and the tuned LLM should be similar (this is measured by Kullback–Leibler divergence)
  see: https://en.wikipedia.org/wiki/Kullback–Leibler_divergence
- Use reward model to penalize the tuned LLM if it produces not preferred answers



02.10
Scaling Laws and Emerging Abilities

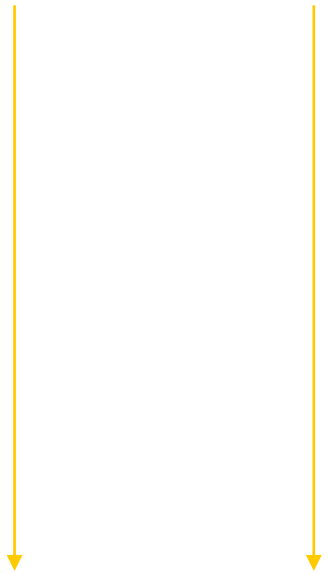
INF Informatik
Hochschule Reutlingen

Large Language Models

Understand Memory Consumption:

MEMORY NEEDED FOR INFERENCE

- 1 Parameter = 4 bytes (32-bit float)



- 1B parameters = 4 x 10^9 bytes = 4GB

6x

- 1B parameters = 24 x 10^9 bytes = 24GB

MEMORY NEEDED FOR TRAINING

	Bytes per parameter
Model Parameters	4 bytes
Adam optimizer (2 states)	+ 8 bytes
Gradients	+ 4 bytes
Activations & temporary memory	+ 8 bytes
TOTAL	4 bytes + 20 bytes

Large Language Models

Memory needed to train larger models

1B param model

24 GB

175B param model
(like GPT 3)

175 * 24 = 4,200 GB
@32-bit full precision

500B param model
(like PaLM)

500 * 24 = 12,000 GB
@32-bit full precision

Large Language Models

Pre-Training Costs

Model	Parameter Size	Data Scale	GPUs Cost	Training Time
GPT-3 (Brown et al., 2020)	175B	300B tokens	-	-
GPT-NeoX-20B (Black et al., 2022)	20B	825GB corpus	96 A100-40G	-
OPT (Zhang et al., 2022a)	175B	180B tokens	992 A100-80G	-
BLOOM (Scao et al., 2023)	176B	366B tokens	384 A100-80G	105 days
GLM (Zeng et al., 2023)	130B	400B tokens	786 A100-40G	60 days
LLaMA (Touvron et al., 2023a)	65B	1.4T tokens	2048 A100-80G	21 days
LLaMA-2 (Touvron et al., 2023b)	70B	2T tokens	A100-80G	71,680 GPU days
Gopher (Rae et al., 2022)	280B	300B tokens	1024 A100	13.4 days
LaMDA (Thoppilan et al., 2022)	137B	768B tokens	1024 TPU-v3	57.7 days
GLaM (Du et al., 2022)	1200B	280B tokens	1024 TPU-v4	574 hours
PanGu-α (Zeng et al., 2021)	13B	1.1TB corpus	2048 Ascend 910	-
PanGu-$\sum$ (Ren et al., 2023b)	1085B	329B tokens	512 Ascend 910	100 days
PaLM (Chowdhery et al., 2022)	540B	780B tokens	6144 TPU-v4	-
PaLM-2 (Anil et al., 2023)	-	3.6T tokens	TPUv4	-
WeLM (Su et al., 2023a)	10B	300B tokens	128 A100-40G	24 days
Flan-PaLM (Chung et al., 2022)	540B	-	512 TPU-v4	37 hours
AlexaTM (Soltan et al., 2022)	20B	1.3 tokens	128 A100	120 days
Codegeex (Zheng et al., 2023)	13B	850 tokens	1536 Ascend 910	60 days
MPT-7B (Team, 2023)	7B	1T tokens	-	-

From: Efficient Large Language Models: A Survey https://openreview.net/pdf?id=bsCCJHbO8A

Large Language Models

Scaling Laws

For large language models, test loss (cross-entropy) follows a smooth power-law decline as you scale:

$$L(N, D) \approx A * N^{-\alpha} + B * D^{-\beta} + \varepsilon$$

where:

- L is loss, proportional to log perplexity.
- N = model parameters
- D = dataset size
- α and β = scaling exponents
- ε = lower bound (infinite scale)
- A and B = empirical constants that determine the magnitude of the parameter-scaling term and the data-scaling term

$PPL = e^L$ these laws directly predict that perplexity falls as you scale.

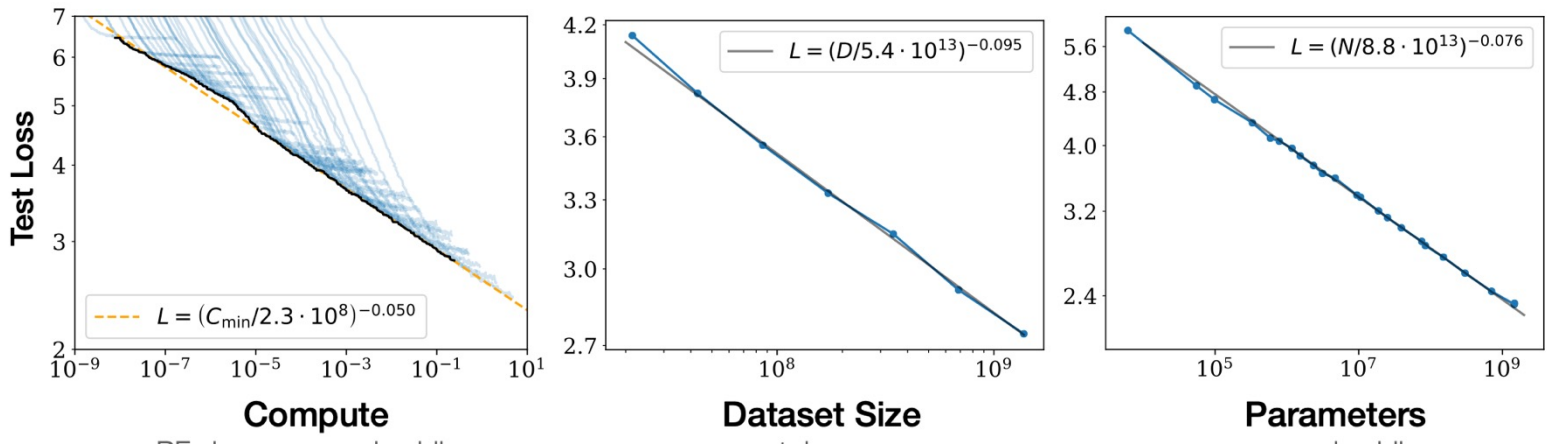
See: https://arxiv.org/pdf/2203.15556

Scaling Laws

Model Performance depends strongly on scale, weakly on model shape (architecture)

Scale consists of three factors:

1. Number of model parameters
2. Size of dataset (tokens)
3. Amount of compute used for training (amount of backpropagation steps to adjust the parameters)



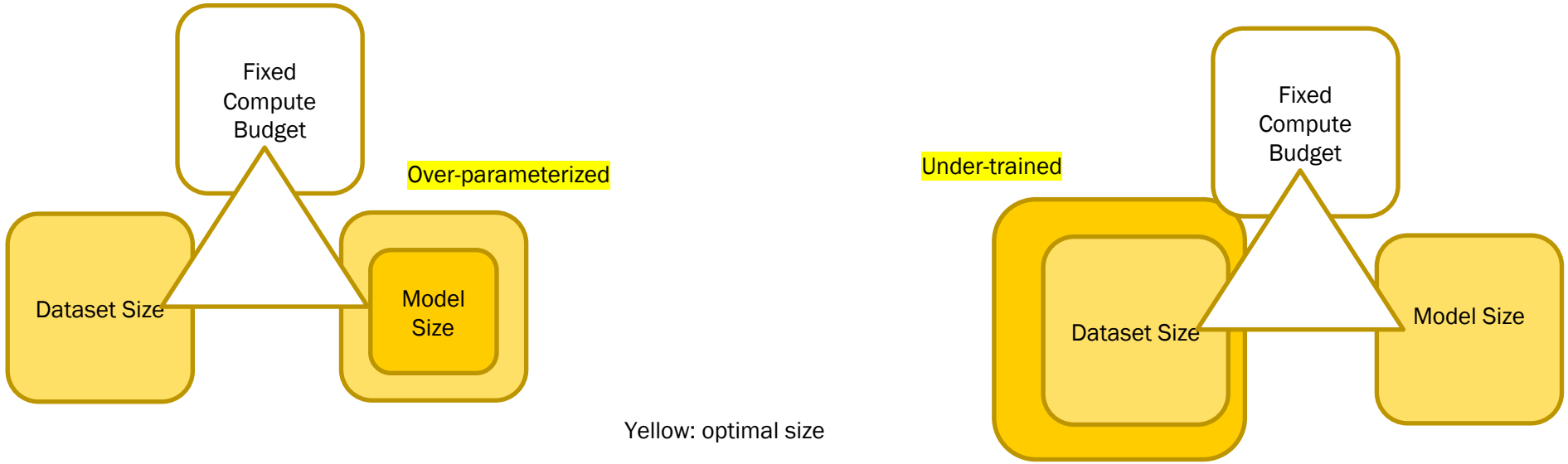
See: https://arxiv.org/abs/2001.08361

Large Language Models

Scaling Laws

The quest for ideal balance between training dataset size, model size, and compute budget continues to shape the development of large language models.

- Very large models may be over-parameterized and under-trained
- Smaller models trained on more data could perform as well as large models



See: https://cthiriet.com/blog/scaling-laws

Large Language Models

As models grow in size, they exhibit **emerging properties** - Why?

Large language models can exhibit unpredictable ("emergent") jumps in capability as they are scaled up.

Emergence refers to the capabilities of LLMs that appear suddenly and unpredictably as model size, computational power, and training data scale up.

- "Emergence is when quantitative changes in a system result in qualitative changes in behavior."
  (rooted back to a 1972 essay called "More Is Different" by Nobel prize-winning physicist Philip Anderson)

- Emergent Abilities of LLMs:
  "An ability is emergent if it is not present in smaller models but is present in larger models."
  (definition from Emergent Abilities of Large Language Models in https://arxiv.org/abs/2206.07682)

From https://cset.georgetown.edu/article/emergent-abilities-in-large-language-models-an-explainer

Large Language Models

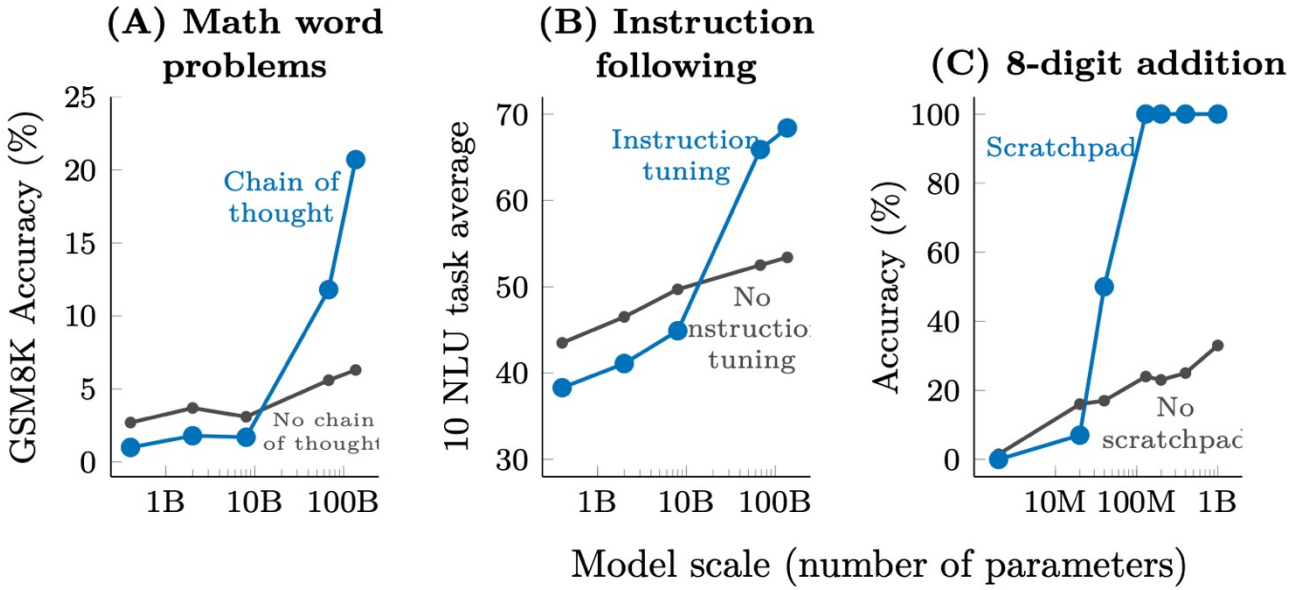
As models grow in size, they exhibit emerging properties - Why?

LLM exhibit ("emergent") jumps in capability as they are scaled up.

A: **chain of thought prompting** surpasses standard prompting without intermediate steps when scaled up to 100B parameter

B: By **finetuning** on a mixture of tasks phrased as instructions, language models have been shown to respond appropriately to instructions describing an unseen task (scaled up to 8B)

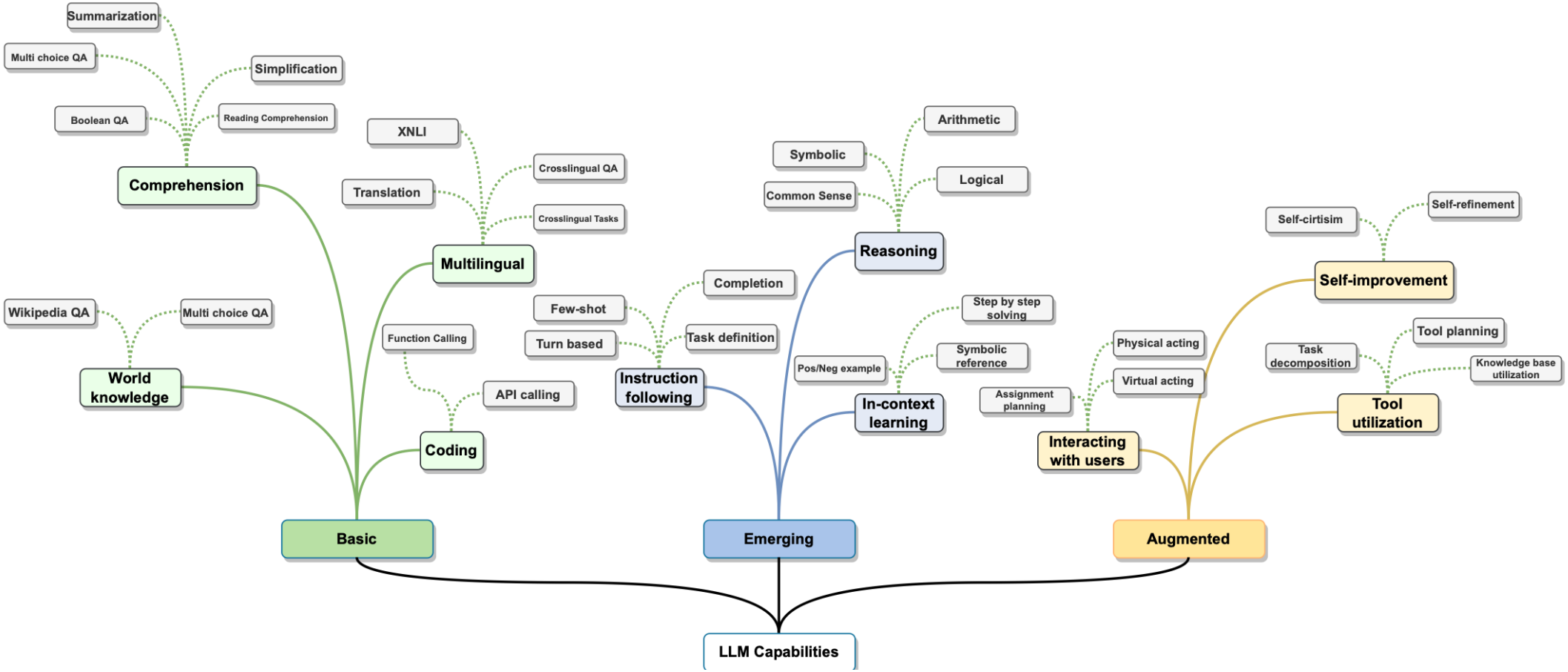
C: For computational tasks involving multiple steps Models use intermediate outputs (“scratchpad”). On 8-digit addition, using a scratchpad only helps for models of 40M parameters or larger



https://arxiv.org/abs/2206.07682

Large Language Models

With growing model size more and more capabilities emerge



From: https://arxiv.org/pdf/2402.06196 Fig-1

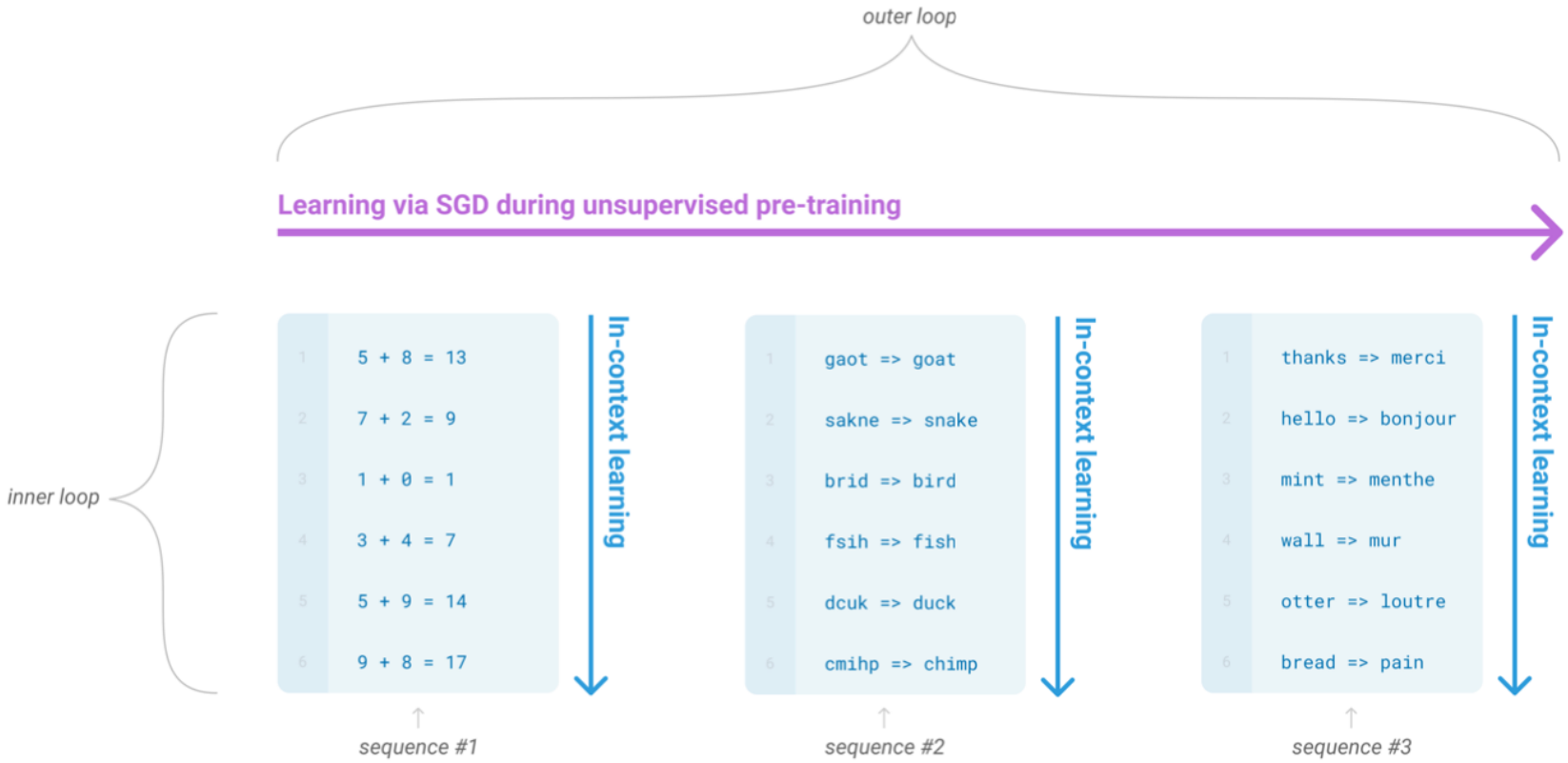
Large Language Models

In-Context Learning:

During unsupervised pre-training, a language model develops a broad set of skills and pattern recognition abilities.

It then uses these abilities at inference time to rapidly adapt to or recognize the desired task.

We use the term "in-context learning" to describe the inner loop of this process, which occurs within the forward-pass upon each sequence.



From: GPT-3 Language Models are Few-Shot Learners, https://arxiv.org/abs/2005.14165

Large Language Models

Larger models make increasingly efficient use of in-context information

The three settings we explore for in-context learning

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
Translate English to French:        ← task description
cheese =>                           ← prompt
```

One-shot

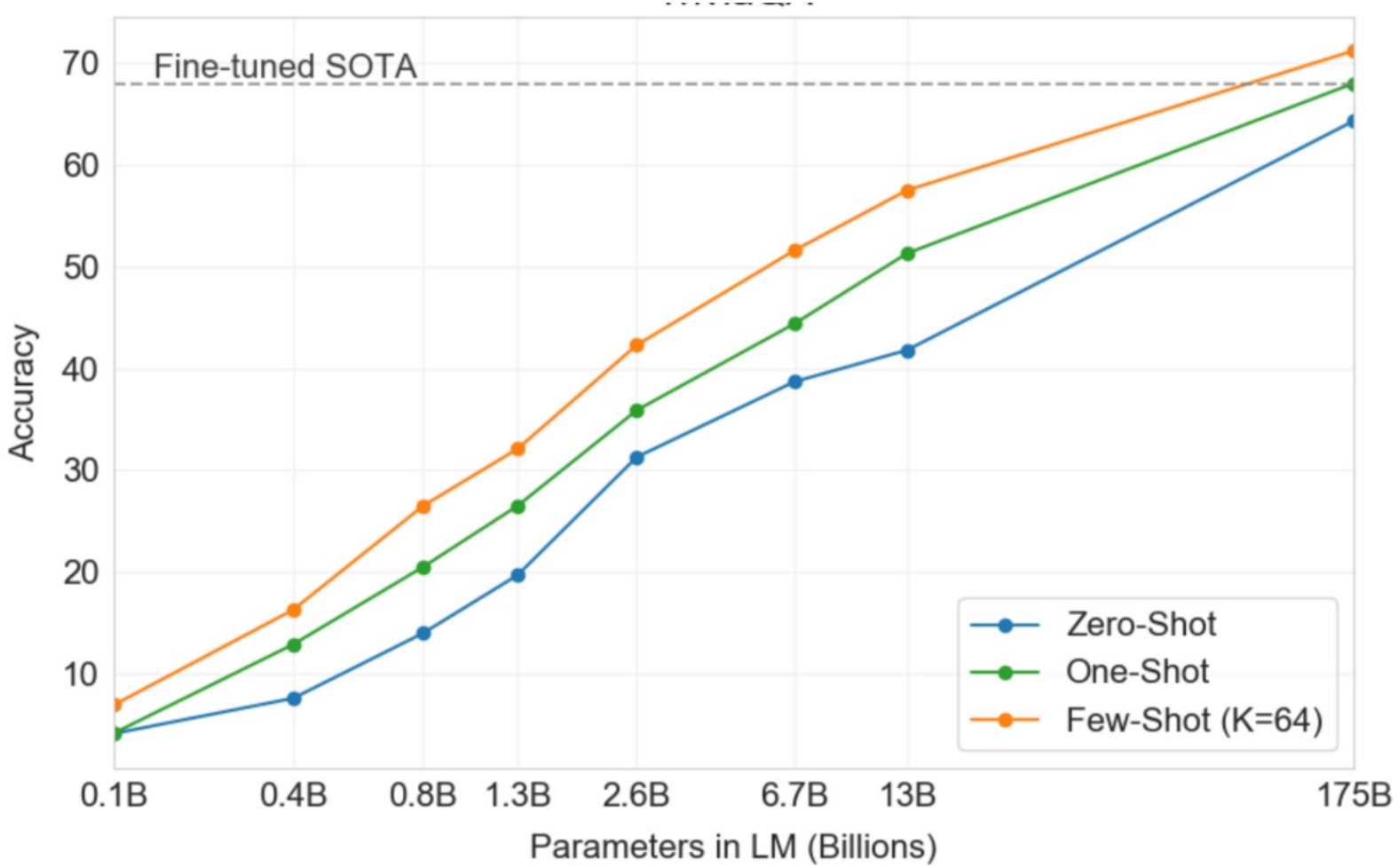
In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
Translate English to French:        ← task description
sea otter => loutre de mer          ← example
cheese =>                           ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

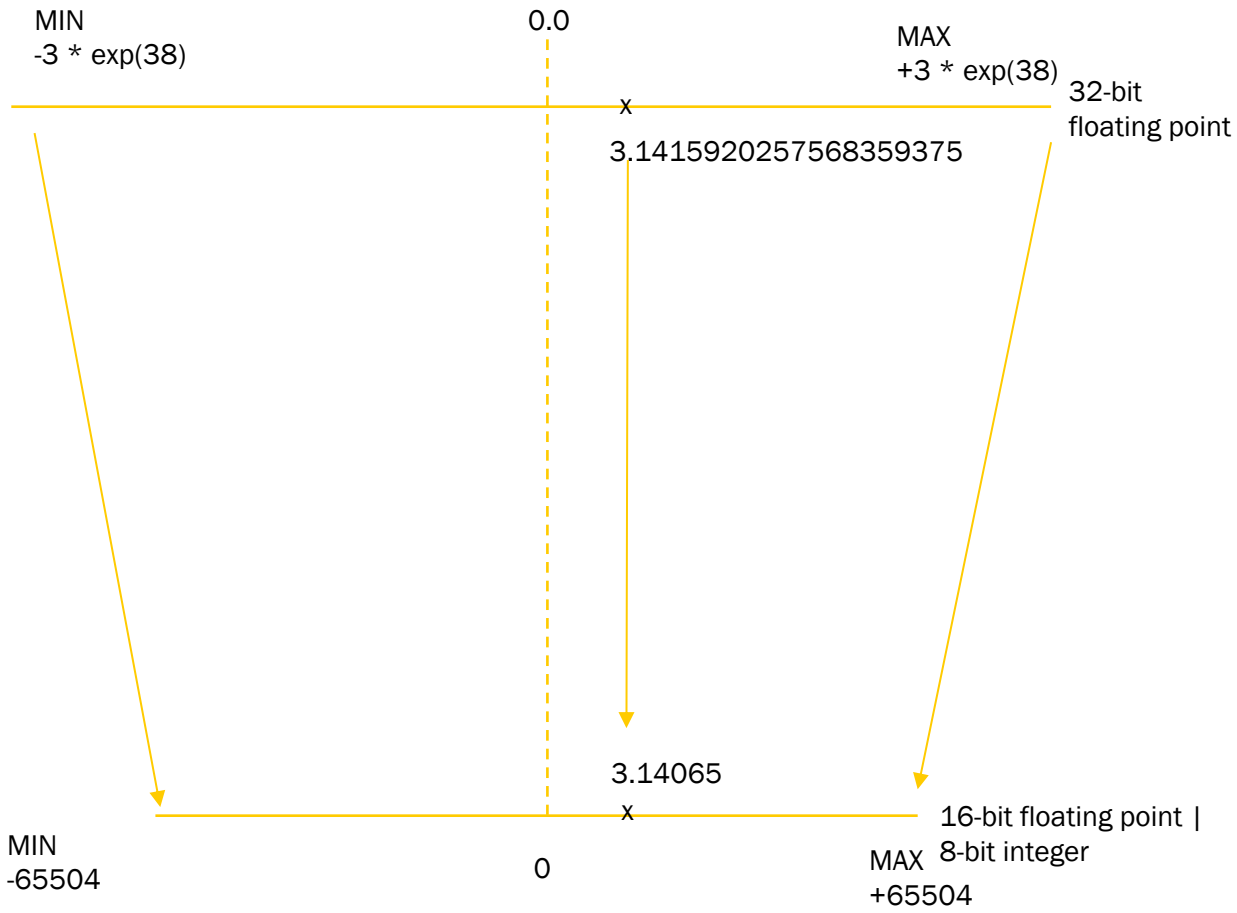
```
Translate English to French:        ← task description
sea otter => loutre de mer          ← examples
peppermint => menthe poivrée
plush girafe => girafe peluche
cheese =>                           ← prompt
```



From: GPT-3 Language Models are Few-Shot Learners, https://arxiv.org/abs/2005.14165

Large Language Models

Quantization:



Quantization is a technique to reduce the computational and memory costs of running inference by representing the weights and activations with low-precision data types like 8-bit integer (int8) instead of the usual 32-bit floating point (float32).

Quantization statistically projects the 32-bit floating point numbers into a lower precision space using scaling factors

Let's store Pi = 3.141592....

FP32	0	10000000	10010010000111111011000
	Sign 1 bit	Exponent 8 bits	Fraction 23 bits
FP16	0	10000	1001001000
	Sign 1 bit	Exponent 5 bits	Fraction 10 bits

Large Language Models

Hochschule Reutlingen

Quantization:

	Bits	Exponent	Fraction	Memory needed to store one value
FP32	32	8	23	4 bytes
FP26	16	5	10	2 bytes
BFLOAT16	16	8	7	2 bytes
INT8	8		7	1 byte

Performing quantization to go from float32 to int8 is tricky. Only 256 values can be represented in int8, while float32 can represent a very wide range of values. The idea is to find the best way to project our range [a, b] of float32 values to the int8 space.

For x (as float32) in [a,b]:

$x = S * (x_q - Z)$ with

x_q is the quantized int8 value associated to x, S and Z are quantization parameters, $x_q = round(x/S + Z)$

From: https://huggingface.co/docs/optimum/concept_guides/quantization

02.11
Prompt Engineering

INF Informatik
Hochschule Reutlingen

Large Language Models

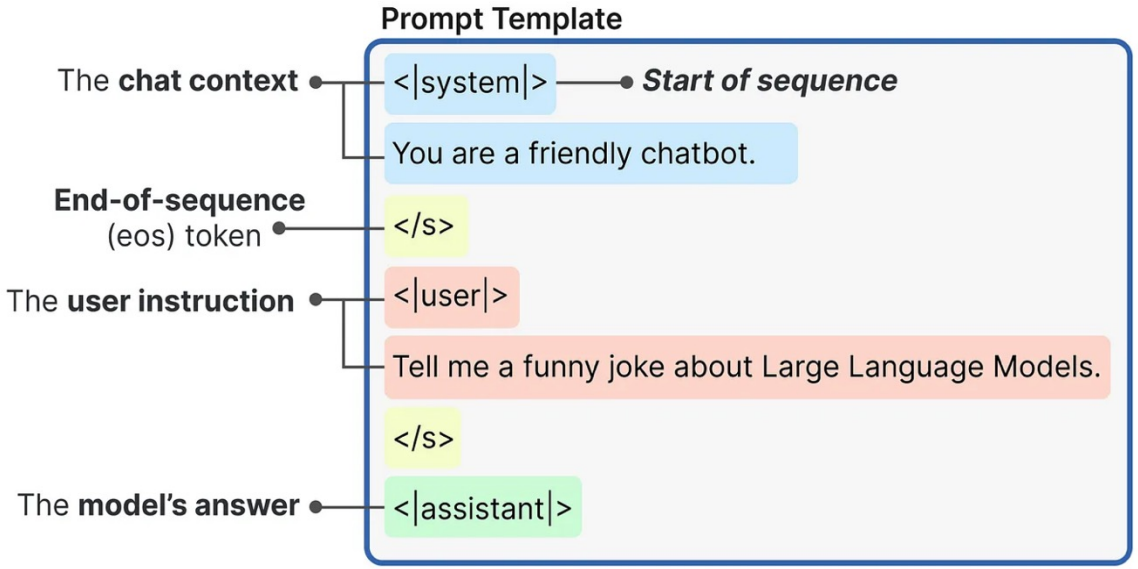
Prompt Engineering

What is a Prompt?

- A prompt is an input to a generative AI model, that is used to guide its output.
- Prompts may consist of text, image, sound, or other media
- Prompts are often constructed via a prompt template:

  A prompt template is a function that contains one or more variables which will be replaced by some media(usually text) to create a prompt. This prompt can then be considered to be an instance of the template.

  A prompt template becomes a prompt when input is inserted into it.



Prompt Engineering

Components of a Prompt

Prompt Component		Example
Directive	Many prompts issue a directive in the form of an instruction or question.	Write a poem about trees.
Output Formatting	Return out in a specific fomat: for example, JSON, CSVs or markdown	List all tree species as CSV.
Style Instructions	Style instructions are a type of output formatting used to modify the output stylistically rather than structural.	Write a clear and curt paragraph about llamas.
Role	A Role, also known as a persona, is a frequently discussed component that can improve writing and style text	Pretend you are a shepherd and write a limerick about llamas.
Additional Information (or context)	It is often necessary to include additional information to provide and set context.	

Large Language Models

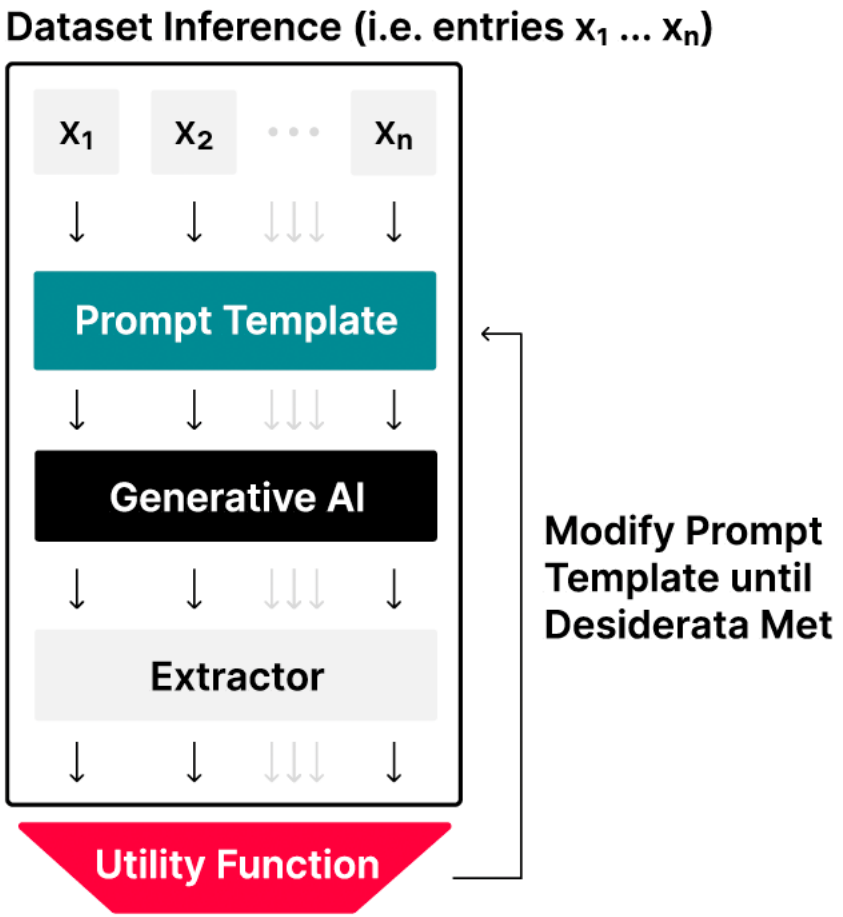
Prompt Engineering

The Prompt Engineering Process consists of

three repeated steps:

1. performing inference on a dataset
2. evaluating performance and
3. modifying the prompt template.

Note that the extractor is used to extract a final response from the LLM output



Source: [PET] Figure 1.4

Large Language Models

Prompt Engineering

Hochschule Reutlingen

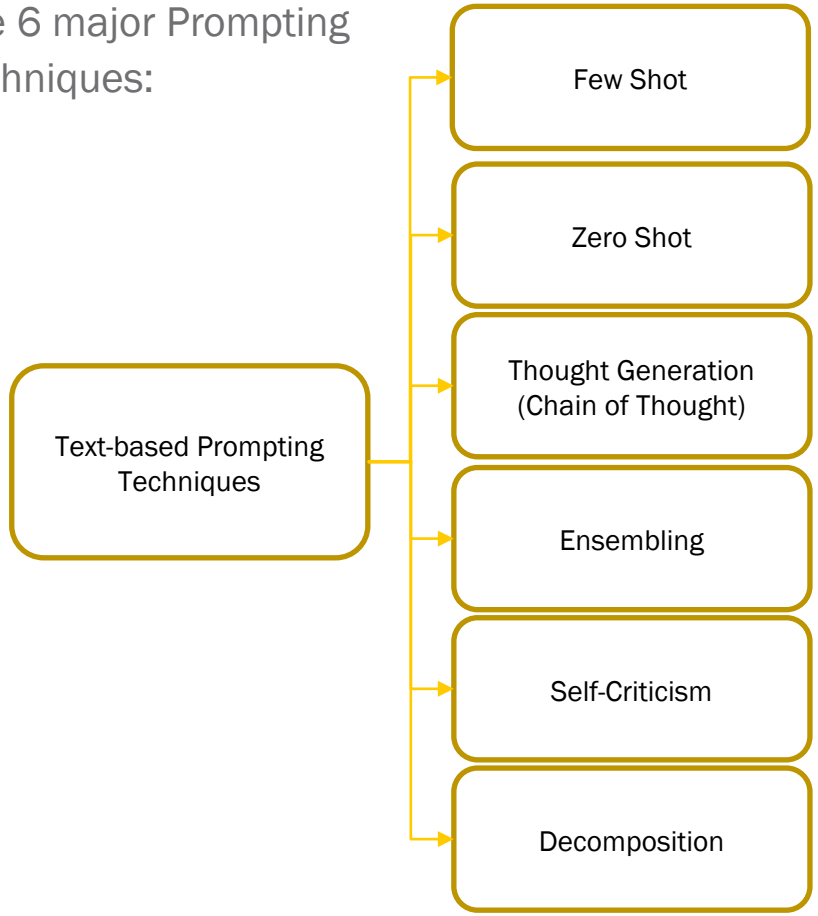
In-Context Learning (ICL):

- ICL refers to the ability of GenAIs to learn skills and tasks by providing them with exemplars and or relevant instructions within the prompt, without the need for weight updates/retraining.
- These skills can be learned from exemplars and/or instructions.
- The word 'learn' is misleading. ICL can simply be task specification – the skills are not necessarily new, and can have already been included in

Source: [PET] Figure 2.3

Prompt Engineering

The 6 major Prompting Techniques:



is the paradigm seen where the GenAI learns to complete a task with only a few examples.

in contrast to Few-Shot Prompting, Zero-Shot Prompting uses zero exemplars.

Thought generation encompasses a range of techniques that prompt the LLM to articulate its reasoning while solving a problem.

In GenAI, ensembling is the process of using multiple prompts to solve the same problem, then aggregating these responses into a final output.

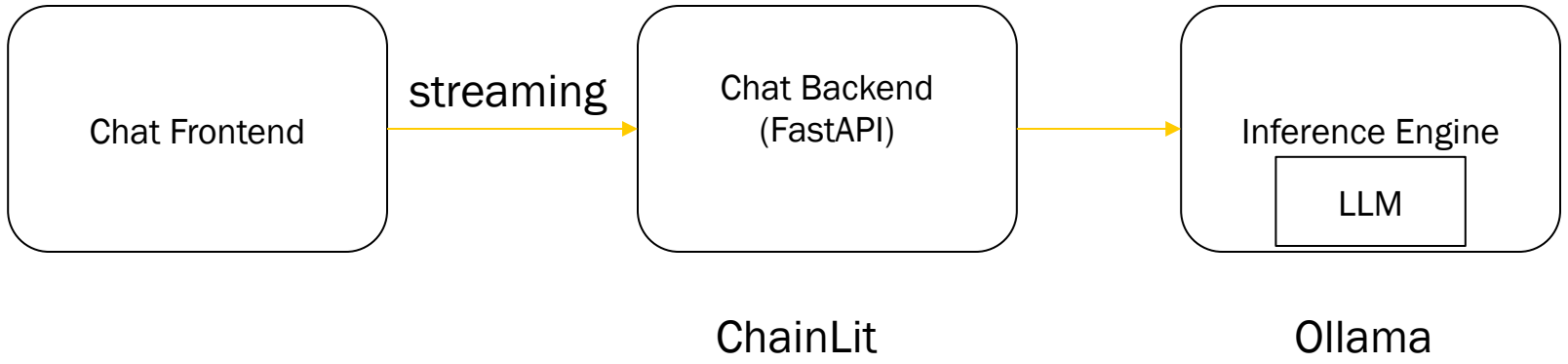
When creating GenAI systems, it can be useful to have LLMs criticize their own outputs. This could simply be a judgement (e.g., is this output correct) or the LLM could be prompted to provide feedback, which is then used to improve the answer.

Decomposing complex problems into simpler sub-questions. This is an effective problem-solving strategy for humans as well as GenAI

02.12
Conversational AI

INF
Informatik
Hochschule Reutlingen

Conversational AI



02.12
Literature

Literatur

Informatik
Hochschule Reutlingen

Further Reading

- [Language] https://quest.mit.edu/research/missions/language
- [Language] Language as a modulator to cognitive and neurological systems https://www.sciencedirect.com/science/article/pii/S0001691825001167#s0040
- [Language Modeling] Lena Voita; https://lena-voita.github.io/nlp_course/language_modeling.html
- [Geometric Deep Learning] https://thegradient.pub/towards-geometric-deep-learning/
- Raymond Lee, “Natural Language Processing”, 2024, Springer
- Manning & Schütze, Foundations of Statistical Natural Language Processing, 1999, MIT Press
- Lena Voita, “Language Modeling” https://lena-voita.github.io/nlp_course/language_modeling.html
- Zero to Mastery Learn PyTorch for Deep Learning: https://www.learnpytorch.io
- Dive into Deep Learning: https://d2l.ai/index.html

Literatur

Informatik
Hochschule Reutlingen

Further Reading

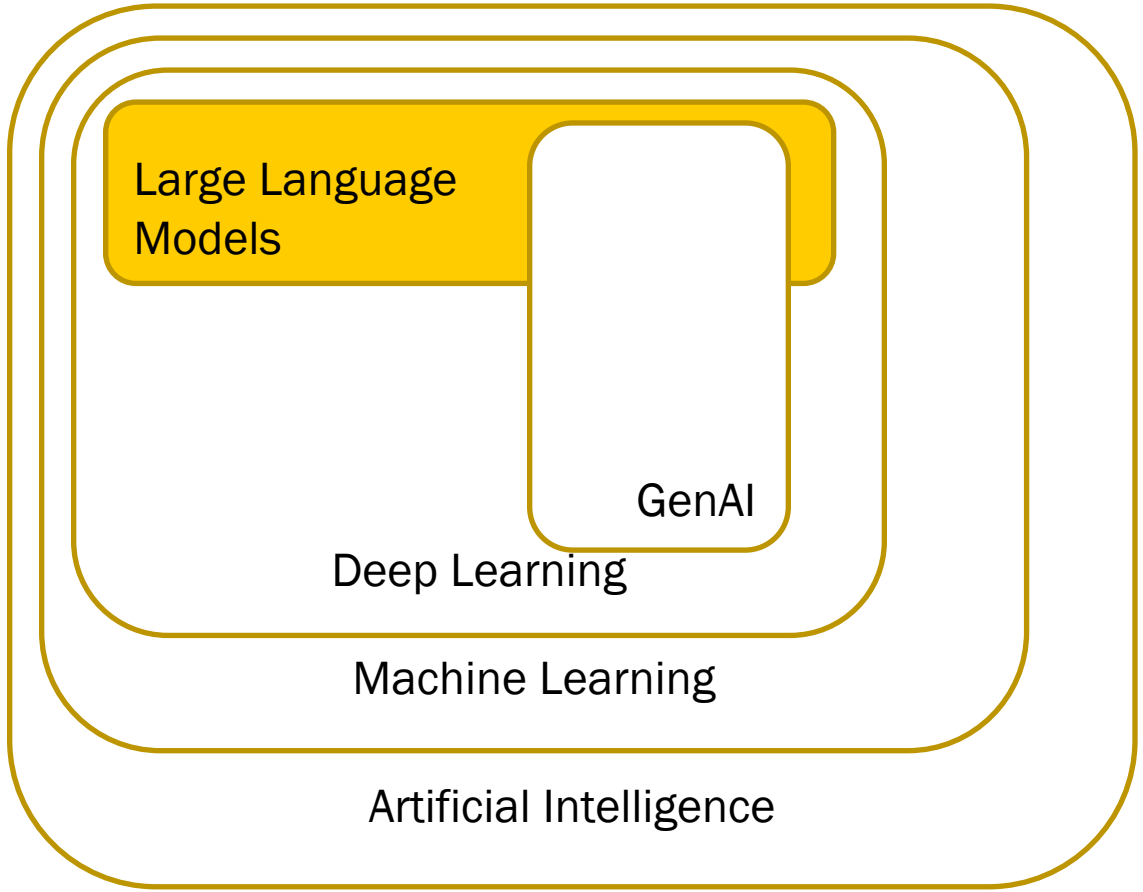
- [DLG] „Generative Deep Learning“, David Foster, 2023, O‘Reilly
- „Learn Generative AI with PyTorch“, Mark Liu, 2024, Manning
- [GAN] „NIPS 2016 Tutorial: Generative Adversarial Networks“, Ian Goodfellow 2027, https://arxiv.org/pdf/1701.00160
- [STA] „Stanford CS236, Deep Generative Models“, 2023, https://deepgenerativemodels.github.io
- [COR] „Cornell CS 6785: Deep Generative Models“, 2024, https://kuleshov-group.github.io/dgm-website/
- [REV] Roberto Gozalo-Brizuela, Eduardo C. Garrido-Merch´an „ChatGPT is not all you need. A State of the Art Review of large Generative AI models“, https://arxiv.org/pdf/2301.04655
- GPT-3: 2020 Language Models are Few-Shot Learners, https://arxiv.org/abs/2005.14165
- Llama: 2023 Open and Efficient Foundation Language Models https://arxiv.org/abs/2302.13971
- Llama 2: 2023 Open Foundation and Fine-Tuned Chat Models https://arxiv.org/abs/2307.09288
- Llama-Omni: 2024 (Sept) Seamless Speech Interaction with Large Language Models https://arxiv.org/abs/2409.06666

INF Informatik
Hochschule Reutlingen

Dominik Neumann
Hochschule Reutlingen, Alteburgstraße 150, 72762 Reutlingen
www.reutlingen-university.de
T. +49 172 9861157
dominik.neumann@reutlingen-university.de
dominik.neumann@exxeta.com

B.01
Generative Artificial Intelligence

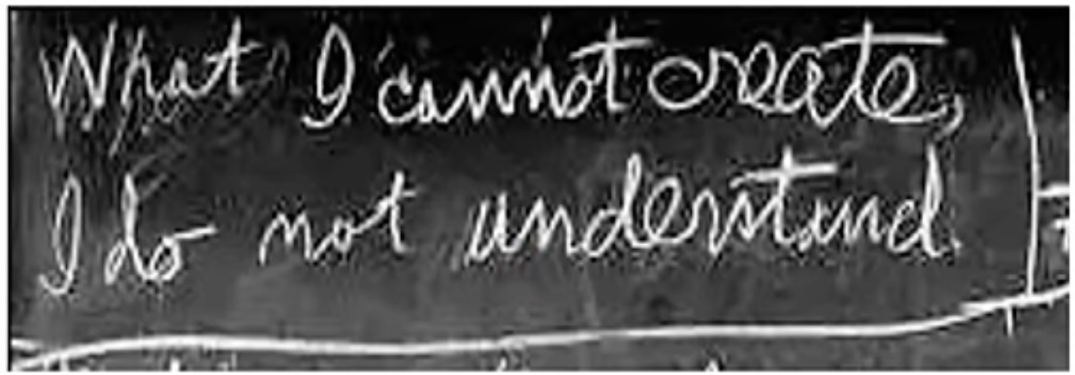
There is an intersection between GenAI & Large Language Models



Large Language Models represent a specific application of deep learning techniques, leveraging their ability to understand, process and generate human like text.

A Large Language Model is a very large deep learning model that is pre-trained on massive amounts of data.

But what is GenAI?

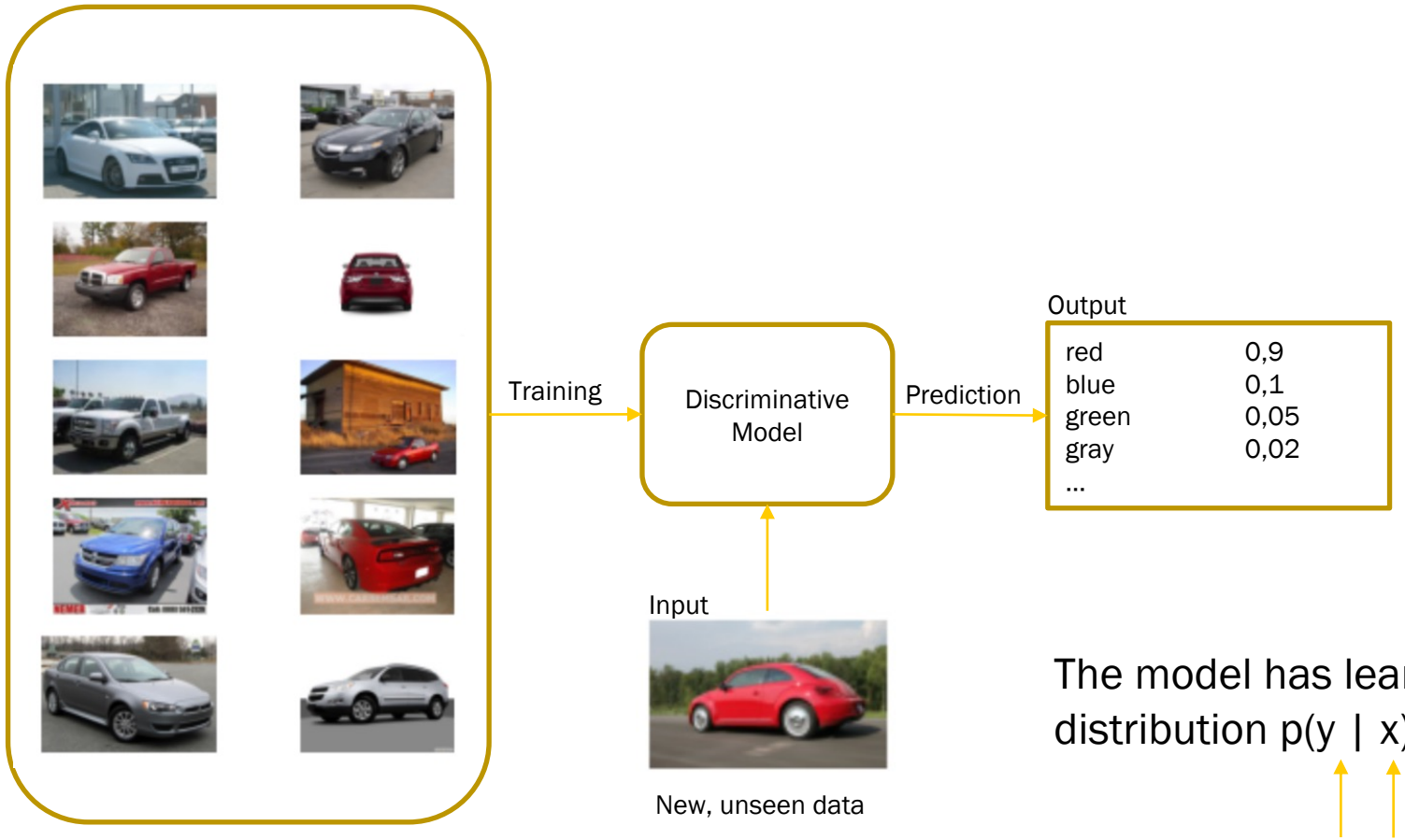


“What I cannot create, I do not understand..”
- Richard Feynman

“a system that can generate realistic data must have gained at least some understanding of the real world.”

Discriminative versus Generative AI

Car color classification as task for a discriminative model



1. The model is trained to classify the color of cars
2. For a picture of a car the model outputs a vector of probabilities for each color class

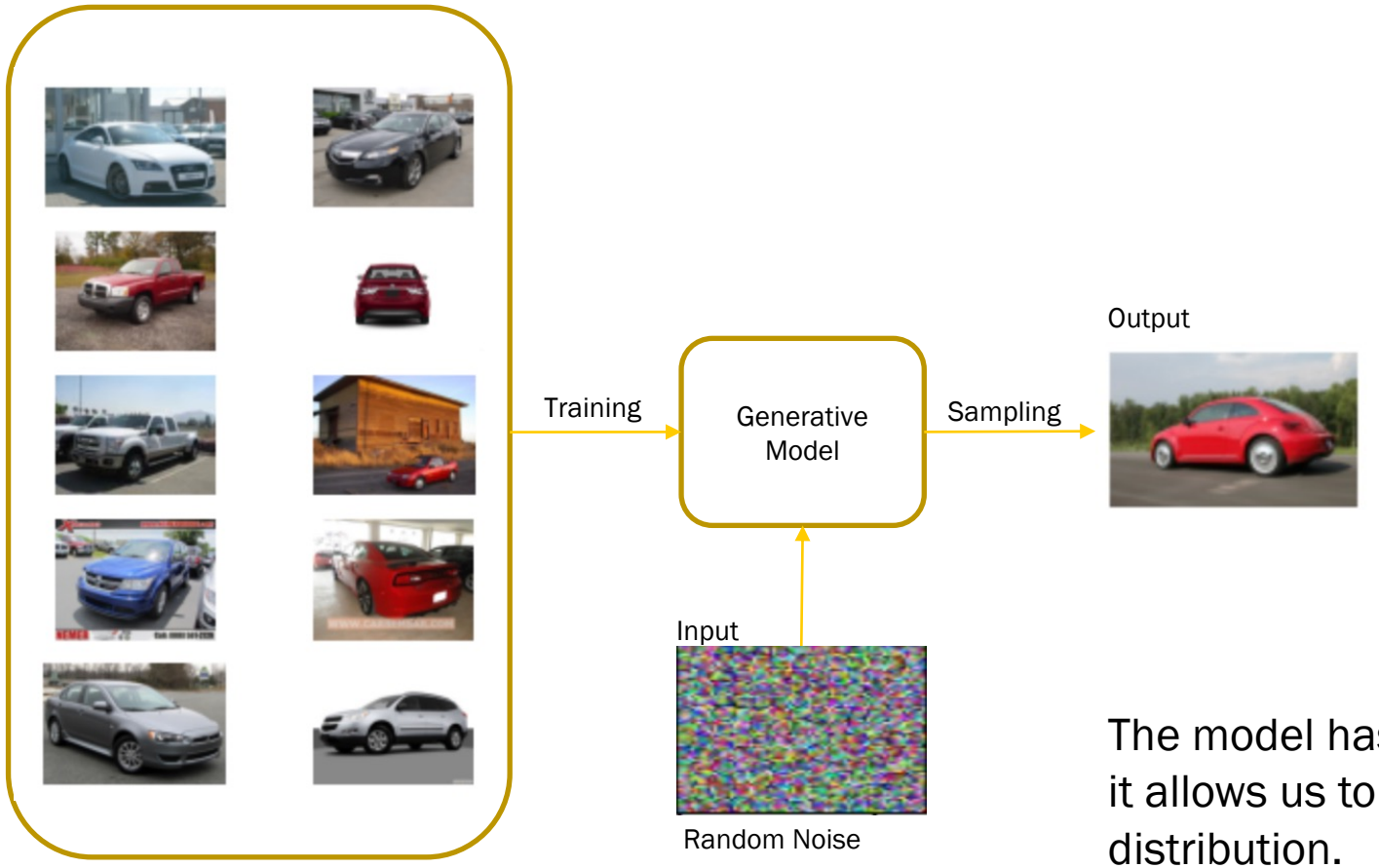
The model has learnt a conditional probability distribution p(y | x).

label sample

In analogy to [DLG] Figure 1-2

Discriminative versus Generative AI

Car data generation as task for a generative model



1. The model is trained to understand pictures of cars
2. The model samples from random noise new pictures of cars

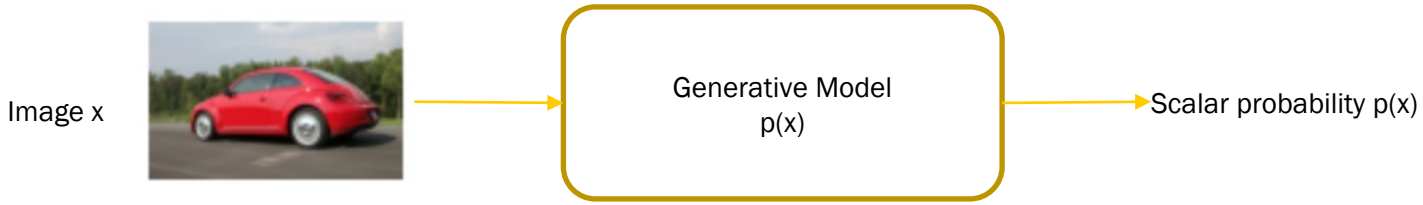
The model has learnt a probability distribution p(x) and it allows us to sample new data from the learned distribution.

In analogy to [DLG] Figure 1-1

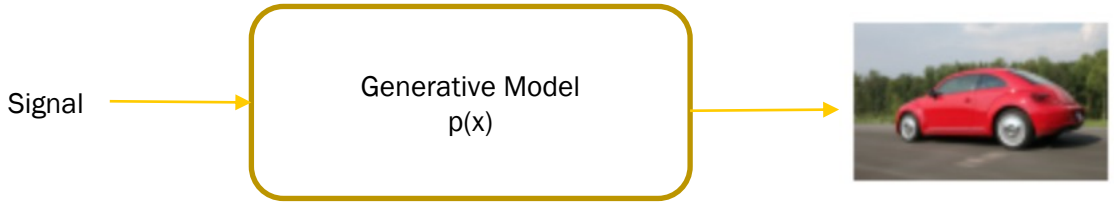
Generative AI

A statistical generative model is a probability distribution p(x):

- Data (images or sequences of text)
- Prior Knowledge (--> parametric Form: e.g. Gaussian; loss function: e.g. maximum likelihood; optimization algorithm, ...)



How likely is that image x according to my statistical generative model?



It is generative, because sampling from p(x) generates new images.

Why is Generation so hard?

“Curse of Dimensionality” in Pixel-Space

Each image has n pixels (black or white)
→ There are 2 x 2 x … x 2 = 2^n possible black & white images.

- for 32x32=784 pixels → 2ˆ784 possible black & white images.

But only some of these pictures representing numbers! Most of them are white noise!

- for rgb 1000x1000 colored pixels → (256 x 256 x236)ˆ1,000,000 possible rgb images.

We have to find a function (probability distribution) that could be used to generate the images we want.

Why is Generation so hard?

Hochschule Reutlingen

“Curse of Dimensionality” in Vocabulary-Space:

- The English language has around 170,000 words.
- The Oxford English dictionary defines approximately 600,000 word-forms.
- There around 1,000,000 words with accounts for multiple forms of the same word and alongside words that are not used in modern English.

- Assume that on average, sentences today range from 15 to 20 words.

→ 1,000,000^15 = 10^6^15 = 10^90 possible word sequences with 15 words!

Compare to the Universe:

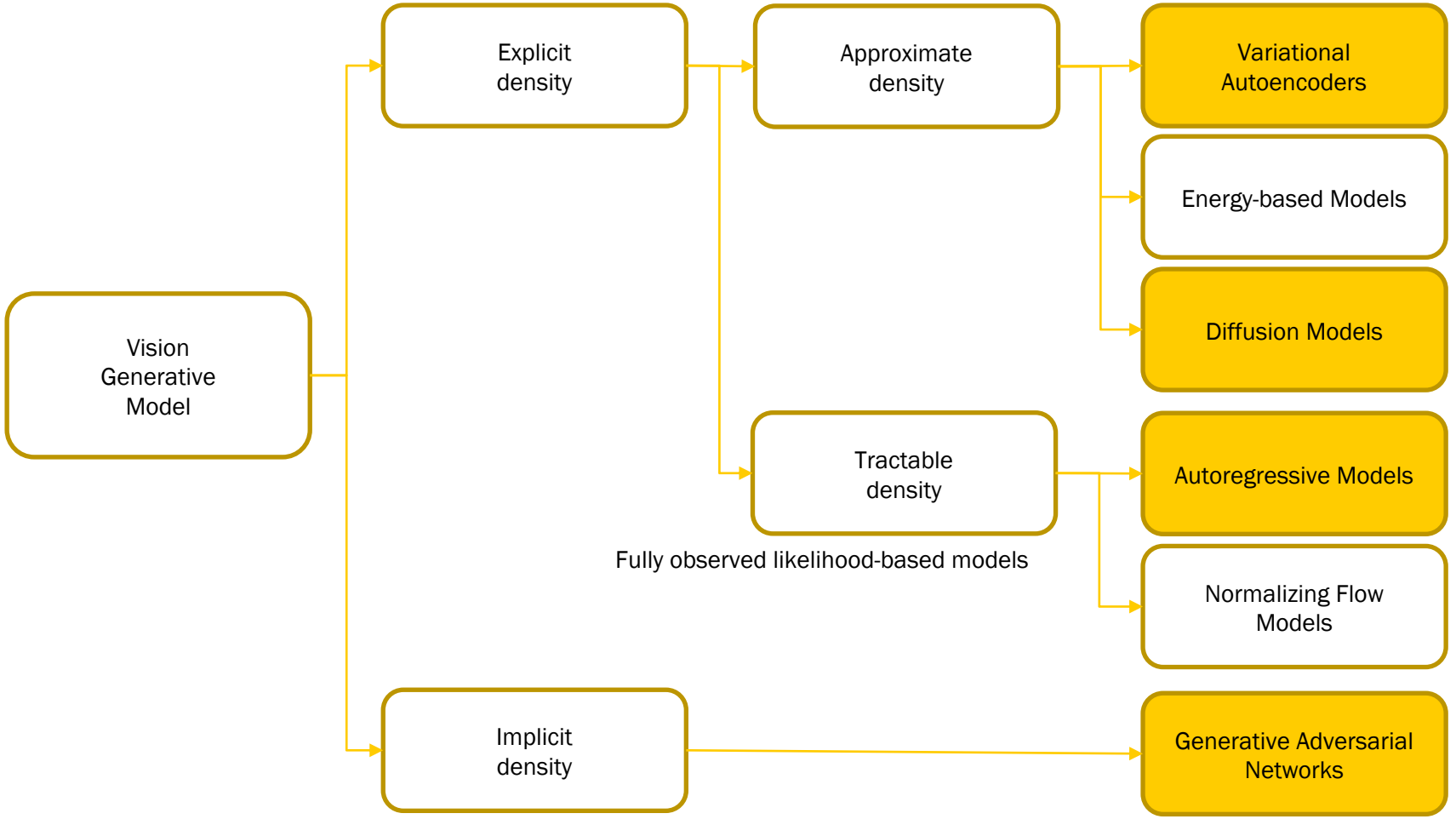
→ 10^78 to 10^82 atoms in the universe

Source:

- Wikipedia – https://en.wikipedia.org/wiki/English_language
- Medium - https://medium.com/@theacropolitan/sentence-length-has-declined-75-in-the-past-500-years-2e40f80f589f
- https://wordsrated.com/how-many-words-are-in-the-english-language/
- Oxford - https://educationblog.oup.com/secondary/maths/numbers-of-atoms-in-the-universe

There are six families of generative models

The main difficulty present in explicit density models is designing a model that can capture all of the complexity of the data to be generated while still maintaining computational tractability.



Compact Represenation learning, Image denoising
Generation of images, text, musical compositions, ...

Object recognition, image restauration, sequence labeling, anomaly detection

State of the art in Text-2-Image Generation: StableDifusion, Imagen, DALL-E 3

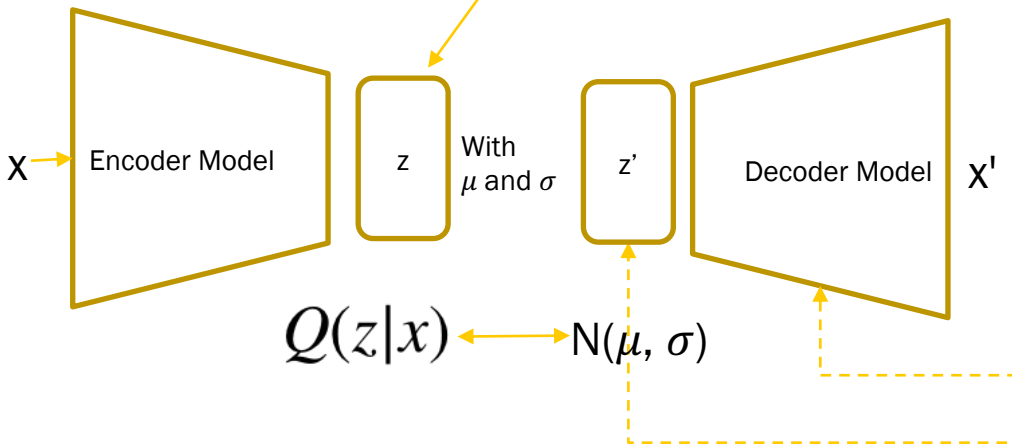
Were state of the art before diffusion models

In analogy to [DLG] Figure 1-10, originated by the taxonomy of deep generative models by Ian Goodfellow [GAN]

Hochschule Reutlingen

Variational Autoencoder

Training Time



$$P(x) = \int P(x|z)P(z)dz$$

Learn this term as a function represented by a NN

Sample for Gaussian Distribution with μ and σ

Loss:
1. Reconstruction error: the output should be similar to the input.
2. Q(z|x) should be similar to the prior N(μ, σ)

1. An input image is passed through an encoder network.
2. The encoder outputs parameters of a distribution Q(z|x).
3. A latent vector z' is sampled from Q(z|x). If the encoder learned to do its job well, most chances are z will contain the information describing x.
4. The decoder decodes z' into an image x'.

03_VAE_MNIST.ipynb

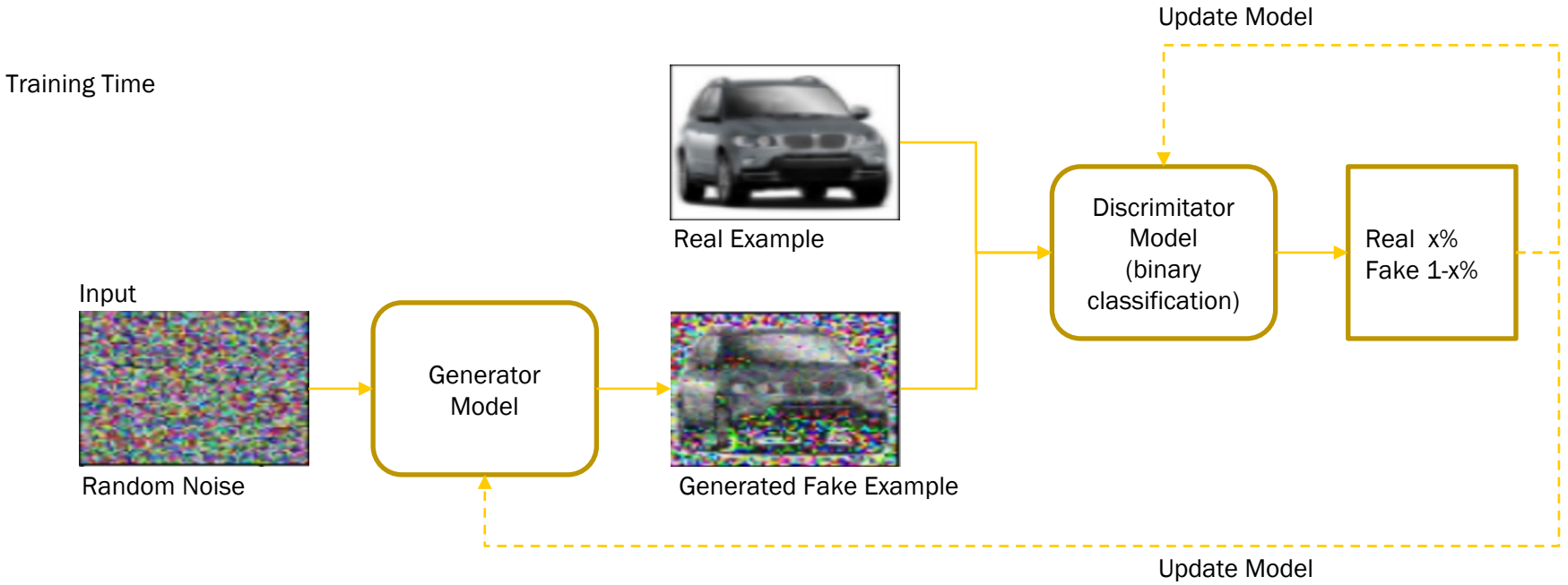
Variational Autoencoder

[1]:
```
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision.datasets import MNIST
import torchvision.transforms as transforms
```

Load MNIST Data

[2]:
```
transform = transforms.Compose([transforms.ToTensor()])

path = "./data"
train_dataset = MNIST(path, transform=transform, download=True)
```

Generative Adversarial Network



2014, Ian Goodfellow

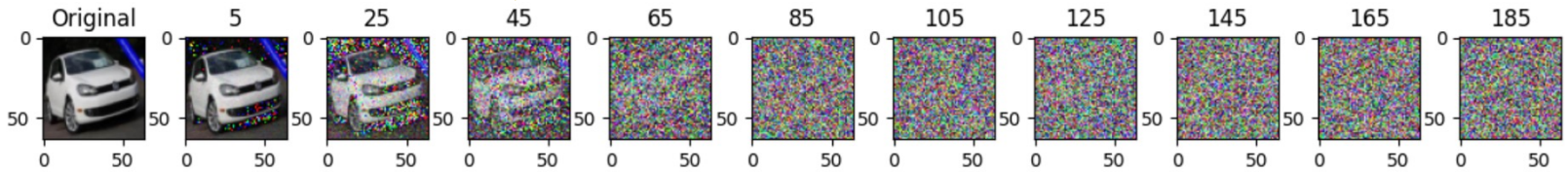
Hochschule Reutlingen

Diffusion Models

Diffusion Models work:

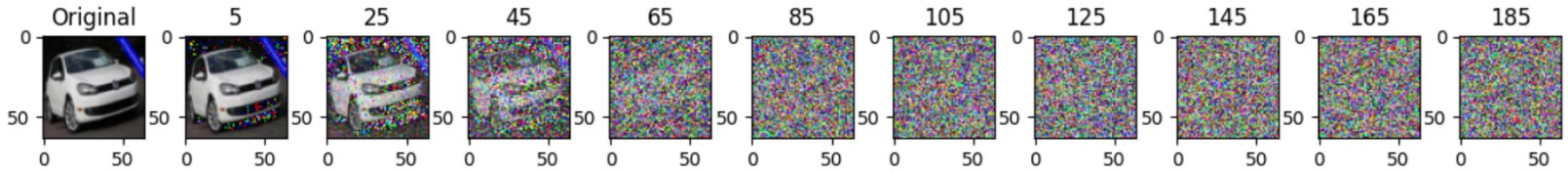
- by destroying an input by gradually adding noise
- they learn a model to reconstruct the previous image
- and then denoising it by the learned model.

Forward process



Markov chain: where each timestep depends only on the previous timestep

Model



use the Model

use the Model

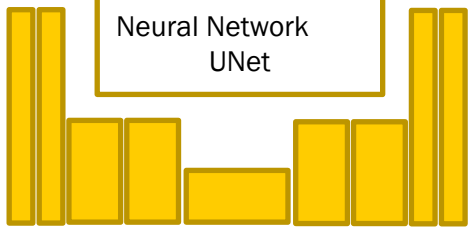
use the Model

Parametrized Backward process

Noise Scheduler

Timestep Encoding

Neural Network UNet



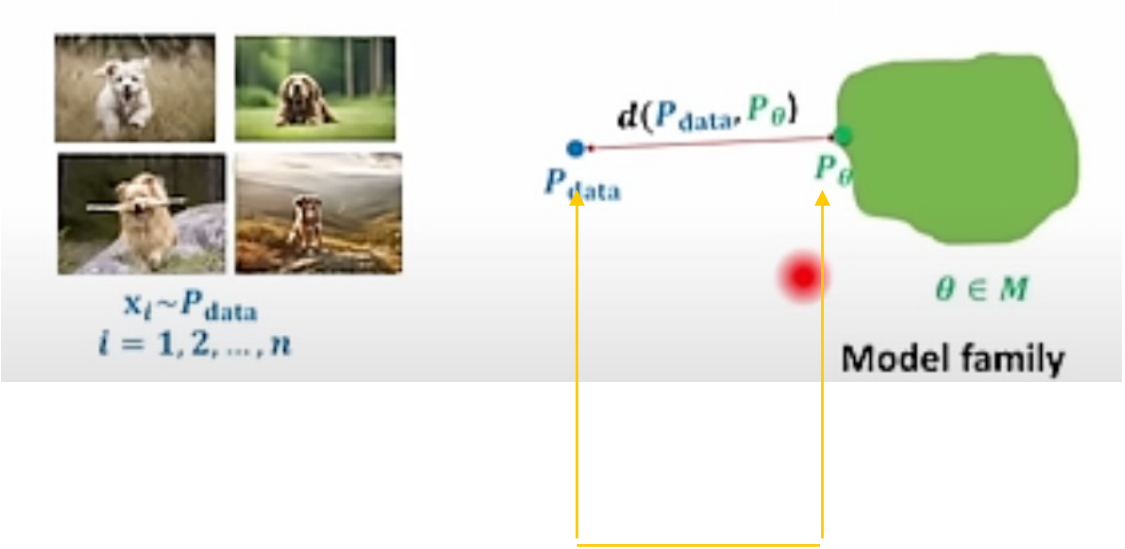
Source: Denoising Diffusion Probabilistic Model - https://arxiv.org/abs/2006.11239

Informatik
Hochschule Reutlingen

Summary

Main Challenges in generative AI are:

- Representing Probability Distributions: How to do we model the distribution?



Loss Function: Notion of Similarity

→ Reformulate the Problem as an Optimization Problem

Our Goal is to learn a probability distribution p (as our model) so that:

1. Generation is possible by sampling:
   →e.g. x_new ~ p(x), x_new should look like a dog

2. Density Estimation could be used for anomaly prediction:
   → p(x) should be high if x looks like a dog and low otherwise (anomaly prediction)

3. Unsupervised representation Learning is possible for similarity use cases:
   → we should be able to learn what the images have in common (their hidden or latent features).

B.02 Representation Learning

INF Informatik
Hochschule Reutlingen

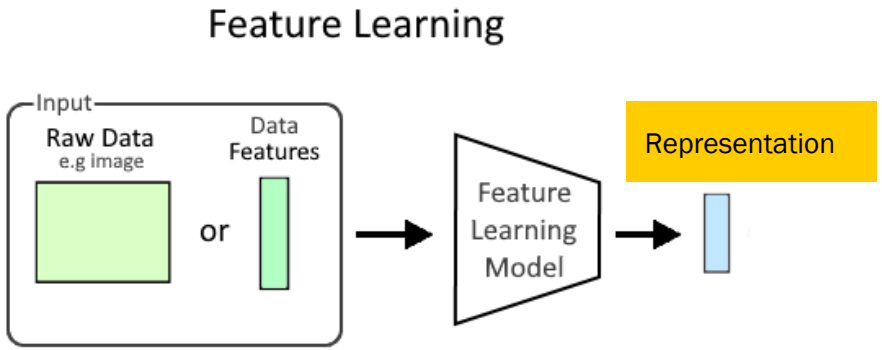
Concept of a Model in Machine Learning

Informatik
Hochschule Reutlingen

Representation Learning

- Representation Learning is a form of information compression.
- We want to extract the relevant and hidden information (latent features) that characterize our objects into a vector representation.
- Having a vector representation of an object in a vector space, then we are able, to use similarity measures to compare objects or construct new objects using the representations.

- Image to Vector
- Word to Vector
- Sentence to Vector
- Graph Node to Vector



Source: Wikipdeoia - By Fgpacini - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=123926277

02_AE_MNIST.ipynb

Autoencoder

The architecture consists of an Encoder and a Decoder module. The Encoder compress the data into an abstract latent space) and the Decoder decompress the encoded information and reconstruct the data.

The goal of an Autoencoder is to learn a compressed representation of the input data, focusing on minimizing the original input and the reconstruction).

see: Mark Liu, "Learn Generative AI with PyTorch", Chapter 7.2

[1]:
```
import torch
import torch.nn as nn

import torchvision
import torchvision.transforms as T

from torchvision.datasets import MNIST
```

Load MNIST data

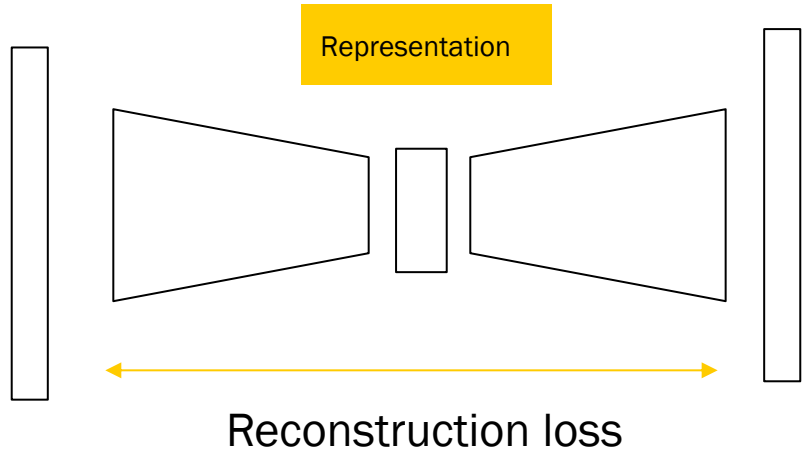
[2]:
```
transform = T.Compose([T.ToTensor()])

train_set = MNIST("./data", train=True, download=True, transform=transform)
test_set = MNIST("./data", train=False, download=True, transform=transform)
```

[3]:
```
for example in iter(train_set):
    print(example[0].size(), example[1])
    break
```

```
torch.Size([1, 28, 28]) 5
```

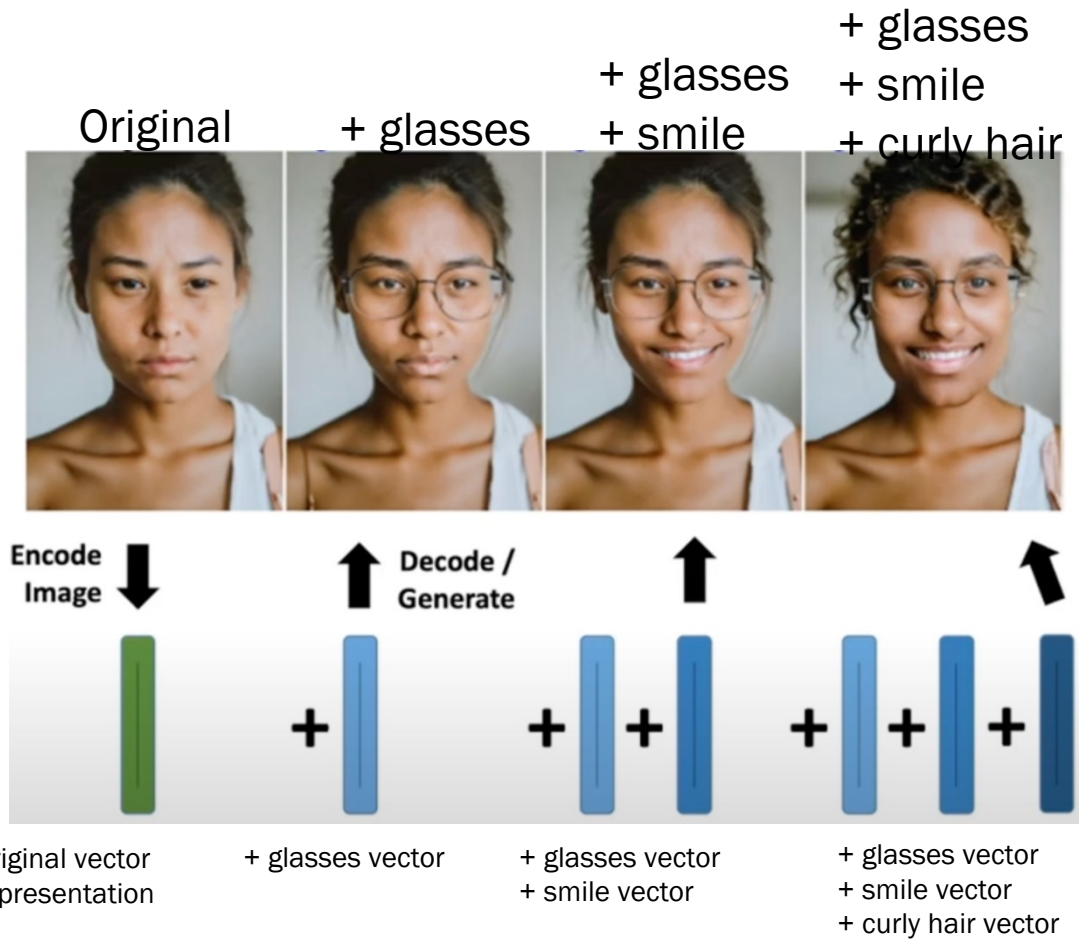
[4]:
```
#create batches with 32 images per batch
batch_size = 32
```

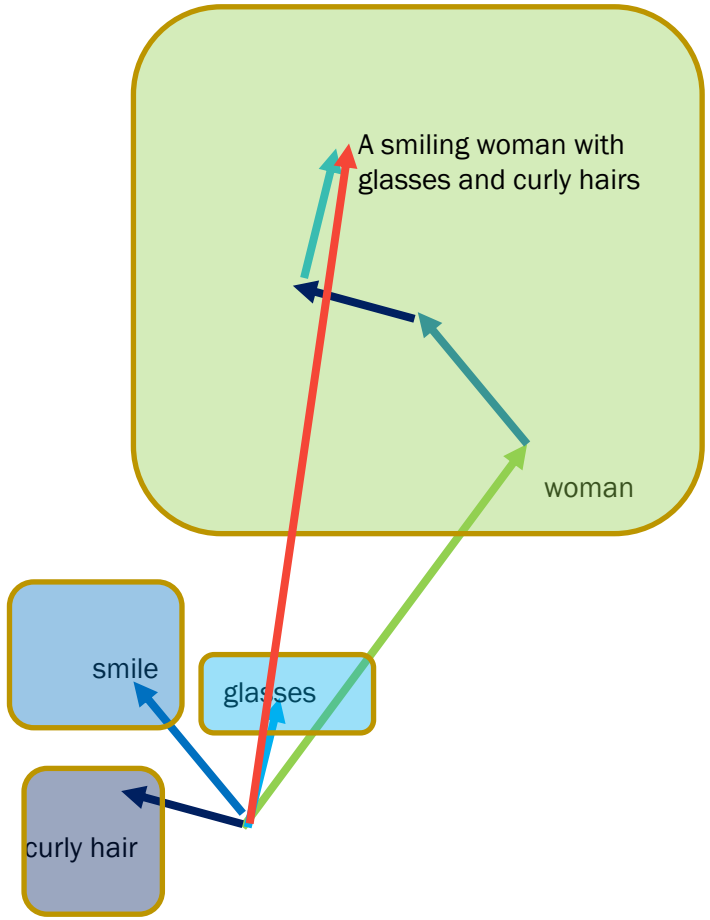
Autoencoder



Concept of a Model in Machine Learning

Representation Learning





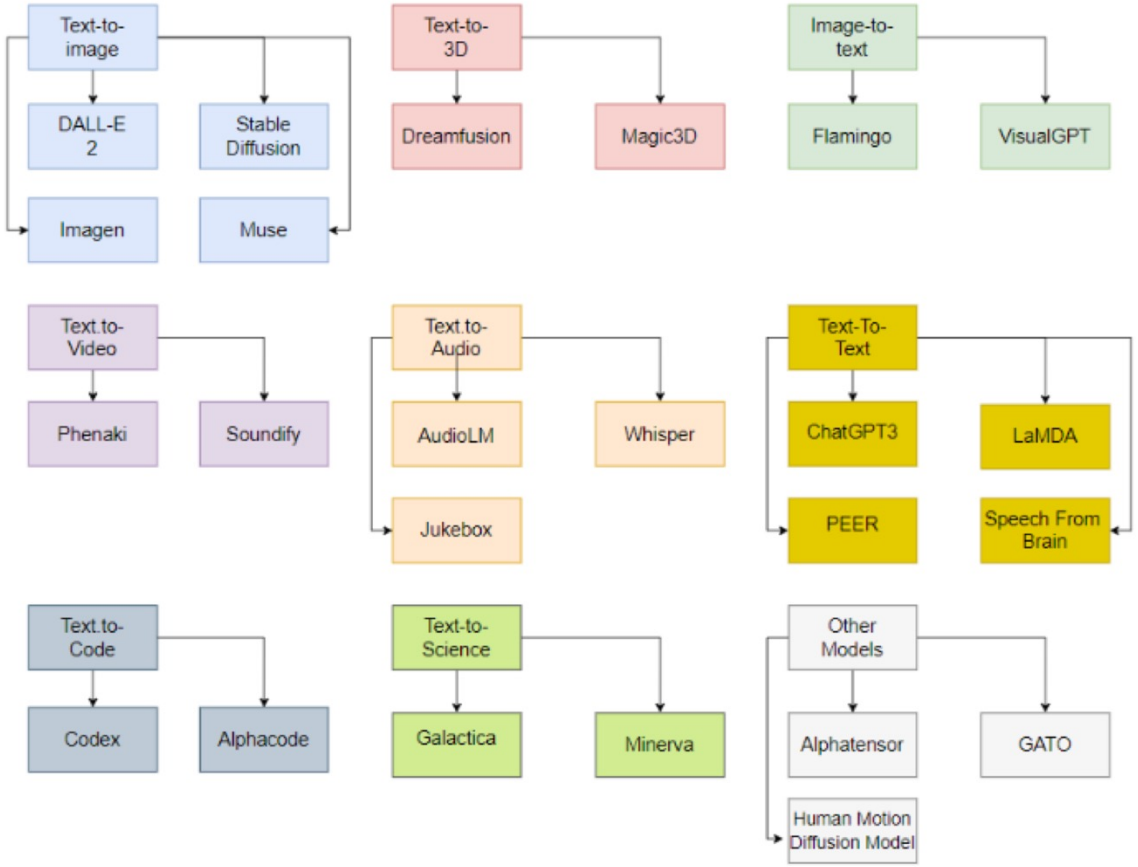
Source: Deep Gernerative Models: https://kuleshov-group.github.io/dgm-website/

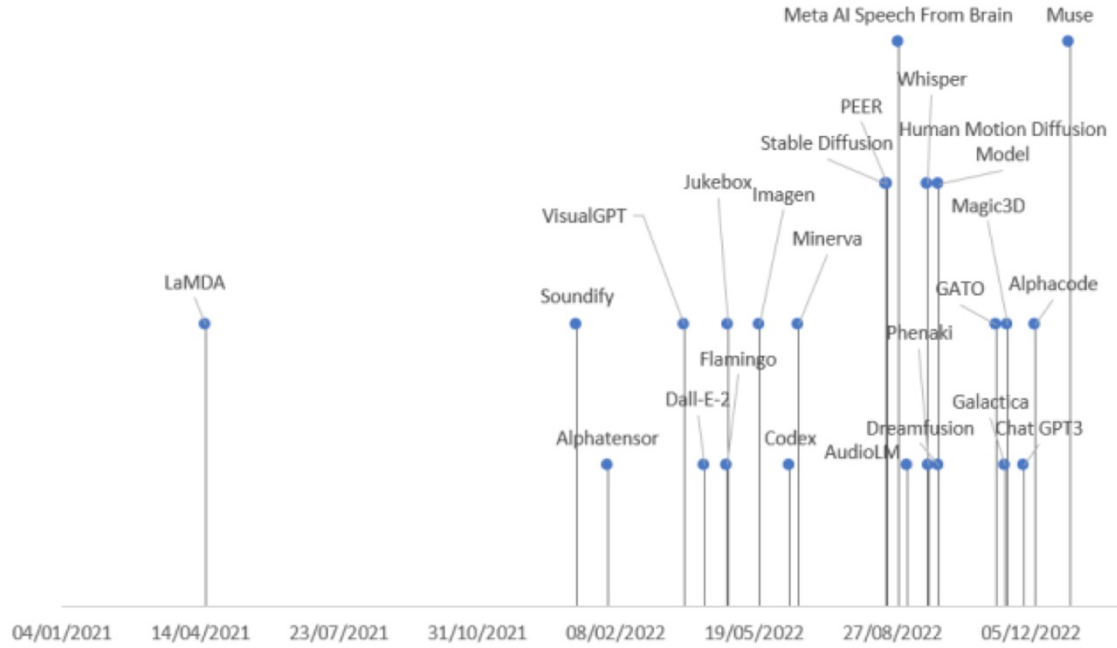
B.03
GenAI Tasks and their Models

INF Informatik
Hochschule Reutlingen

GenAI Models

A taxonomy of the most popular generative AI models according to their task (2022)

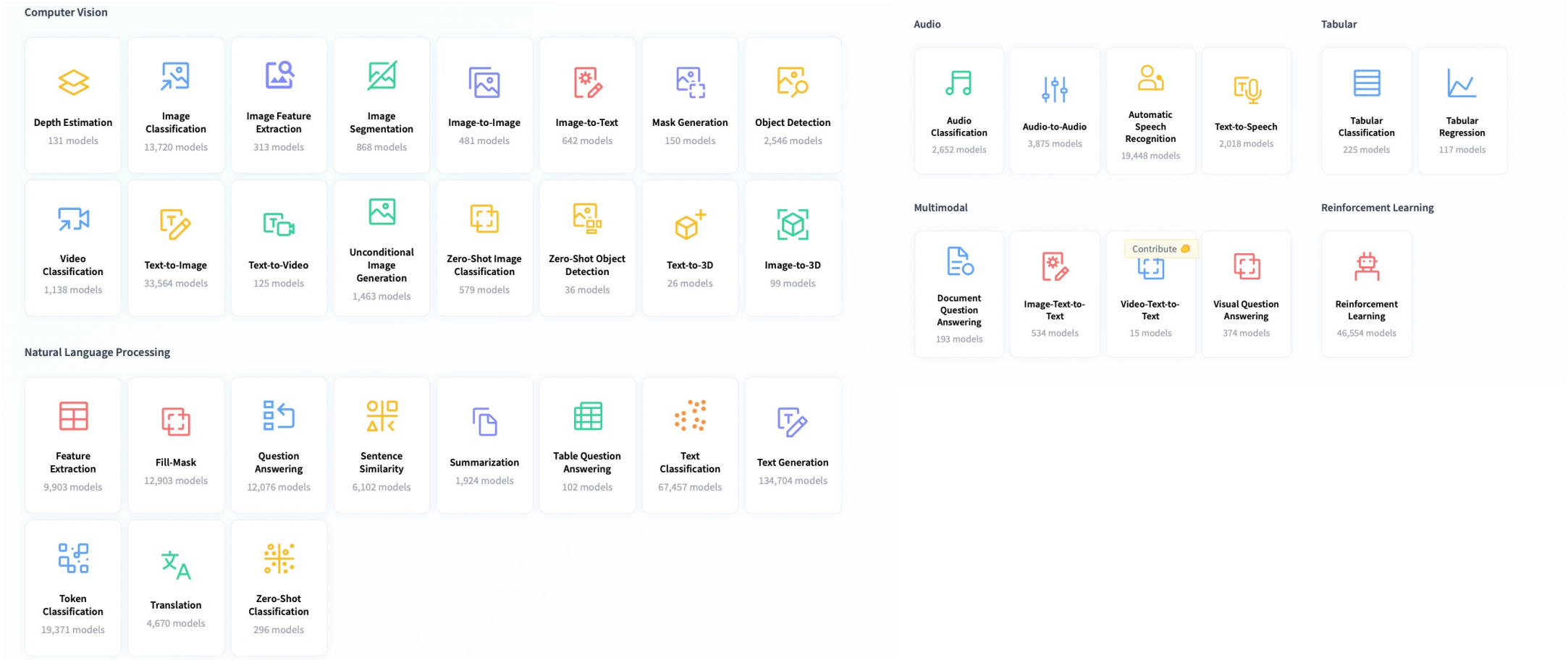




Source: [REV] https://arxiv.org/pdf/2301.04655

GenAI Tasks

Task to Model by Huggingface



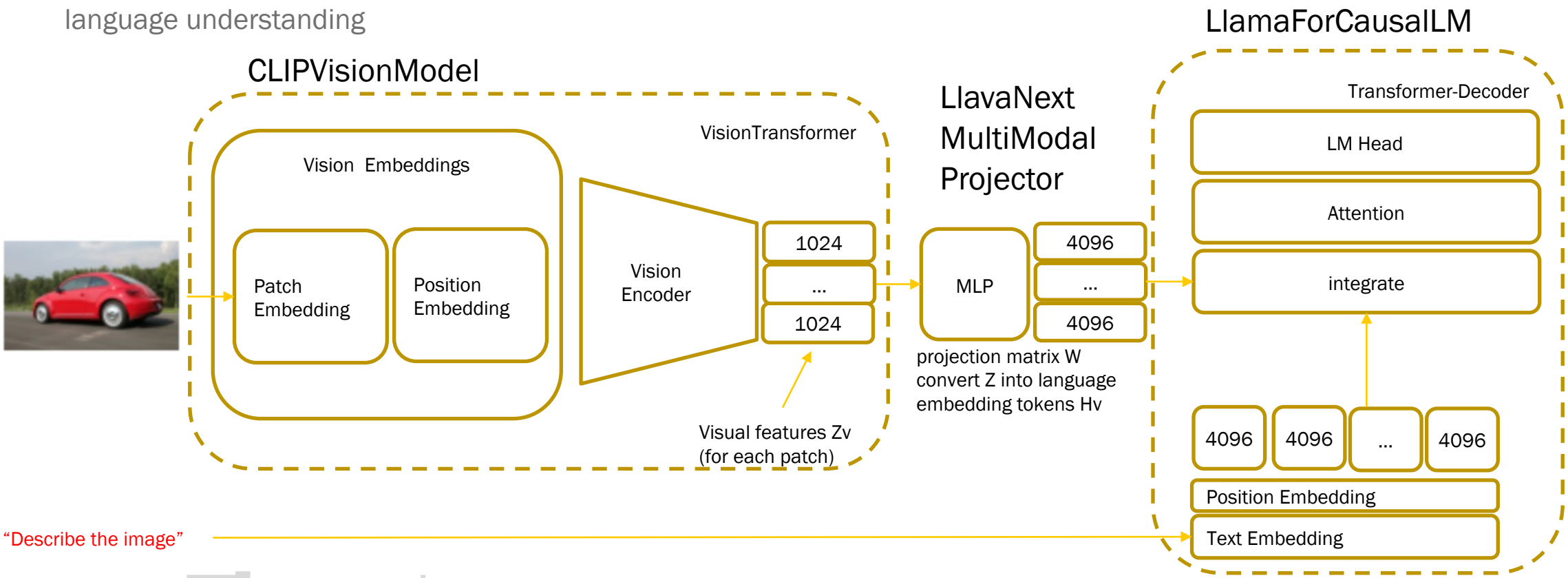
Source: https://huggingface.co/tasks

GenAI Tasks

Example: Image-Text-2-Text (multimodal)

Large Language and Vision Assistant (LLaVA)

LLaVA is an end-to-end trained large multimodal model that connects a vision encoder and an LLM for general-purpose visual and language understanding



Source: https://arxiv.org/pdf/2304.08485

B.04
Compound AI Systems

INF Informatik
Hochschule Reutlingen

Compound AI Systems

LLM Challenges

Informatik
Hochschule Reutlingen

Challenge	
Hallucination	Models always generate text as an extrapolation from the prompt you provided. Hallucination is the default. Sometimes the model generates text that is incorrect, nonsensical, or not real.
Attribution	Since LLMs are not databases or search engines, they would not cite where their response is based on. We have neither traceability nor explainability.
Staleness	Model contain incomplete, outdated, or wrong knowledge due to outdated training data.
Generalization	If we have use cases with non-publicly accessible knowledge, then the knowledge is not part of the chosen LLM. We want to retrain the LLM or enhance I with the knowledge.
Revision	How can incorrect knowledge or facts be replaced in an LLM?

Model Limitation

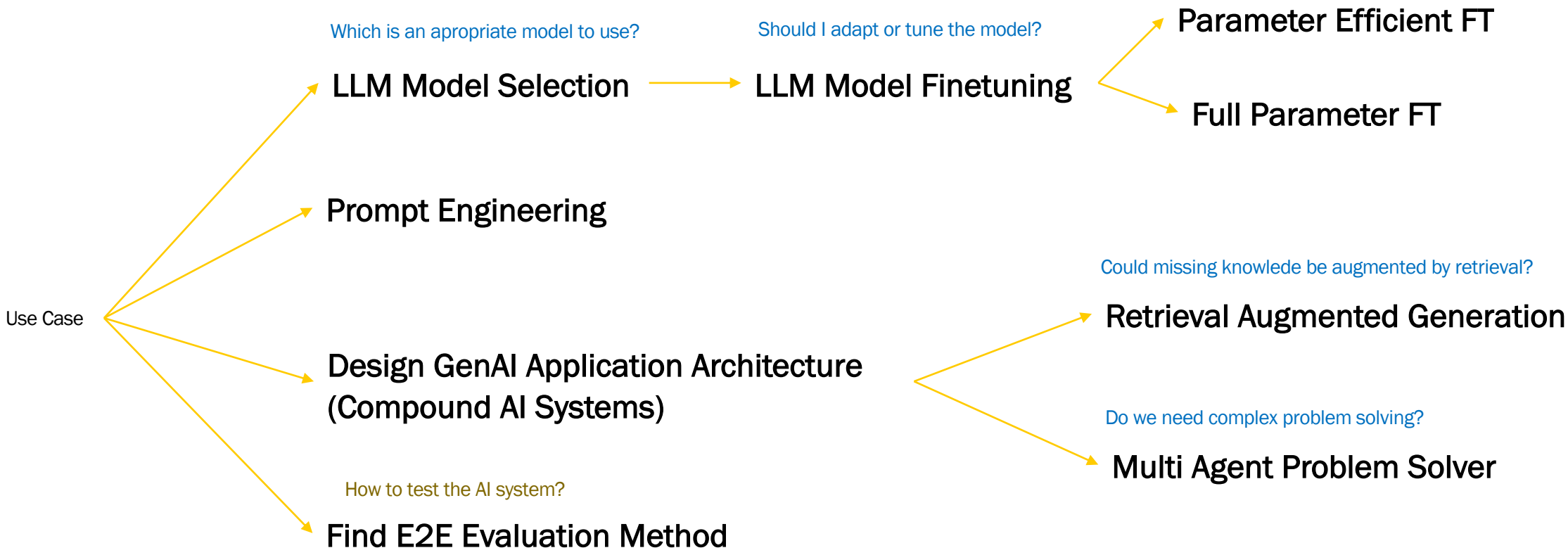
Despite the successes achieved through scaling, a language model is limited in its area of application.

1. Models are limited in what they know about the world.
2. Models are limited in kind of tasks they can solve
3. And, in addition, models are hard to adapt

Compound AI Systems

How to deal with challenges

Hochschule Reutlingen



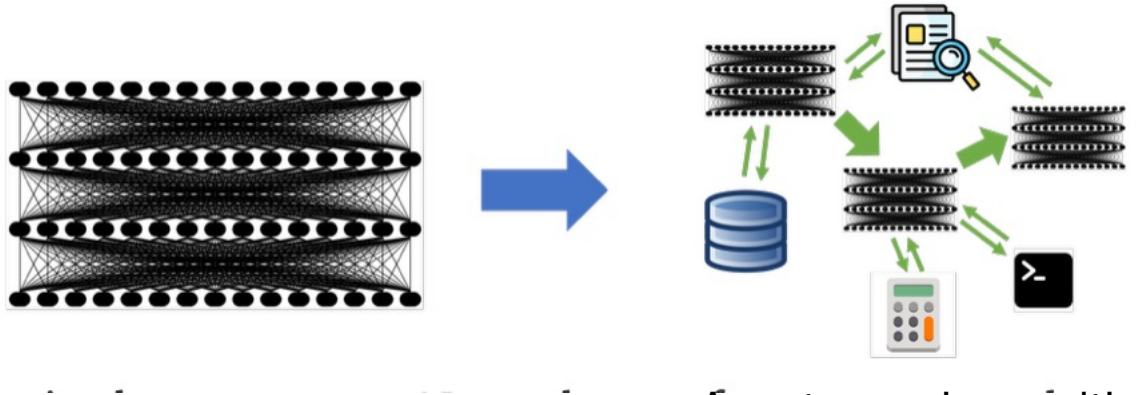
Compound AI Systems

The Shift from Models to Compound AI Systems

Hochschule Reutlingen

"Compound AI System is a system that tackles AI tasks using multiple interacting components, including multiple calls to models, retrievers, or external tools." – Berkeley Artificial Intelligence Research

In contrast, an AI Model is simply a statistical model, e.g., a Transformer that predicts the next token in text.



A single model

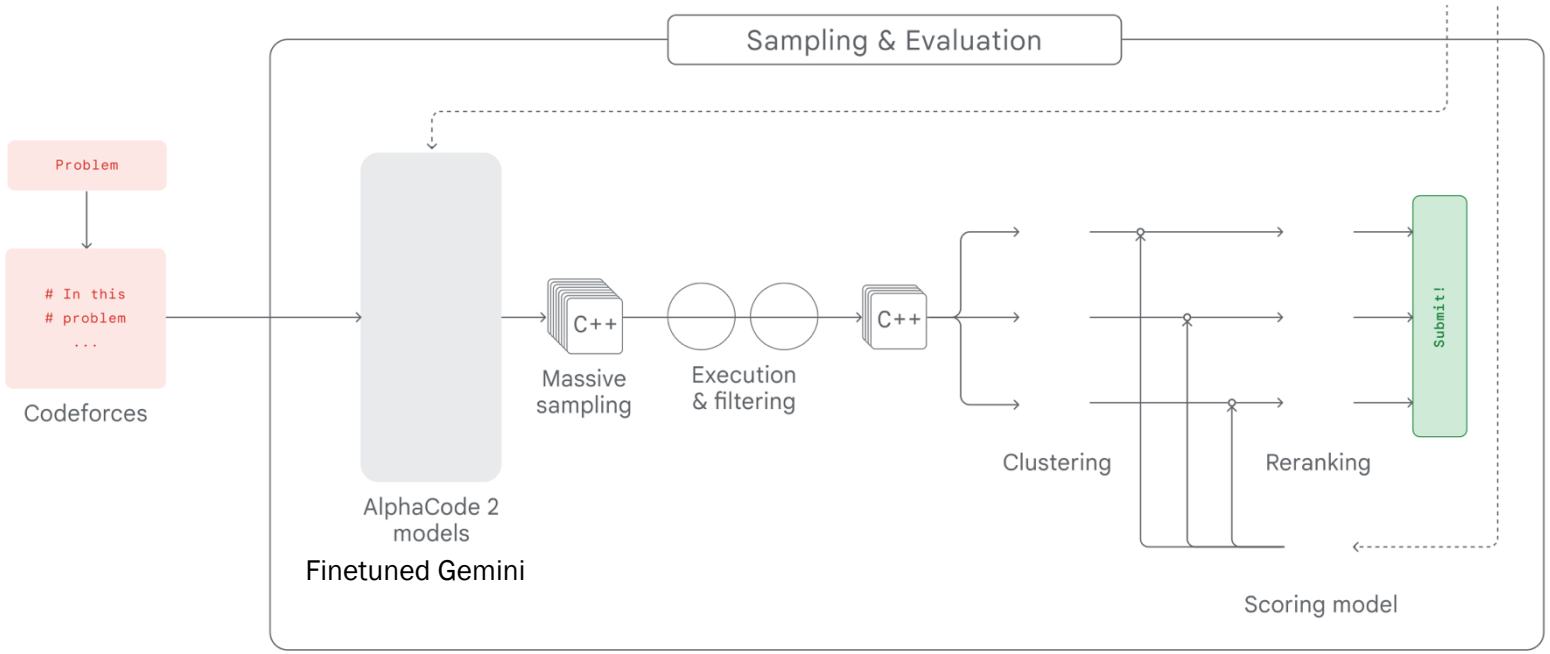
A system using multiple interacting components

See: https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/

Compound AI Systems

Compound AI systems will likely be the best way to maximize AI results in the future!

1. Some tasks are easier to solve with system design or tools in combination with models: AlphaCode 2

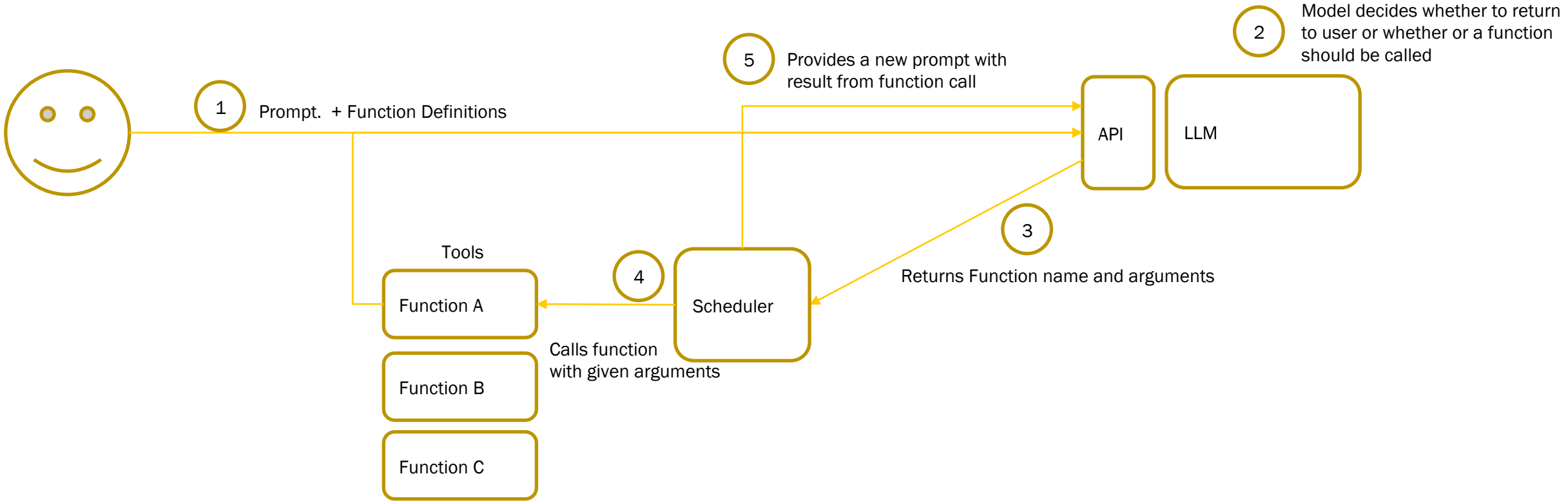


See: AlphaCode 2 - https://storage.googleapis.com/deepmind-media/AlphaCode2/AlphaCode2_Tech_Report.pdf

Compound AI Systems

Compound AI systems will likely be the best way to maximize AI results in the future!

2. Example: Function Calling



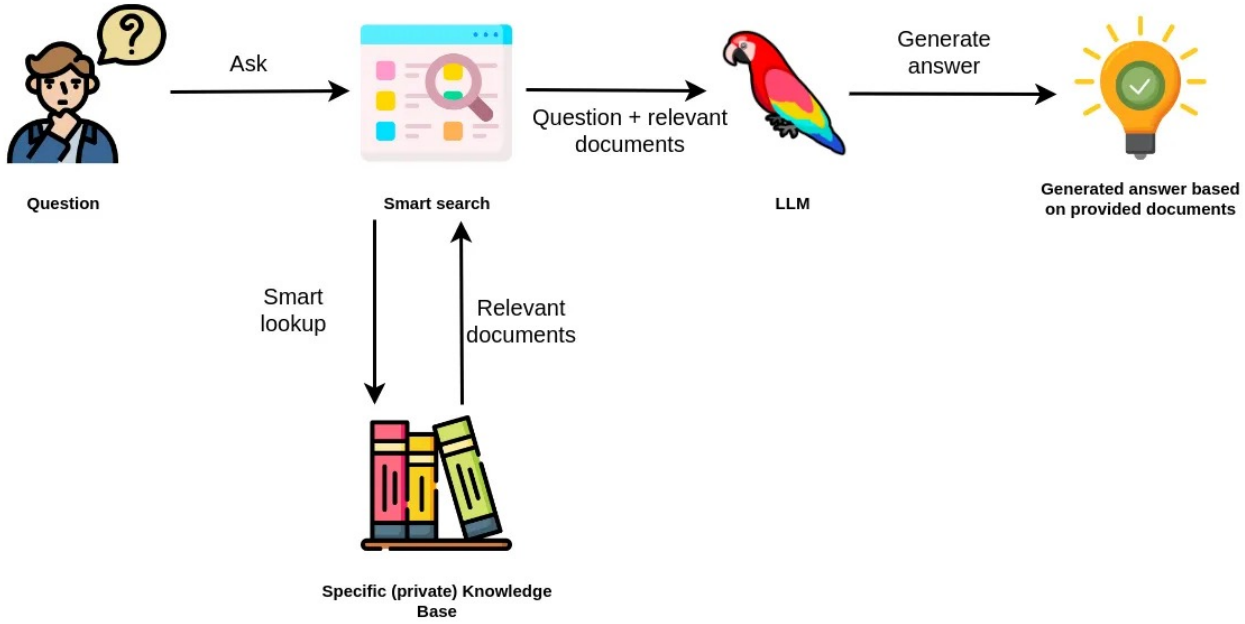
Compound AI Systems

Compound AI systems will likely be the best way to maximize AI results in the future!

3. Systems can be dynamic. Machine learning models are inherently limited because they are trained on static datasets, so their "knowledge" is fixed.

   Therefore, developers need to combine models with other components, such as search and retrieval, to incorporate timely data.

   Retrieval-Augmented Generation:



See: https://medium.com/neo4j/knowledge-graphs-llms-multi-hop-question-answering-322113f53f51

Compound AI Systems

Design Patterns

Conversional AI

Simulate a conversation with the feeling of having a conversation with a human.

- conversational memory
- dialogue generation
- Examples: ChatGPT, Anthropic Claude, Perplexity.AI

Retrieval Augmented Generation

Knowledge Retrieval and understanding is key

- Access to contextual data

CoPilot

Assists a human in his work.
Key differentiator for becoming a CoPilot is understanding of the environment in which the user works.

- access to tools and data,
- reasoning and planning capabilities,
- and specialized profiles
- Examples: GitHub CoPilot

Multi Agent Problem Solver

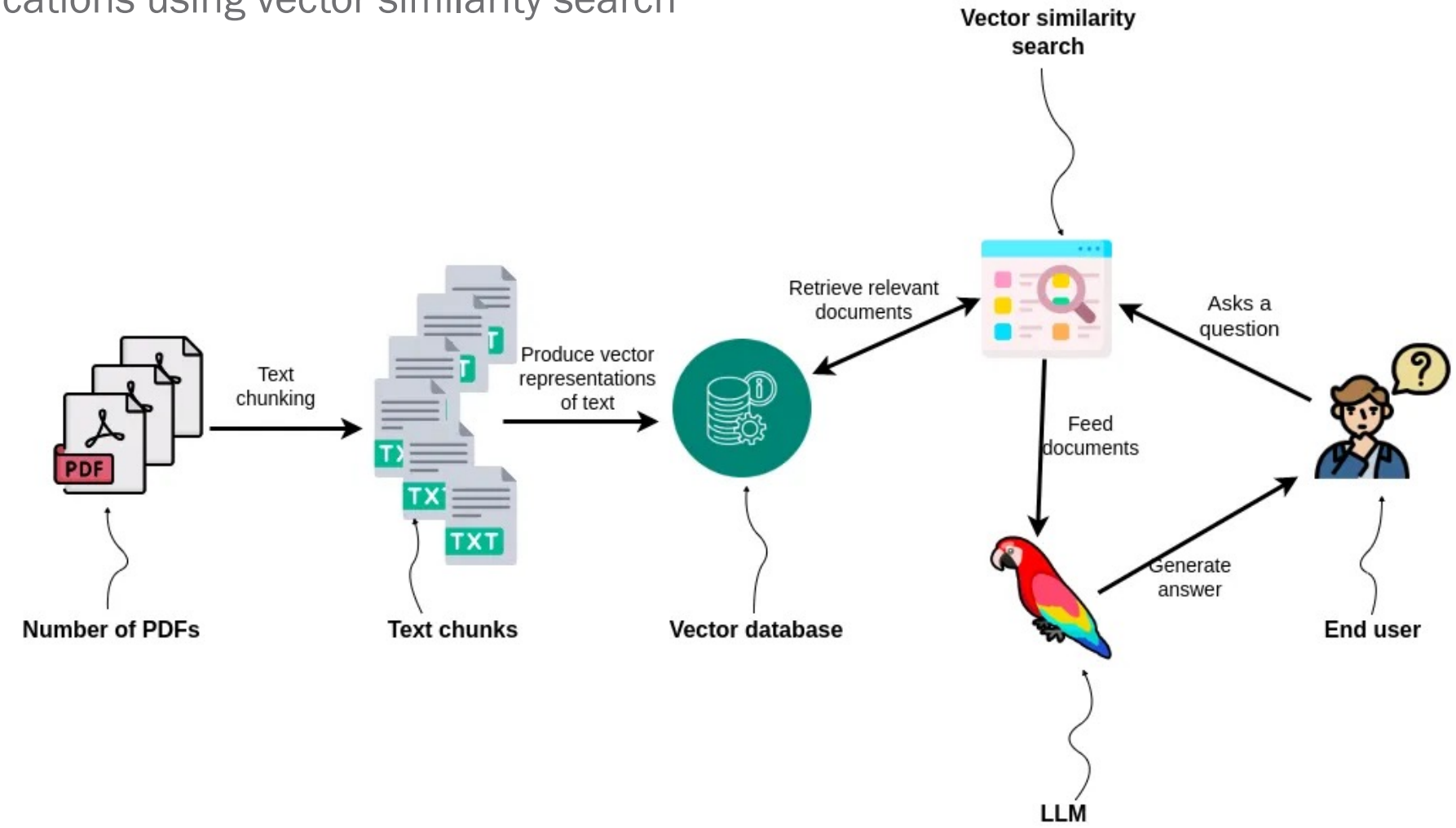
Multiple agents collaborate to solve a problem

Each agent has access to its own set of tools and can assume a very specific role while reasoning and planning its actions.

- Example: Devin (https://preview.devin.ai)

Compound AI Systems

RAG applications using vector similarity search



https://medium.com/neo4j/knowledge-graphs-llms-multi-hop-question-answering-322113f53f51

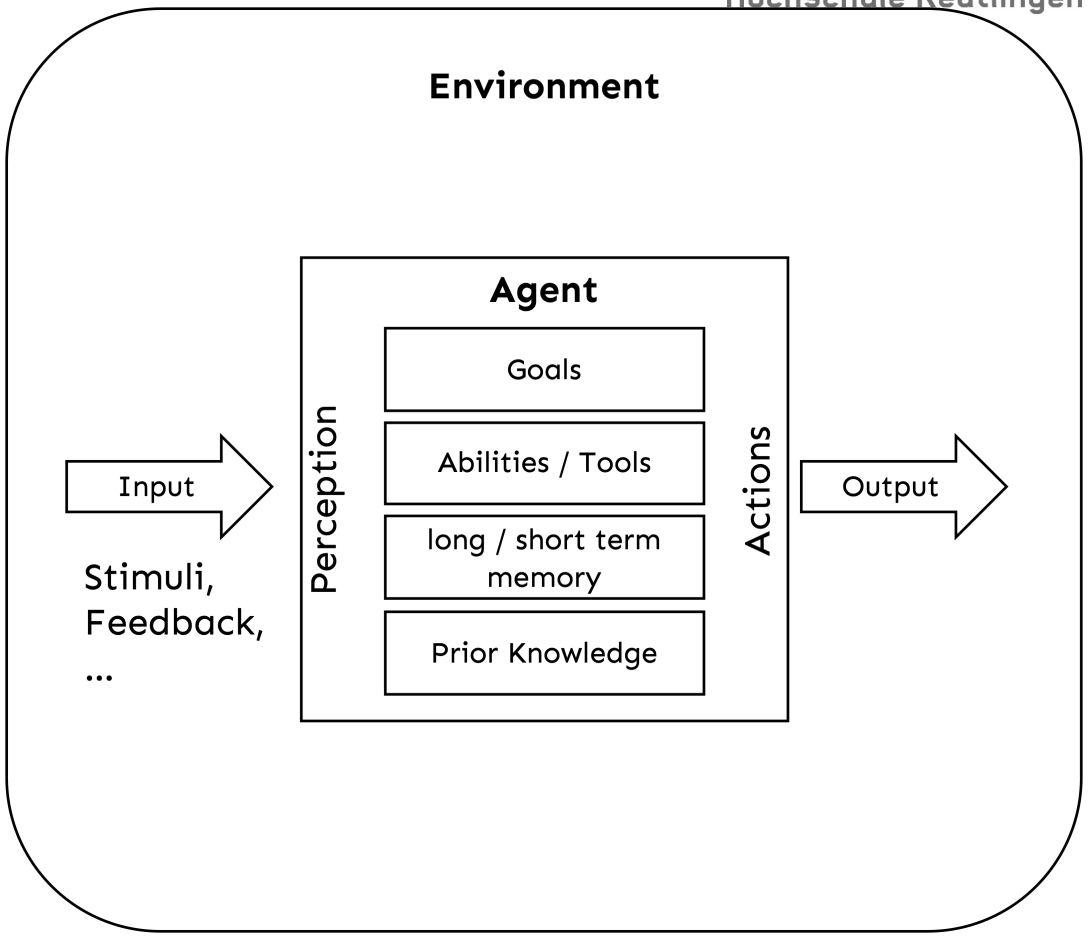
Compound AI Systems

Multi Agent Problem Solver

AI is about practical reasoning:

- **reasoning** in order to do something.
- A coupling of **perception**, **reasoning**, and **acting** comprises an agent.
- An agent acts in an **environment**.
- An agent's environment often includes other agents. An agent together with its environment is called a **world**.

Agent = Large Language Model (LLM) + Observation + Reasoning + Action + Memory



Source: https://artint.info/3e/html/ArtInt3e.Ch1.S3.html

Compound AI Systems

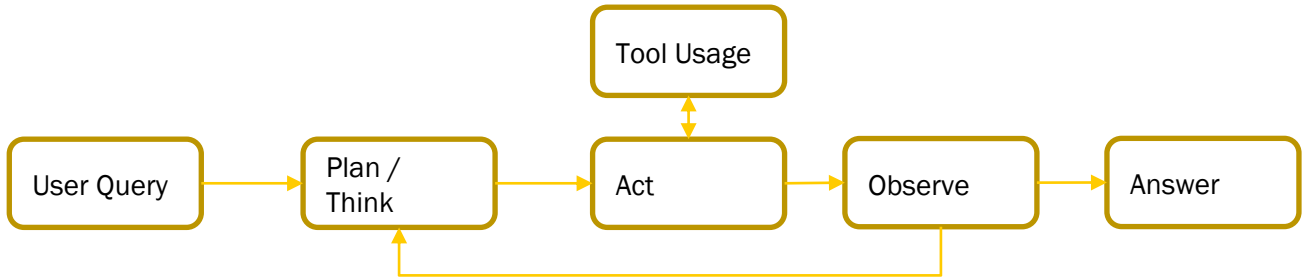
Components of LLM Agents

1. Reasoning: Put the model at the core of how problems being solved.
   The model will be prompted to come up with a plan and to reason about each step of the process.

2. Acting: Done by external programs, also known as tools.
   The model can define when to call them and how to call them in order to execute the solution to the question being asked. A Tool could be Search, Mail, Calculator, ..., some program code, external API

3. Memory: Process Steps of the reasoning phase or History of Conversation

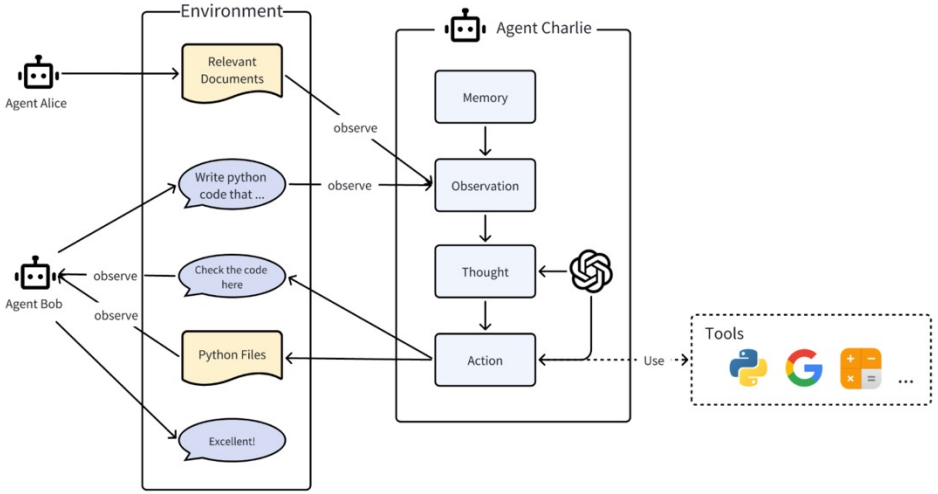
ReAct Pattern:



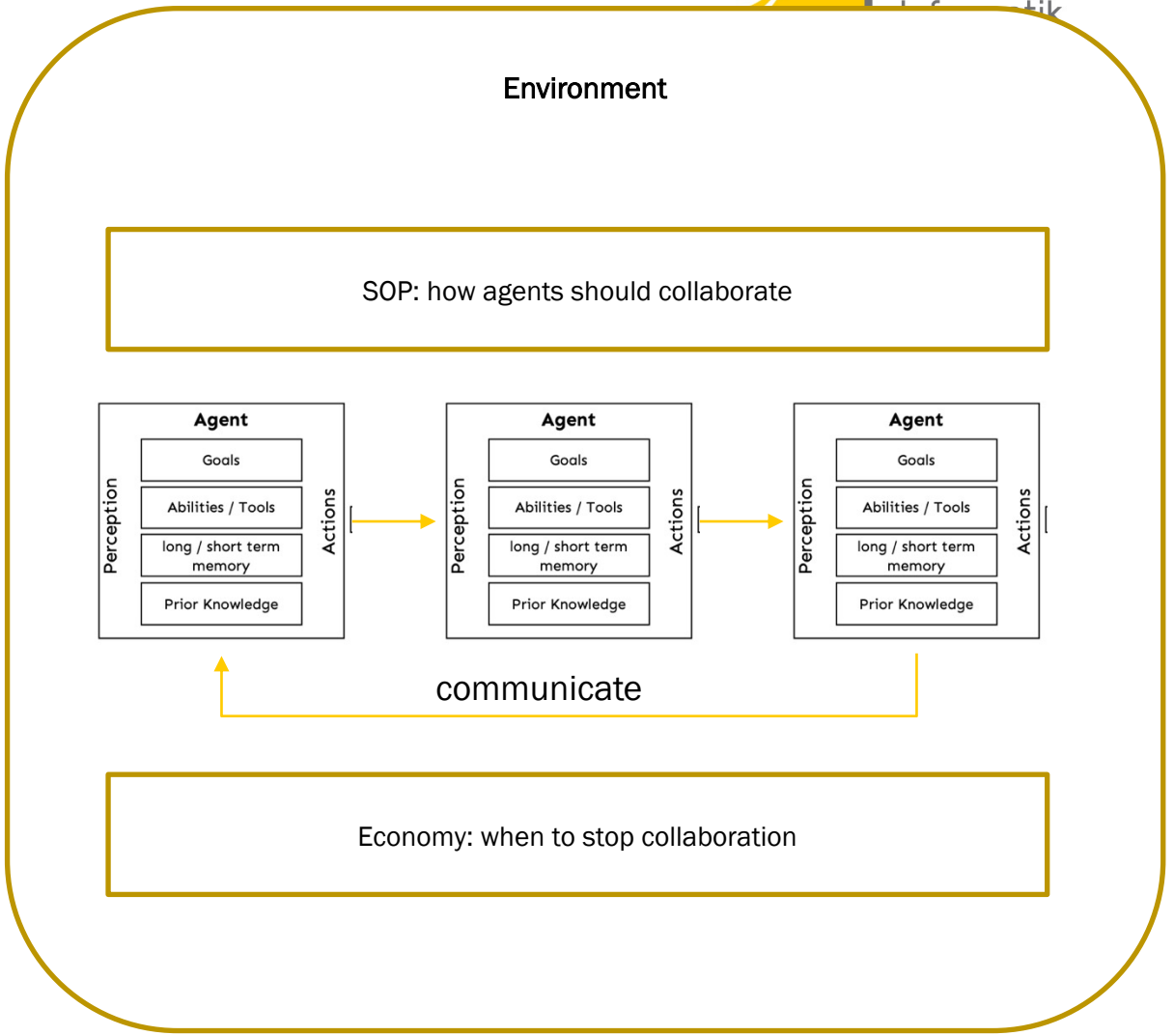
Compound AI Systems

Multi Agent Problem Solver

A MultiAgent System can be thought of as a society of agents.



MultiAgent = Agents + Environment + Standard Operating Procedure (SOP) + Communication + Economy

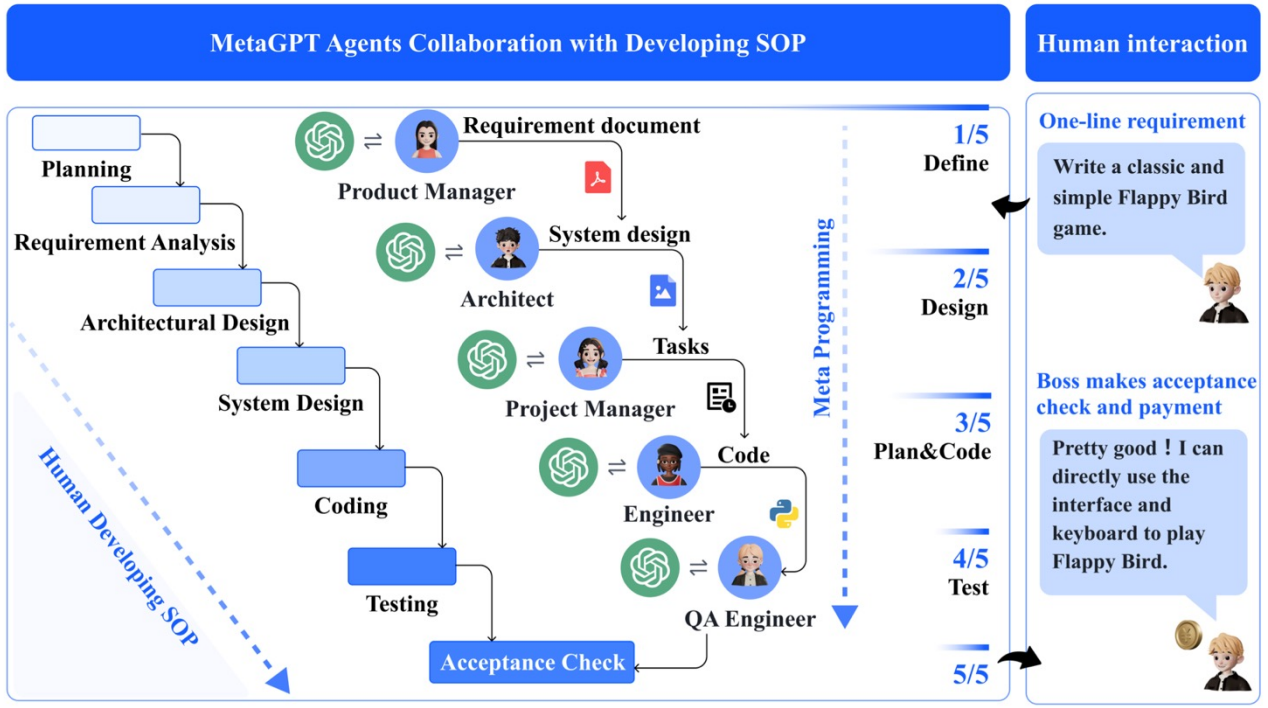


Compound AI Systems

MetaGPT

Core Ideas:

- Meta Programming: MetaGPT is a meta-programming technology that utilizes SOPs to coordinate LLM-based multi-agent systems.
- SOPs Integration: The authors argue that the decomposition of human work experience as defined in corporate standard operating procedures (SOP) might be the way forward.
- Role-centric Collaboration: A shift from generic to specific agents using detailed descriptors and role-specific prompts.



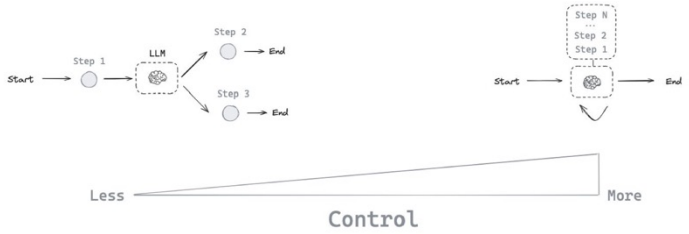
See: METAGPT: META PROGRAMMING FOR A MULTI-AGENT COLLABORATIVE FRAMEWORK - https://arxiv.org/pdf/2308.00352

Compound AI Systems

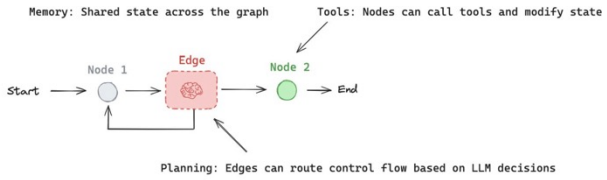
LangGraph

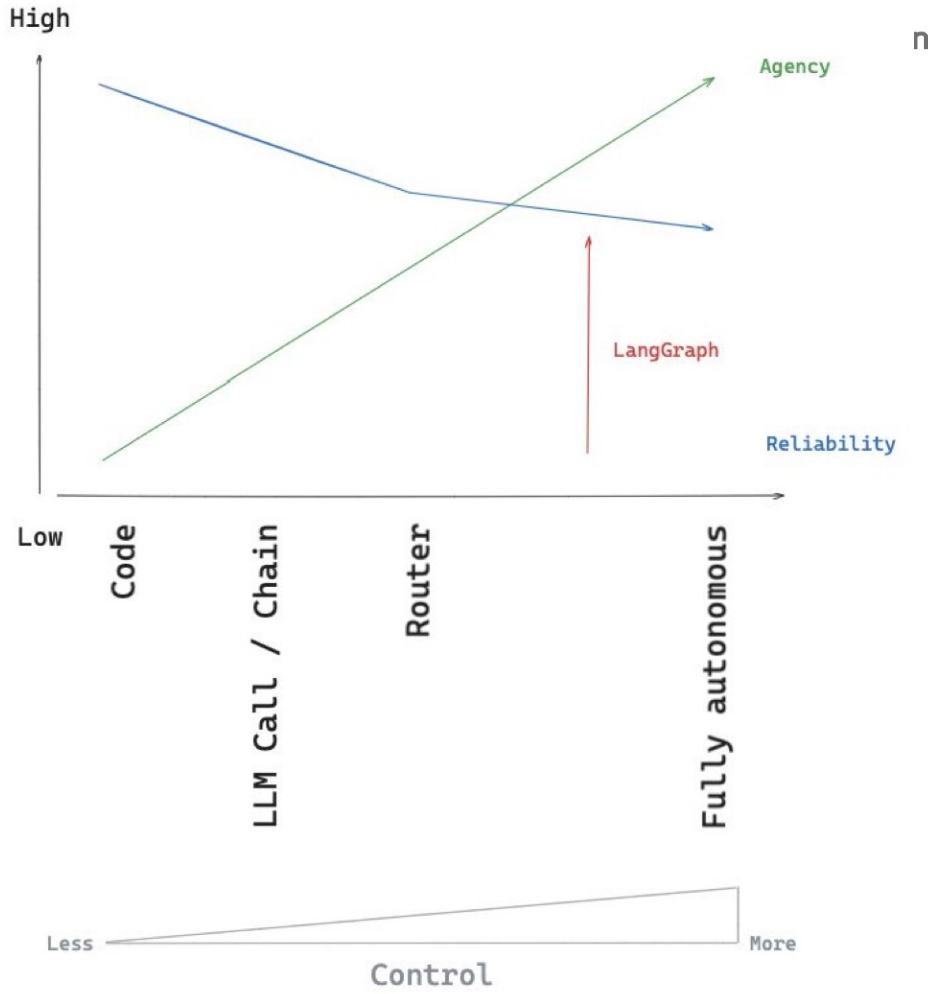
Core Ideas:

- LangGraph is an orchestration framework for complex agentic systems.
- An Agen… Router … Fully Autonomous … an LLM



- LangGra… …espect to reliabili… …ed as graphs





See: https://files.cdn.thinkific.com/file_uploads/967498/attachments/ecd/3cc/6d3/LangChain_Academy_-_Introduction_to_LangGraph_-_Motivation.pdf

Compound AI Systems

Comparing some Multi Agent Problem Solver Framworks

Informatik
Hochschule Reutlingen

Framework	
AutoGen https://microsoft.github.io/autogen/	Robust framework for complex, large-scale LLM applications that integrate multiple agents, tools, and human feedback. It excels in environments demanding dynamic, customizable agent interactions across various application domains.
MetaGPT https://www.deepwisdom.ai/	Predefined complex multi-agent interactions and behaviors. Ideal for network-heavy asynchronous operations and projects that demand advanced collaboration capabilities without heavy customization.
CrewAI https://www.crewai.com/	Production-ready applications where structured task delegation and clear, reliable execution are crucial, and where baked-in framework analytics are a non-issue.
LangGraph https://python.langchain.com/docs/langgraph/	Handling complex task interdependencies. Graph-based approach good for visualizing task interdependencies and agent relationships.
AutoGPT https://autogpt.net/	Auto-GPT is built on the same technology like ChatGPT. The primary difference between the two is that Auto-GPT can function autonomously without the need for human agents, whereas ChatGPT requires human prompts to operate.